

Virtual Memory

— Chapter 9

2023年12月

叶冠华

g.ye@bupt.edu.cn

School of Computer Science (National Pilot Software Engineering School)



北京邮电大学

Review

- 死锁
- 定义、形式化描述、起因
- 死锁产生的必要条件： Mutual exclusion, Hold and wait, No preemption, Circular wait
- 死锁的处理策略：
 - Deadlock prevention： 破坏4个必要条件中的一个或几个
 - Deadlock avoidance： Resource-allocation graph algorithm, Banker's Algorithm
 - Deadlock detection and recovery

Review

- 内存管理
- 地址: symbolic address, logical address, physical address
- 地址重定位(address binding/relocating/mapping/translation)
- 内存管理单元(MMU): Logical address -> Physical address, Memory protection, TLB
- $\text{Physical address} = \text{Logical address} + (\text{Partition/Segment No.} \rightarrow \text{查表} \rightarrow \text{base})$
 - 固定分区(MFT): Fixed Partition Table, Internal Fragmentation
 - 动态分区(MVT): Partition Table(First-fit, Best-fit, Worst-fit), External Fragmentation
 - 分段 (Segmentation): Segmentation Table, Smaller External Fragmentation
- $\text{Physical address} = (\text{Logical address取高位} \rightarrow \text{查表} \rightarrow \text{Frame No.}) \oplus (\text{Logical address取低位} \rightarrow \text{Offset})$
 - 分页 (Page): (Multi-level) Page Table, Smaller Internal Fragmentation
- MFT/ MVT: 一个进程使用一个区; Segmentation/ Page: 一个进程使用多个段/页

Introduction

MOV Y, X

X in page_i, Y in page_j,
but only page_i is loaded in memory.

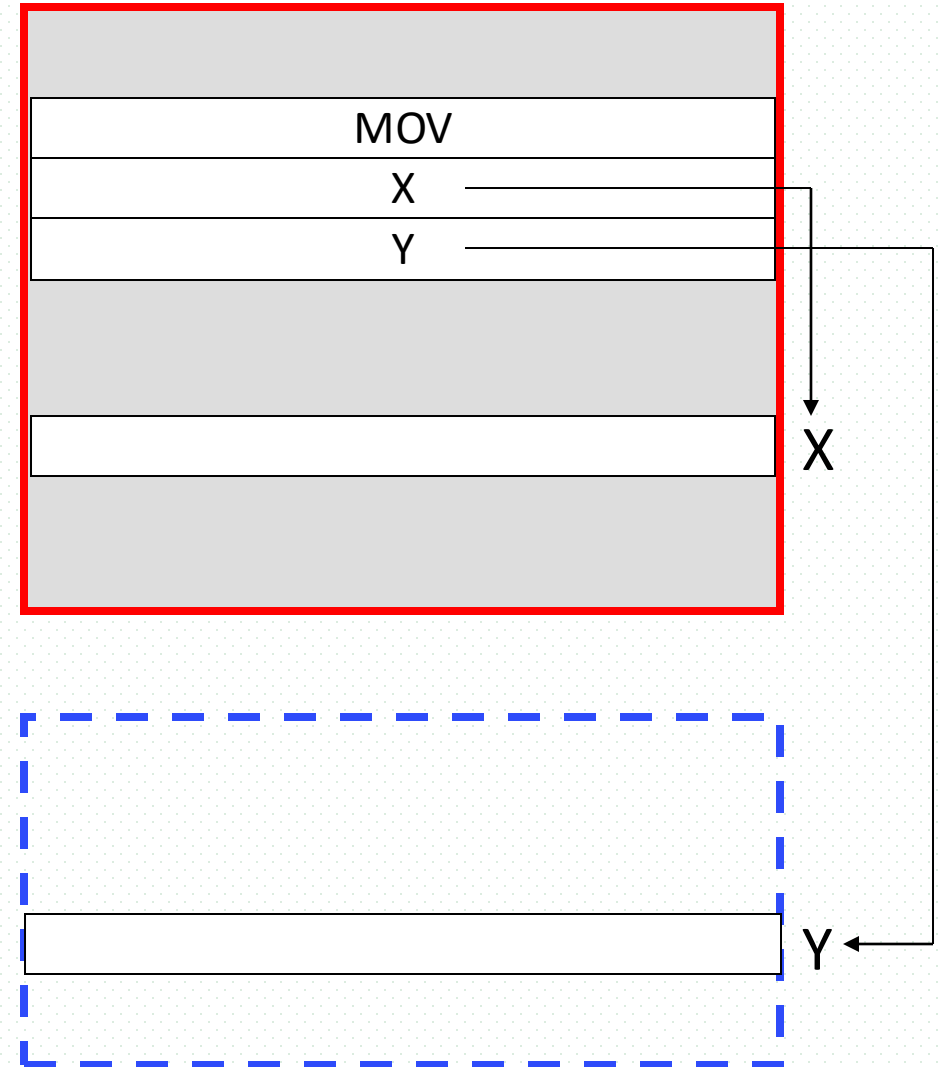
MOV executing:

- (1) 取指令, fetch MOV in page_i
- (2) 译码, decode
- (3) 取操作数X, access address X in memory, success
- (4) 取操作数Y, access address Y in memory, fail, since page_j is not in memory

logical
address
space

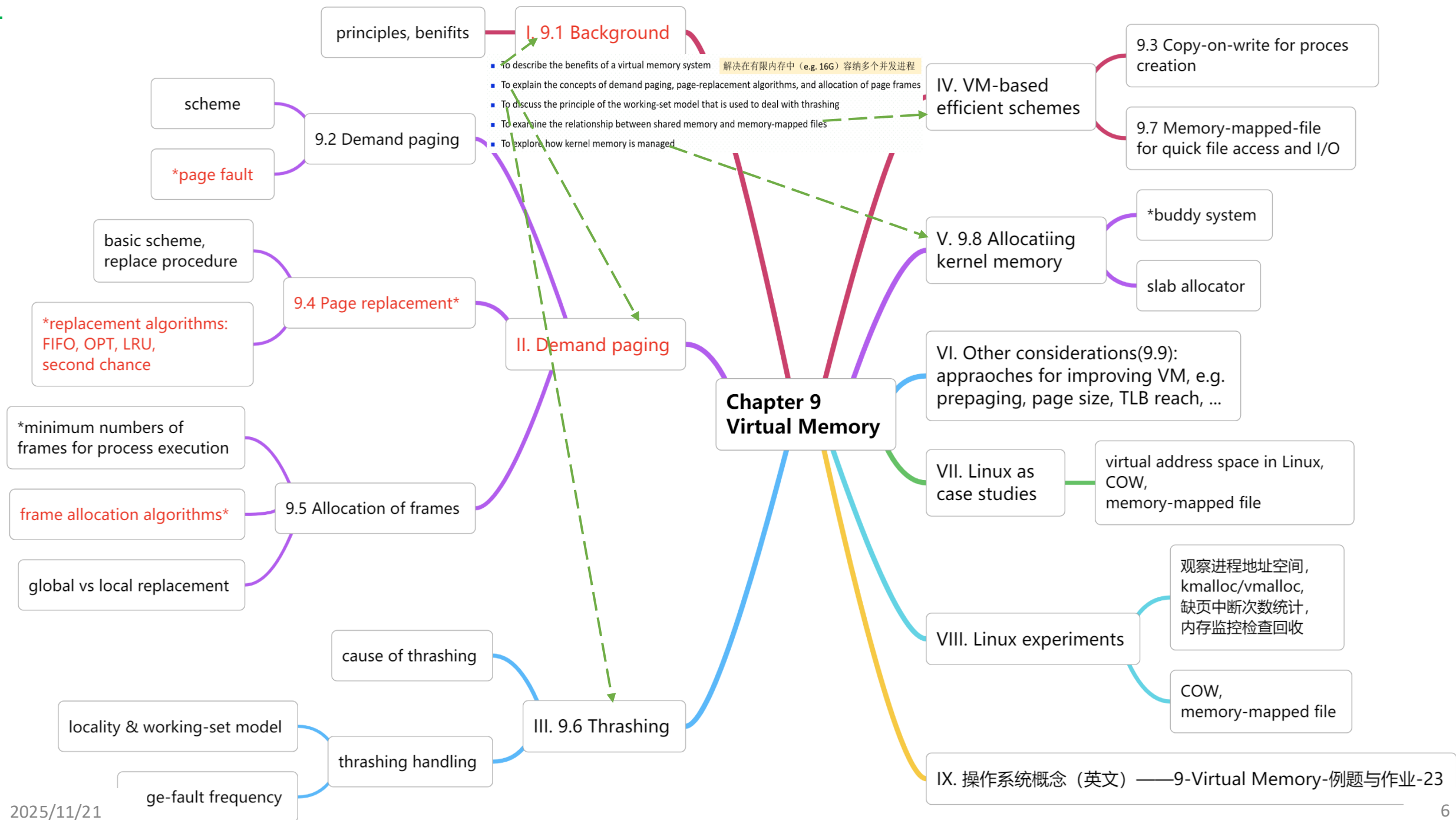
page_i
in memory

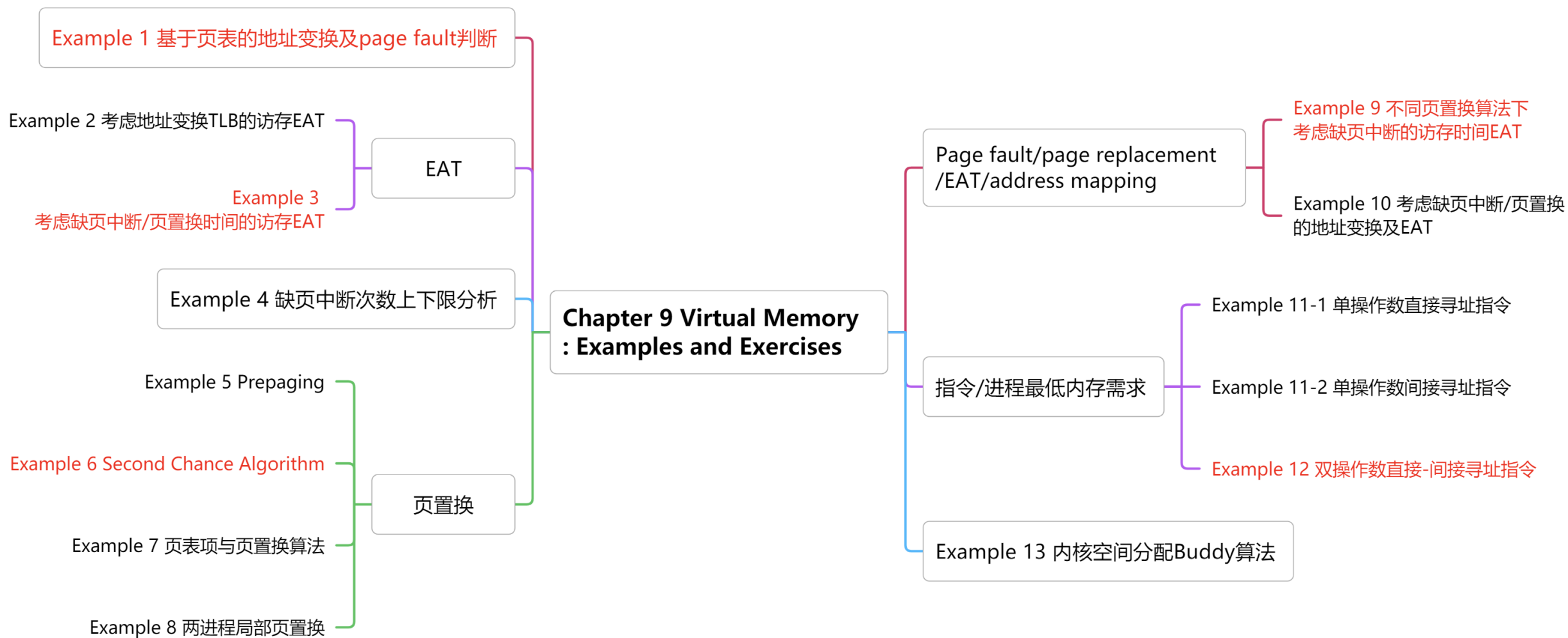
page_j
not in
memory



Introduction

- For executing processes, the instructions being executing must be in physical memory
 - ▣ memory mechanisms in Chapter 8 , such as contiguous allocation and noncontiguous allocation(e.g. paging) meet this strict requirement by loading all the process into memory once at a time before the process begins to run
 - ▣ however, in the memory mechanisms mentioned above, logical address space is limited by the memory size, or by the physical address space
 - e.g. 32-bit logical address, $2^{32}=4\text{GB}$ logical address space
- Virtual memory (or VM, 虚拟内存管理) is a memory management scheme that
 - ▣ allows the logical address space of processes to be larger than the available physical address spaces !
 - ▣ thus, enables the processes which may not be completely in memory to be executed
 - only parts of the process' codes and data (e.g., parts of pages, parts of segmentations) reside in memory





Terminology-第9章

virtual memory	虚拟内存(管理)
demand-paging, page fault, page fault trap, page-fault rate	请求页式管理, 缺页/页错误, 缺页异常/缺页中断, 缺页率
page-replacement, (memory) reference, reference string, local replacement, global replacement, Belady's Anomaly, OPT, LRU, Second chance algorithm, Modify/dirty bit,	页置换, (内存)访问, 访问串, 局部置换, 全局置换, 贝拉迪异常, 最优置换, 最近最少使用页置换, 二次机会算法, 修改位/脏位

working set, thrashing, temporal locality, spatial locality	工作集, (页面)颠簸/抖动, 时间局部性, 空间局部性
copy-on-write	写拷贝
memory-mapping file	内存映射文件
Prepaging	预分页

Introduction

- In multiprogramming VM systems, four issues must be addressed on
 - ▣ **WHEN?** When a process is to execute an instruction or to access a data item that is not in memory, OS loads the page in which the instruction or the item exists by means of **demand-paging (请求页式管理/请求调页)** (§9.2).
 - ▣ **WHERE?** OS uses **frame-allocation policies/algorithms** (§9.5) to allocate free frames for the loaded pages, and a frame can be shared by more than one process in the system.
 - ▣ **WHO?** If there are not enough free frames for allocation, OS uses **page-replacement** (§9.4) policies/algorithms to find some free frames
 - ▣ **HOW?** Some measures based on **working set** must be taken to prevent **thrashing(抖动)** to guarantee system performance (§9.6)

Introduction

- In addition
 - ▣ copy-on-write (§9.3) supports rapid process creation in VM system
 - by allowing the parent and children processes to initially share the same pages
 - ▣ memory-mapping file (§9.7) simplifies and speed up files access
 - by treating reads and writes to files as routine memory access

9.1 Background: Virtual memory

- Logical address space, physical address space
 - ▣ logical address, usually start at address 0, contiguous addresses until end of space
 - ▣ physical memory is organized in page frames, and physical address space is limited by the size of main memory
 - ▣ MMU must map the logical address to the physical address
- The system/maximum logical address space is dependent not only on the length of machine instruction, also on
 - ▣ other factors in practical systems, i.e. Intel x86/Linux, 48-bit logical address, 40-bit physical address
- The process' physical address space may be less than the system logical space
 - ▣ e.g. Fig. 9.1

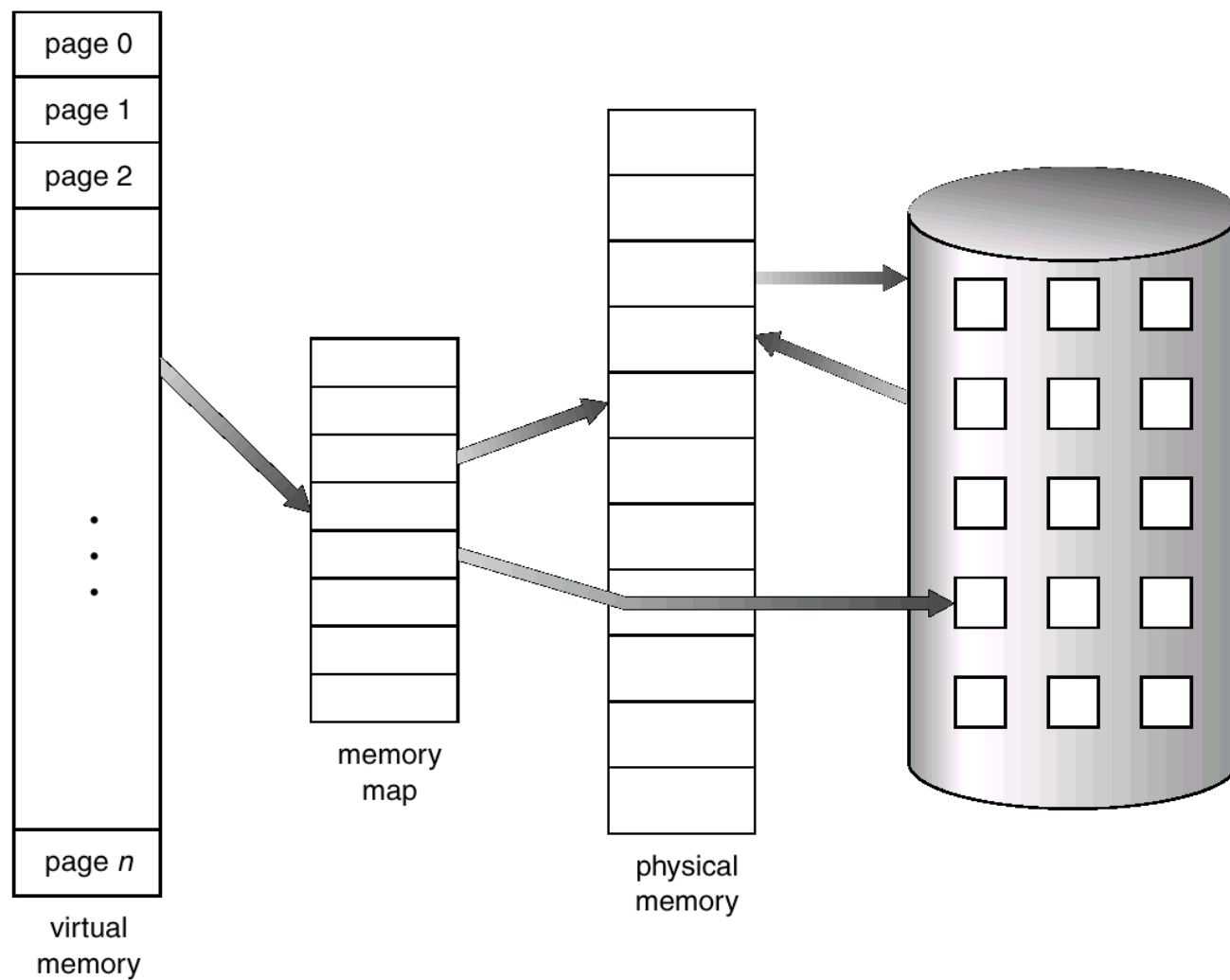


Fig.9.1. Virtual memory that is larger than physical memory

Background

- Code needs to be in memory to execute, but entire program rarely used—程序部分装载进内存
 - ▣ error code, unusual routines, large data structures are temporally stored in secondary storage, e.g. disk
- Why
 - ▣ during execute a program, entire program code not needed at same time
- Consider ability to execute **partially-loaded** program
 - ▣ program no longer constrained by limits of physical memory
 - ▣ each program takes less memory while running -> more programs run at the same time
 - increased CPU utilization and throughput with no increase in response time or turnaround time
 - ▣ less I/O needed to load or swap programs into memory -> each user program runs faster

9.2 Demand Paging

■ Principles

- 进程逻辑空间按页page进行组织，但对用户是非可见的
- 进程开始执行之前，只将逻辑地址空间中当前经常反复执行和调用部分所在的pages调入内存, 而其它pages则在进程执行过程中根据需要动态装入
 - dynamic loading
- 当进程执行某条指令时，如果该指令不在内存,或执行某条指令过程中需要访问的其它数据或指令时不在内存中时，则发生缺页错误 (page fault)
- OS通过缺页中断 (page fault trap) 从外存 (e.g. pagefile in Windows) 中将需要访问的指令或数据所在page调入内存
- 地址变换:
 - 与paging机制类似，系统根据page#查出相应frame#，根据frame#与页内偏移(offset)合并，得到实际物理地址
- 当缺页中断将某个page调入内存时，如果没有free frame，可通过page replacement为调入的page分配空闲内存
- 采用与paging类似机制，实现存储保护与共享

Demand Paging

- Key schemes for demand paging
 - ▣ **page fault trap**/缺页中断、 缺页异常
 - ▣ **page replacement**/页置换
 - ▣ frame allocation /frame分配
- Demand paging versus swapping
 - ▣ swapping
 - *swap in/out* among processes
 - ▣ demand paging
 - *swap in/out* among pages of one or more processes

Hardware Supports

- MMU (e.g. the relocation register, Table Walk Unit in ARM CPU) is needed for address translation
- The page table is needed, which contains
 - ▣ page# → frame#, for address binding
 - ▣ valid-invalid bit with each page table entry to determine whether pages accessed by processes are
 - legal or illegal, i.e. in process's logical address space or not
 - in memory or in secondary storage
 - ▣ values of **Valid-Invalid Bit** indicating the associated page
 - 1 ⇒ legal and in-memory,
 - 0 ⇒ illegal **or** legal but currently on the disk
 - ▣ initially valid–invalid is set to 0 on all entries
- Swapping space in secondary memory is needed to store the pages that are not in main memory, (e.g. **pagefile** in Windows)

Hardware Supports

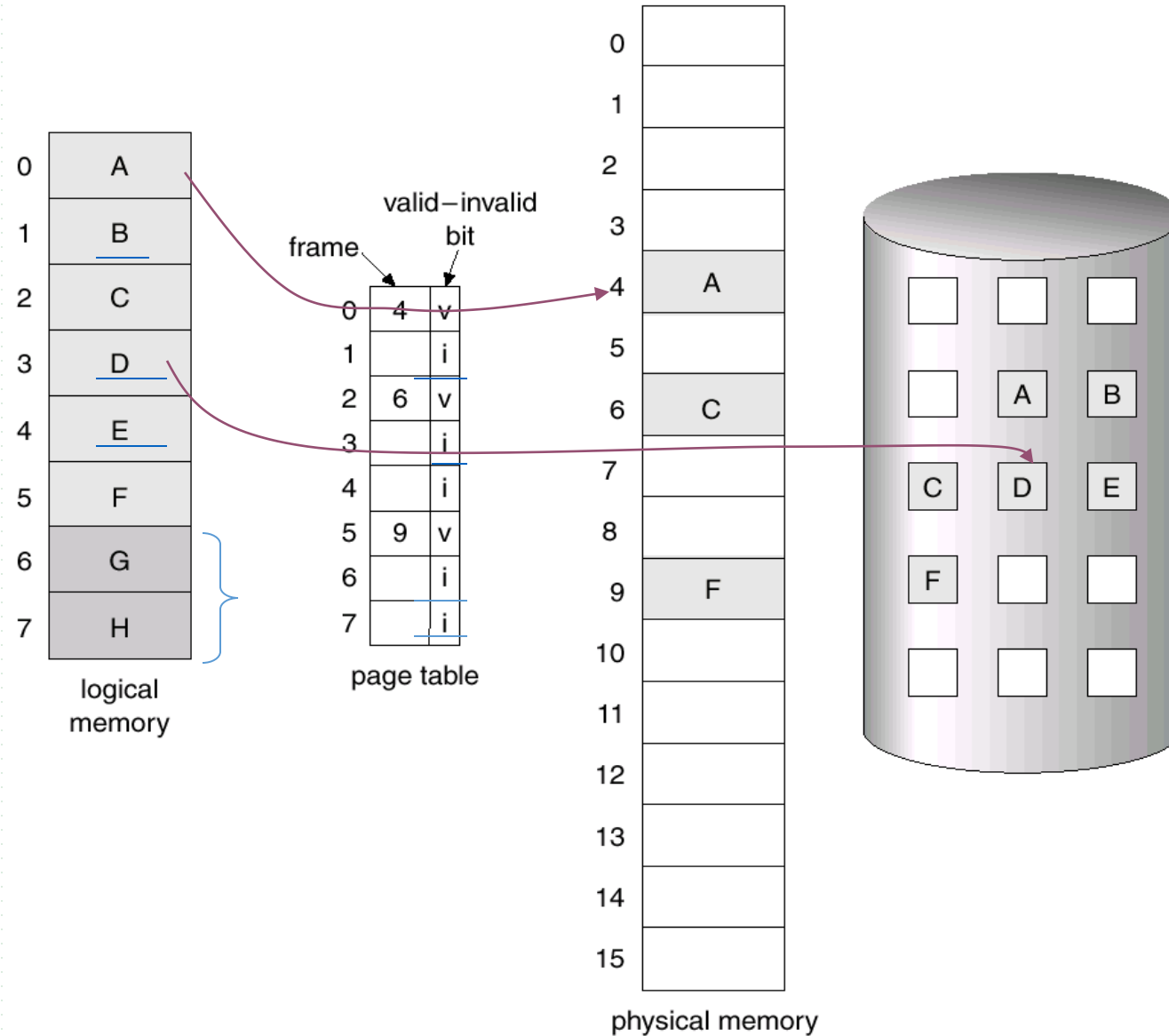


Fig.9.5 Page table when some pages are not in main memory

Working Set

■ Working Set

- 一个进程可以直接访问到、不引发缺页中断的物理内存
- e.g. page 0/2/5, frame 4/6/9 in the previous slide ▶
- 包括：专用工作集的内存数量 + 共享工作集的内存数量

■ 专用/私人(private)工作集

- 工作集的一个子集
- 进程正在使用、且无法与其他进程共享的内存数量
 - private code or data specific to a process

■ 共享工作集

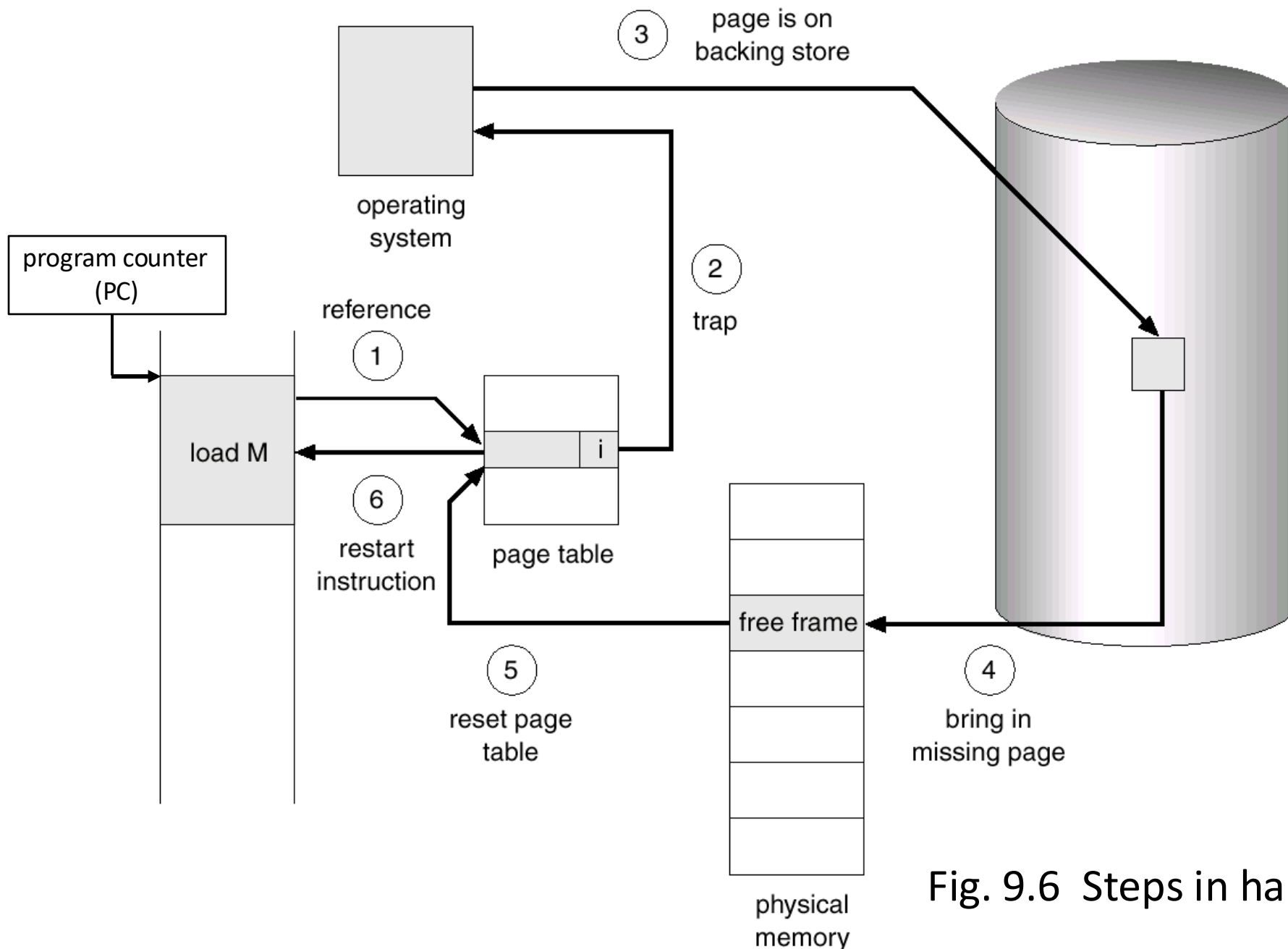
- 进程正在使用、且可以由其他进程共享的内存
 - e.g. C++ library functions shared by processes

Process Executing and Page Fault Handling in Fig. 9.6

- While the process accesses the pages that are memory resident, i.e. **Valid-Invalid Bit** =1, execution proceeds normally
- Otherwise, that is **Valid-Invalid Bit** =0, a **page fault (缺页)** occurs, and the system handles this fault by **page fault trap (缺页中断/异常)** as follows, *refer to Fig.9.6* 
 - ▣ step1. determine the reference (访问地址) is legal or illegal, on the basis of *an internal table* in PCB
 - ▣ step2. if the reference is invalid,
then terminated the process
else **go to** the following steps to *swap* the demanded page *in*

Process Executing and Page Fault Handling

- ❑ step3. get a free frame for the page, from the free-frame list or by page replacement
- ❑ step4. schedule a disk operation to read desired page into the newly allocated free frame
- ❑ step5. modify the internal table kept with the process and the page table to indicate that the page is in memory (reset validation bit in the page table to 1)
- ❑ step6. restart the instruction interrupted by page fault trap
- Pure demand paging
 - ❑ start executing a process with no page in memory
 - ❑ never bring a page into memory until it is required
- **Locality of reference** (as described in 9.6.1) prevents the scenario in which *multiple page faults per instruction* occur



addressing for operand M:
M/page size $\rightarrow$ page#
e.g.
790/4096 $\rightarrow$ page 0
4100/4096 $\rightarrow$ page 1

Fig. 9.6 Steps in handling a page fault

Performance: EAT in demand paging

- Performance criteria

- ▣ Effective Access Time, EAT

- memory access time plus page fault handling time

此处EAT与8.5.2基于页表/TLB的地址变换中的EAT不一样

- Page Fault Rate $0 \leq p \leq 1.0$

- ▣ if $p = 0$ no page faults
 - ▣ if $p = 1$, every reference is a fault

- Effective Access Time (EAT)

$$\text{EAT} = (1 - p) \times \text{memory access}$$

+ p (page fault overhead

+ swap page out

+ swap page in

+ restart overhead

)

} page-fault service

Example One

- Memory access time = 200 nanoseconds
- Average page-fault service time = 8 milliseconds
- $$\begin{aligned} \text{EAT} &= (1 - p) \times 200 + p (8 \text{ milliseconds}) \\ &= (1 - p) \times 200 + p \times 8,000,000 \\ &= 200 + p \times 7,999,800 \end{aligned}$$
- EAT is directly *proportional to the* page fault rate
- If one access out of 1,000 causes a page fault, i.e. $p=0.001$, then
$$\text{EAT} = 8.2 \text{ microseconds.}$$

This is a slowdown by a factor of 40!!

例题

- 1 下列因素影响请求分页系统有效 (平均) 访存时间的是:
 - I. 缺页率
 - II. 磁盘读写时间
 - III. 内存访问时间
 - IV. 执行缺页处理程序的 CPU 时间

A. II, III B. I, IV C. I, III, IV D. I, II, III, IV

- 2 对于采用虚拟内存管理方式的系统，下列关于进程虚拟地址空间的叙述中，错误的是()
 - ❑ A. 每个进程都有自己独立的虚拟地址空间
 - ❑ B. C语言中 malloc () 函数返回的是虚拟地址
 - ❑ C. 进程对数据段和代码段可以有不同的访问权限
 - ❑ D. 虚拟地址的大小由主存和硬盘的大小决定

例题

- In a demand paging system, the page size is 1024 bytes, and the page table of a process is as follows, would the following virtual addresses (assuming use decimal values) result in a page fault ? and why ?

- ▣ (1) 2500
- ▣ (2) 5100

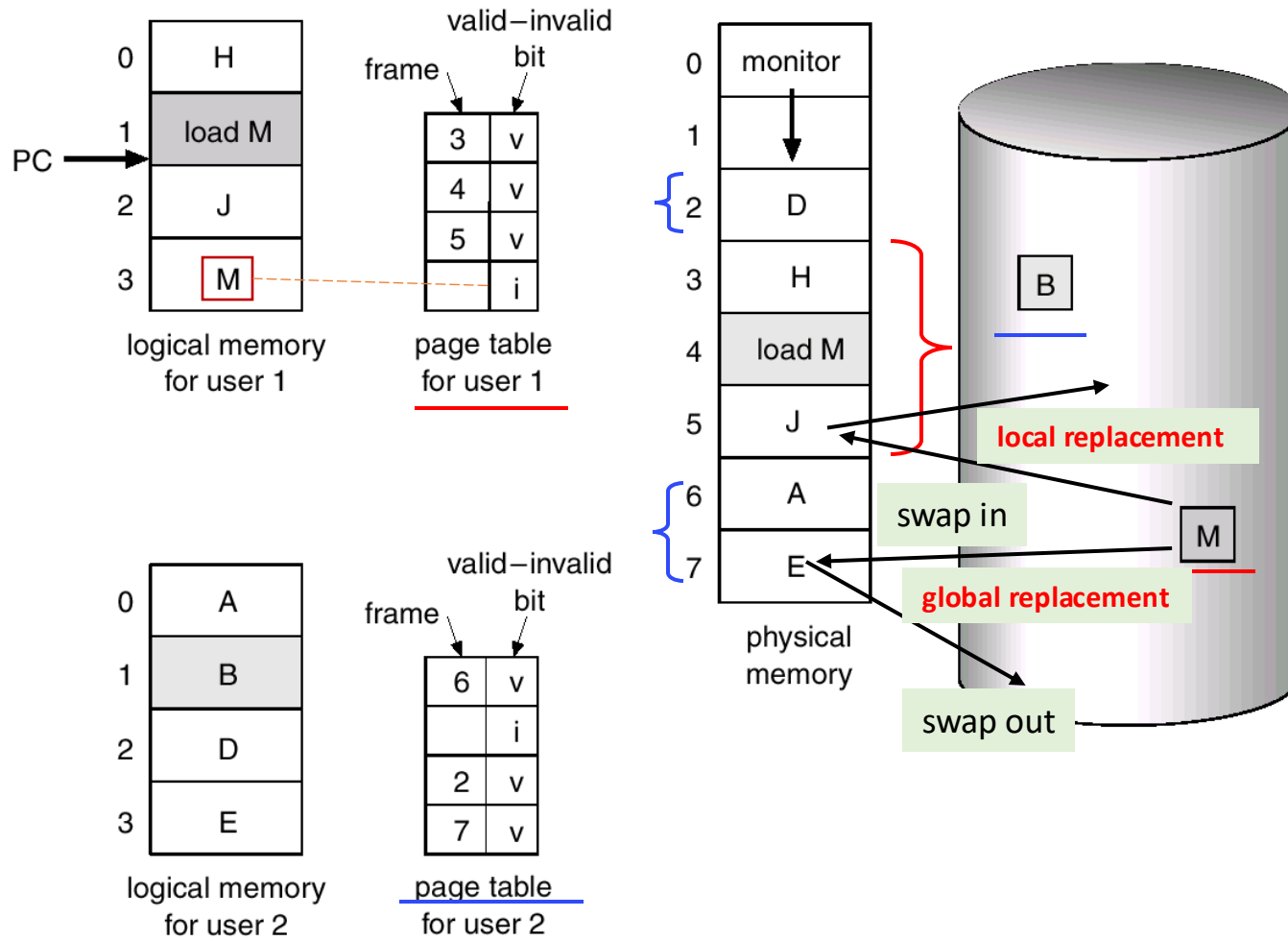
P#	valid/invalid	frame number
0	i	20
1	v	15
2	v	20
3	v	29
4	i	80
5	i	53

9.4 Page Replacement

- Basic Scheme
- Page Replacement Algorithms
 - ▣ FIFO
 - ▣ OPT
 - ▣ LRU
 - ▣ Second chance algorithm

Basic Scheme

- **Over-allocation** results in page fault, and *page replacement* is needed
 - ▣ over-allocation of memory
 - numbers of pages of all processes in the system exceed the size of physical memory
 - ▣ e.g. Fig. 9.9
- By page-fault service and page replacement, a process with large virtual address size can run on a smaller physical memory



- total size of P_1 and P_2 : 8 pages
- available physical memory for P_1 and P_2 : frame₂ ~ frame₇
- if P_1 executes loads M in page₁, a page-fault trap occurs, but there is no free frame for holding **M** in page₃

Fig.9.9 Need for page replacement

Basic Scheme

- Page replacement procedure:
 - ▣ step1. find the location of the desired page on the disk
 - ▣ step2. find a free frame
 - if there is a free frame, use it.
 - if there is no free frame, use a **page replacement algorithm** to select a **victim frame**
 - swap out the victim page to the disk; Update the page table.
 - ▣ step3. read the desired page into the (newly) free frame; Update the page table.
 - ▣ step4. restart the process

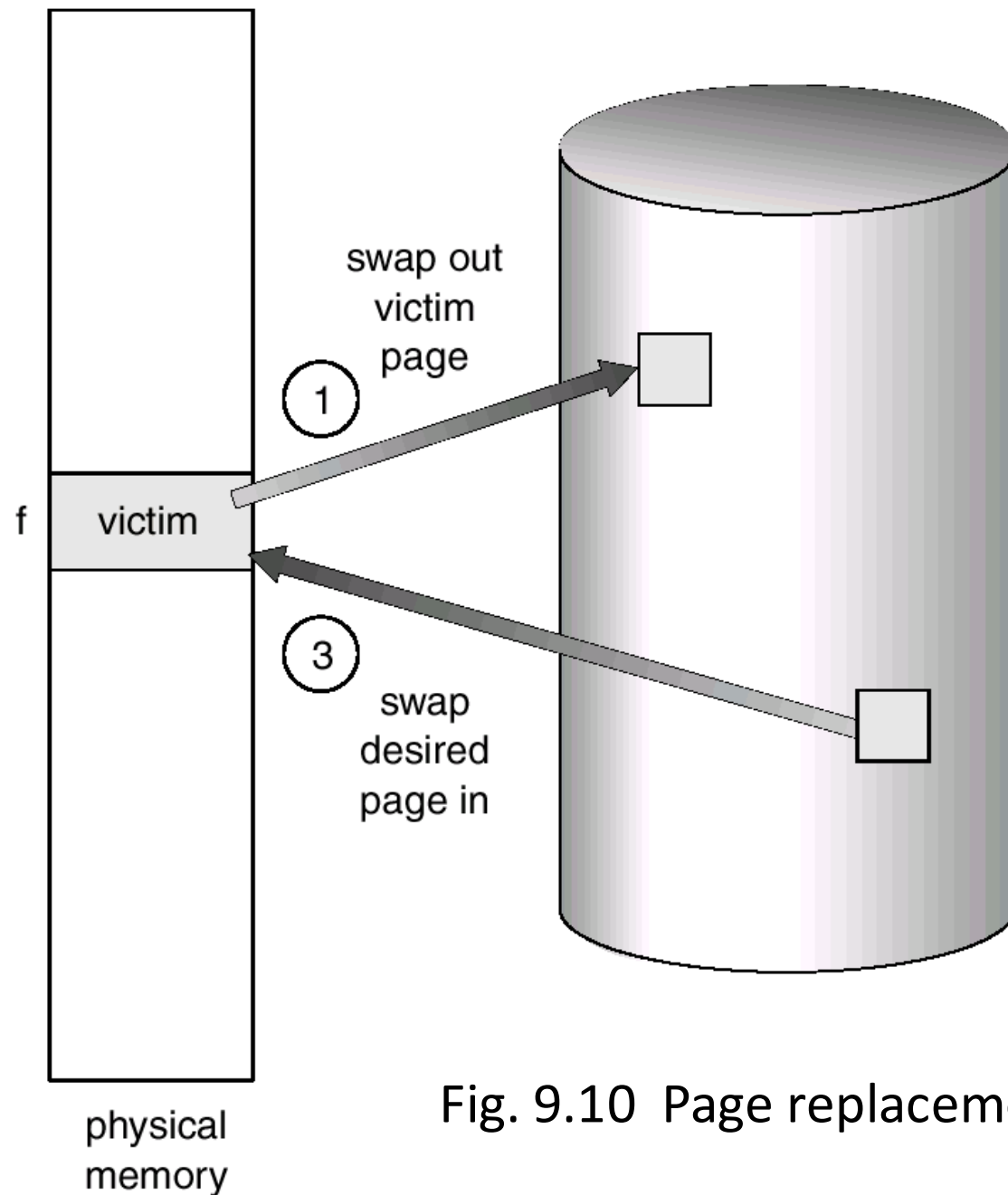
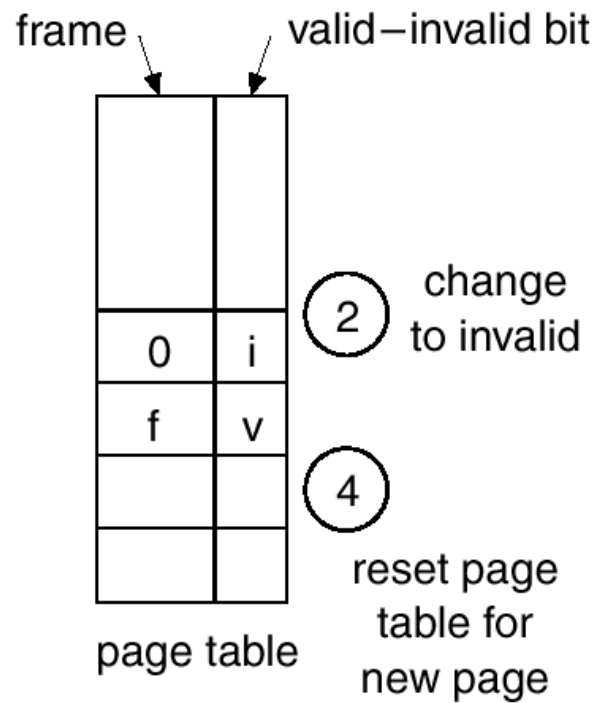


Fig. 9.10 Page replacement

Basic Scheme

- **Modify (dirty) bit** is used to reduce overhead of page transfers
 - ▣ dirty bit, indicating whether or not the contents of the page has been modified
 - 1, modified dirty page
 - 0, not modified, clean page, e.g. pages storing codes/instructions
 - ▣ only modified, i.e. **dirty**  ges in **victim frames** are written/swapped to disk
 - ▣ if a victim page (e.g. a page contains only instructions) is not modified/written between after being loaded and before being replaced, it should not be swapped out to the disk

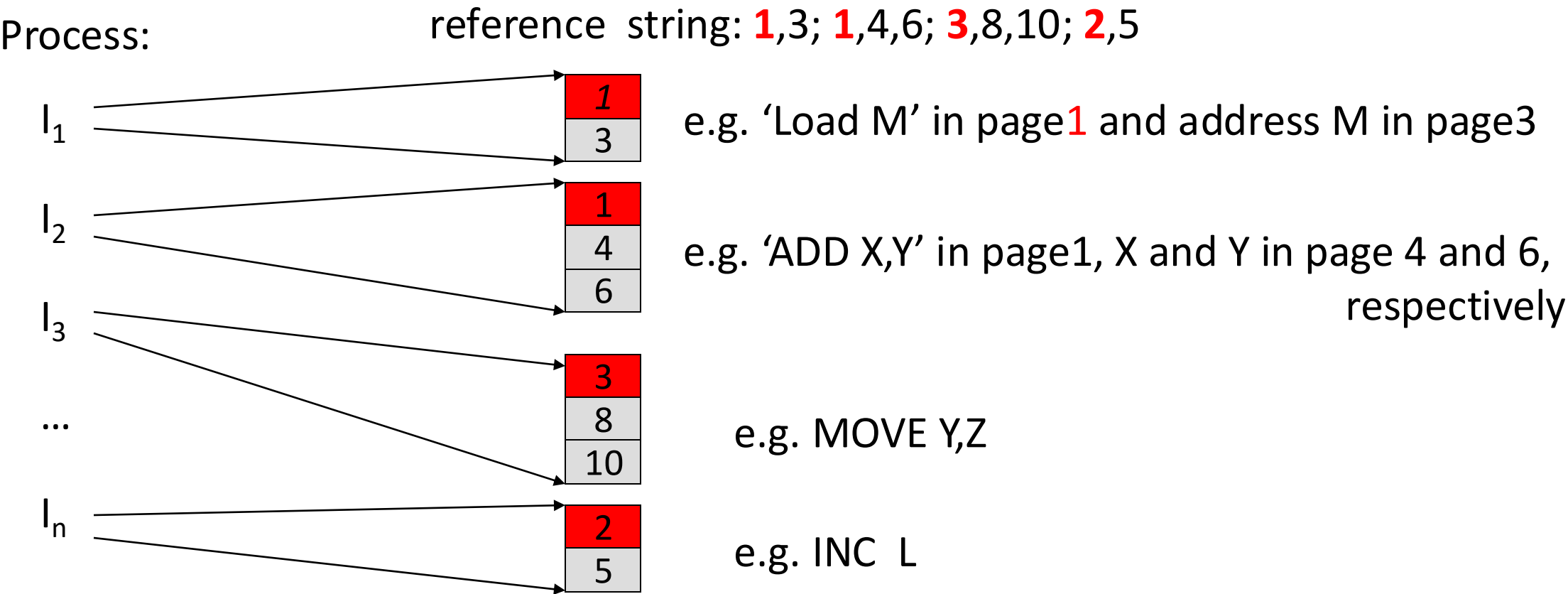
Basic Scheme

- As mentioned above, two strategies are needed for page fault trap in demand-paging
 - ▣ frame allocation algorithms
 - ▣ page replacement algorithms
- Page-fault rate is used as the evaluation criteria for page replacement algorithms
 - ▣ lowest page-fault rate to reduce expensive I/O operations

Basic Scheme

- How to evaluate page replacement algorithms
 - ▣ (memory) **reference**[内存访问/引用]
 - the numbers of the pages accessed, i.e. the address of memory units accessed
 - ▣ evaluate the different algorithms by running it on a particular string of memory references (defined as **reference string**), and then computing the number of page faults on that string

Four instructions executed sequentially: Load M; ADD X, Y; MOVE X, Y; INC L



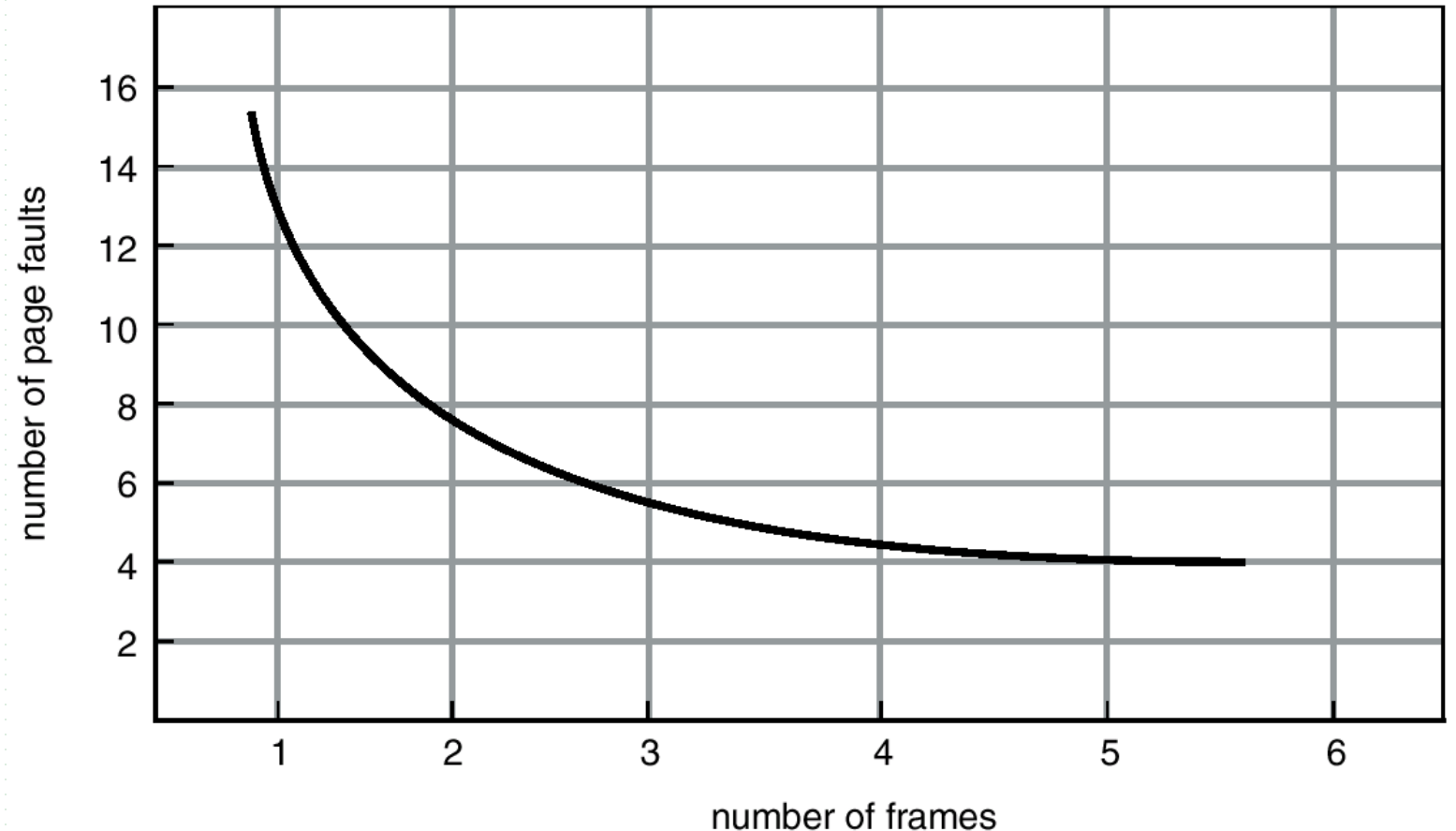
i page i holding an instruction

j page j holding the operands of an instruction

Basic Scheme

- A “good” algorithm and evaluation results
 - ▣ with the number of free frames increases, the number of page faults occurring does not increase
 - similar to that in Fig.9.11, number of page faults vs. number of available frames

Fig.9.11 Page faults vs the number of frames



Basic Scheme

- The test bed for evaluating of the following algorithms

- ▣ reference string: 20 references

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

- ▣ **three** frames available

First-In-First-Out (FIFO)

■ Principles

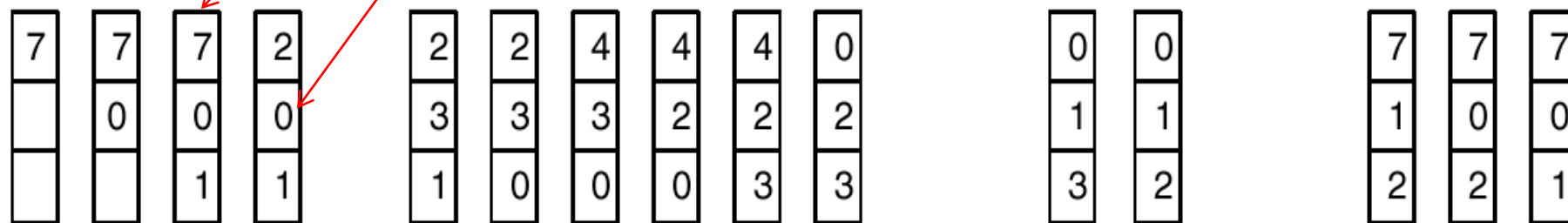
- ❑ the oldest page in memory is replaced
 - age of page: the time that the page is brought into memory and allocated a allocated a frame
- ❑ a FIFO queue is used to hold all pages in memory, and the queue length is the number of available frames in memory
- ❑ when a page is brought into memory, it is inserted into the tail of the queue,
- ❑ the page at the head of the queue, the oldest page in queue, is selected as the victim and replaced

■ Evaluation example, Fig.9.12

- ❑ number of page fault is 15

reference string

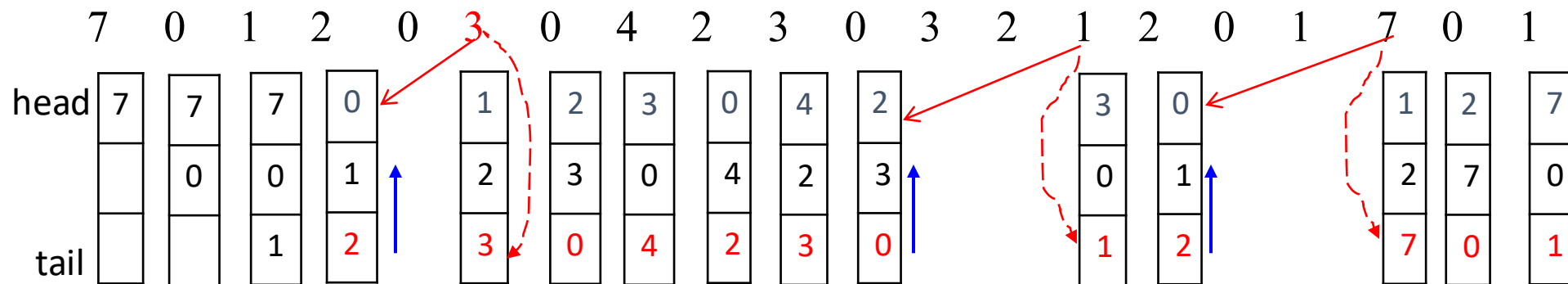
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

Fig. 9.12 FIFO Page Replacement

time

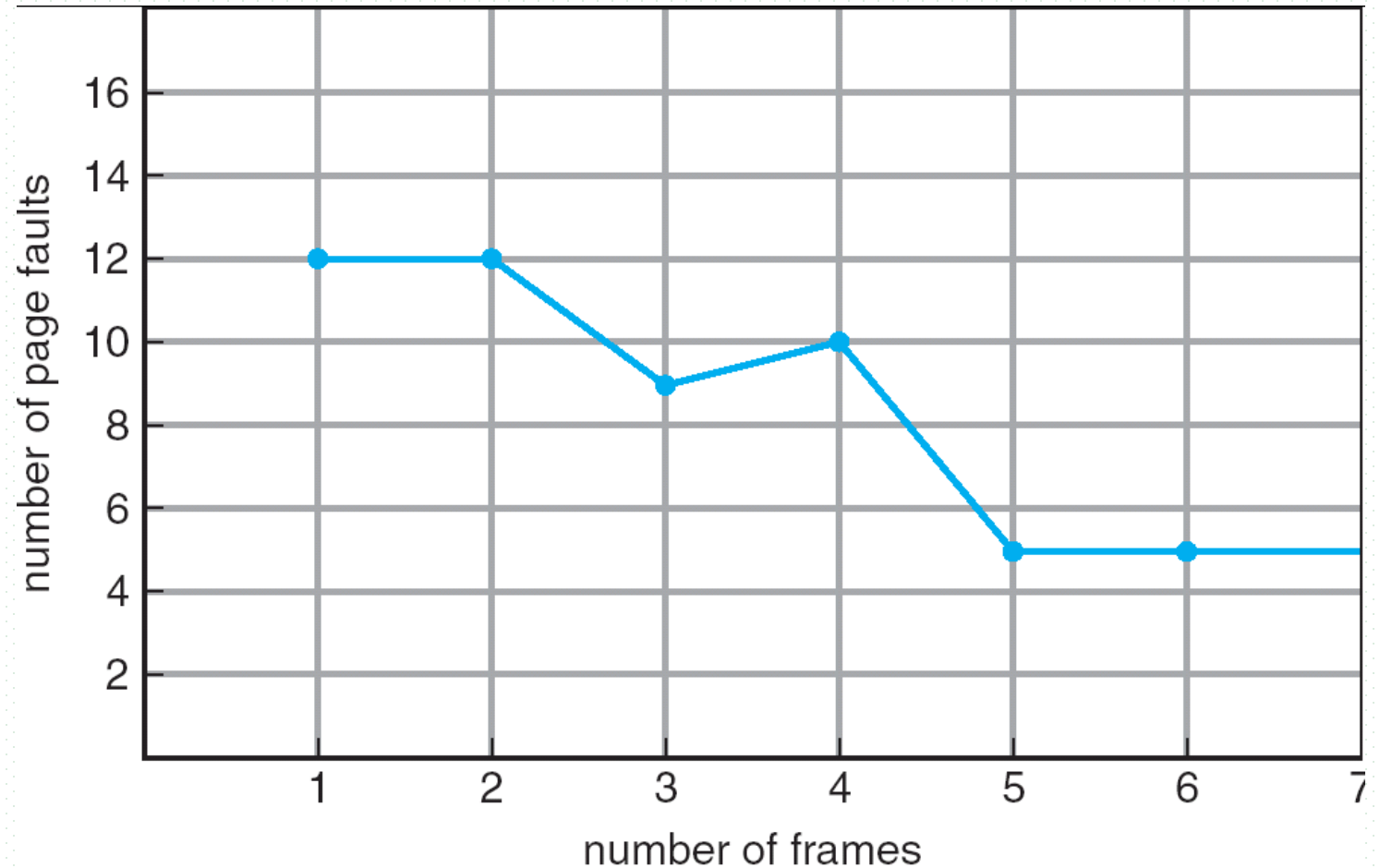


- 替换(swap out)FIFO队列**头部**oldest page
- 新加入(swap in)的page加入队**列尾**

FIFO

- FIFO replacement characteristics/demerits
 - ▣ can not avoid Belady's Anomaly
 - as the number of the frames increases, page faults rate increases

Fig. 9.13 FIFO Illustrating Belady's Anomaly with reference string:
1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5



- Principles

- replace the page that will not be used for the longest period of time
- also known as **OPT** or **MIN**

/* 淘汰在将来再也不会被访问，或最远的将来才被访问的那些页 */

- OPT is **opt**imal, having minimum page fault number, compared with other algorithm
 - never suffering from Belady's Anomaly
- OPT cannot be implemented in practice!
 - how to know which page will no longer be used ?
- As a benchmark or test bed, OPT is used for measuring how well the other algorithms perform

- Evaluation example, Fig. 9.14
 - ▣ the number of page faults is 9

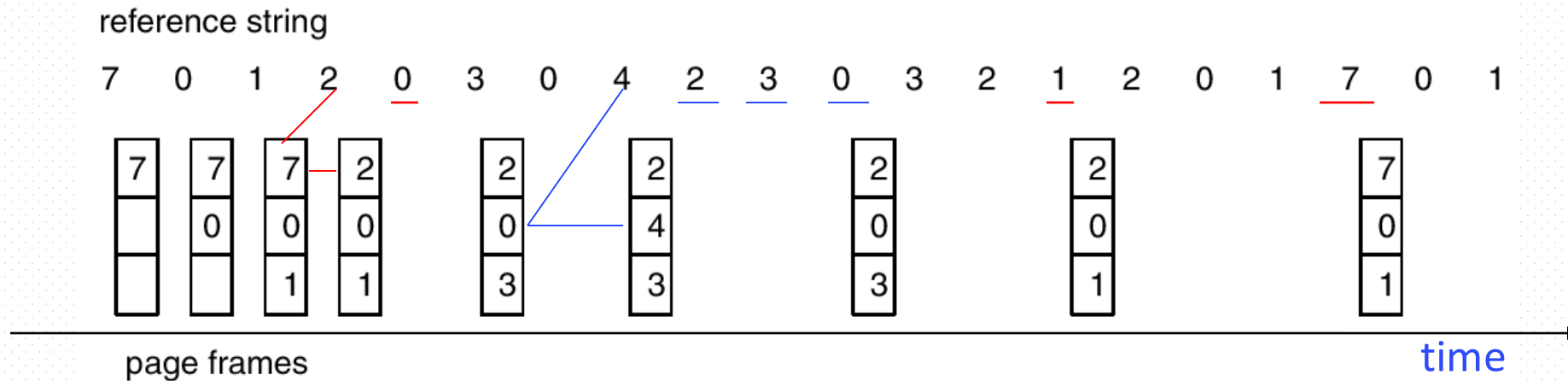


Fig. 9. 14 Optimal Page Replacement

LRU

- Least Recently Used (LRU) Algorithm (最近最少使用算法)

- ▣ use past pattern of page references as a predictor of future references
- ▣ replace the page that has not been used for the longest period of time

/*选择到目前为止，最长时间未被使用的的页；

- 依据：过去一段时间内未被访问（不活跃）的页，在最近的将来最大可能地也不会被访问

LRU

- Evaluation example, Fig. 9.15
 - ▣ the number of page faults is 12

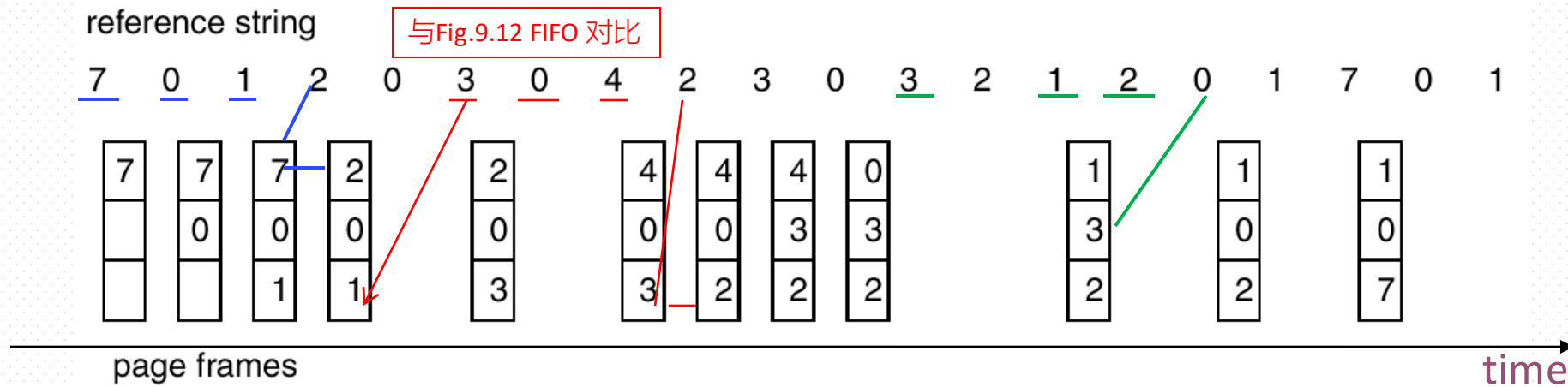


Fig. 9.15 LRU Page Replacement

LRU

- Often used in OS and other areas, e.g. DBMS, with a good performance
 - ▣ e.g. block replacement in DBMS buffer, shown in next slide
- Two schemes for LRU implementation
 - ▣ counter
 - ▣ stack

- Most DBMS replace the block **least recently used** (LRU strategy)
 - // refer to **page replacement** in **Operating System Concepts**, for more detail
 - ▣ Idea behind LRU – **use past pattern of block references as a predictor of future references**
 - ▣ LRU can be bad for some queries
- Queries have well-defined access patterns (such as sequential scans), and a database system can use the information in a user's query to predict future references
- Mixed strategy with hints on replacement strategy provided by the query optimizer is preferable

Transaction data accesses on data item **x** and **y** issued by T_i ,
e.g. *select, insert, delete, update, ...*

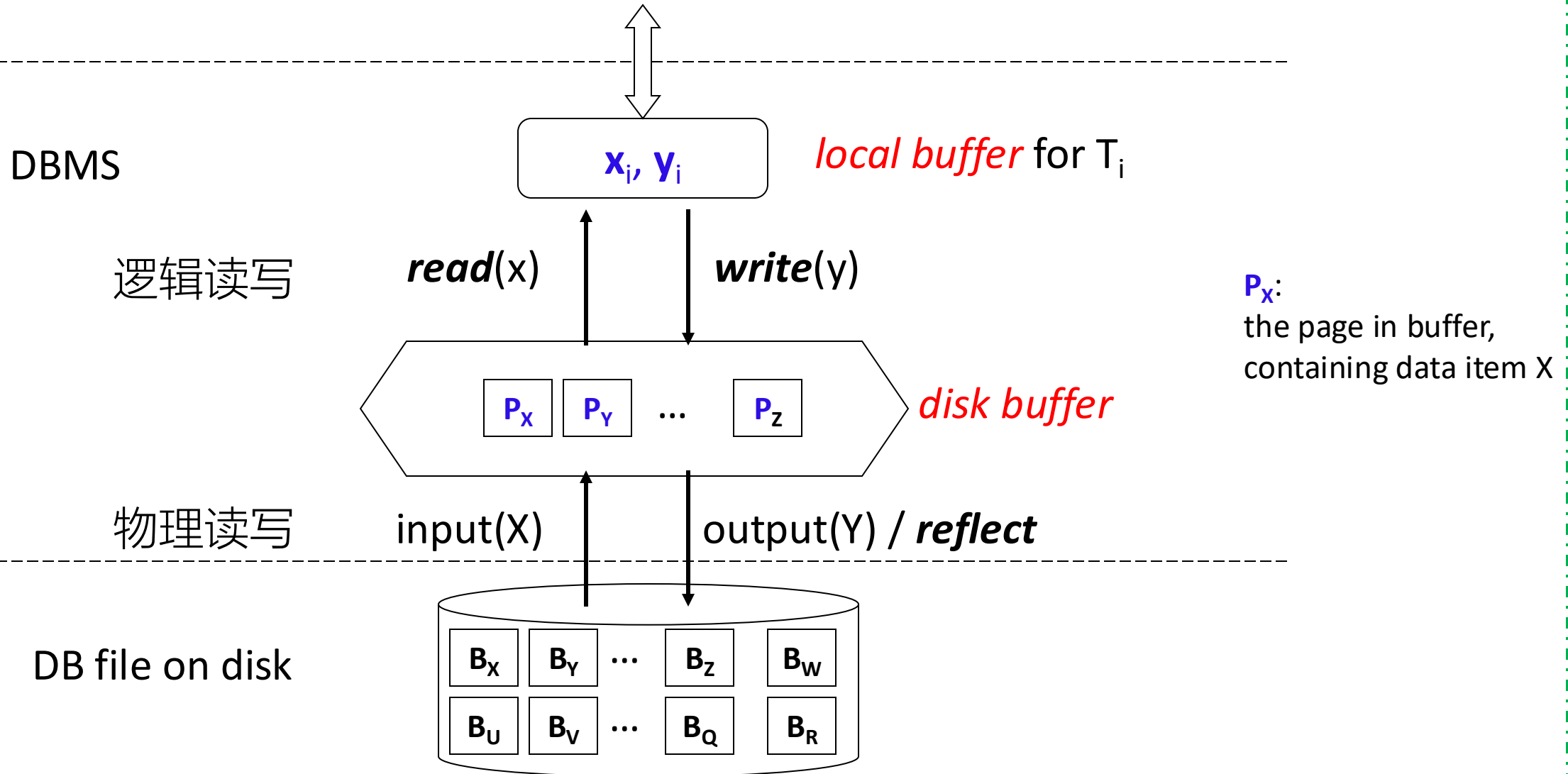


Fig.13.0.3 *read* and *write* operations in Database System Concepts

Buffer-Replacement Policies in DBMS

面向磁盘的数据库性能开销分析（2008年，SIGMOD论文）

- 假设一次业务处理（如SQL 查询）的总开销是100%，其中
 - 7%不到，处理业务逻辑
 - **34%**，缓冲区管理，如缓冲区的加载替换、地址转化等
 - 14%，处理对于内存数据结构互斥访问加锁Latching
 - 16%，处理事务并发访问时，事务处理加锁Locking
 - 12%，处理日志Logging
 - 16%，对B树索引的处理

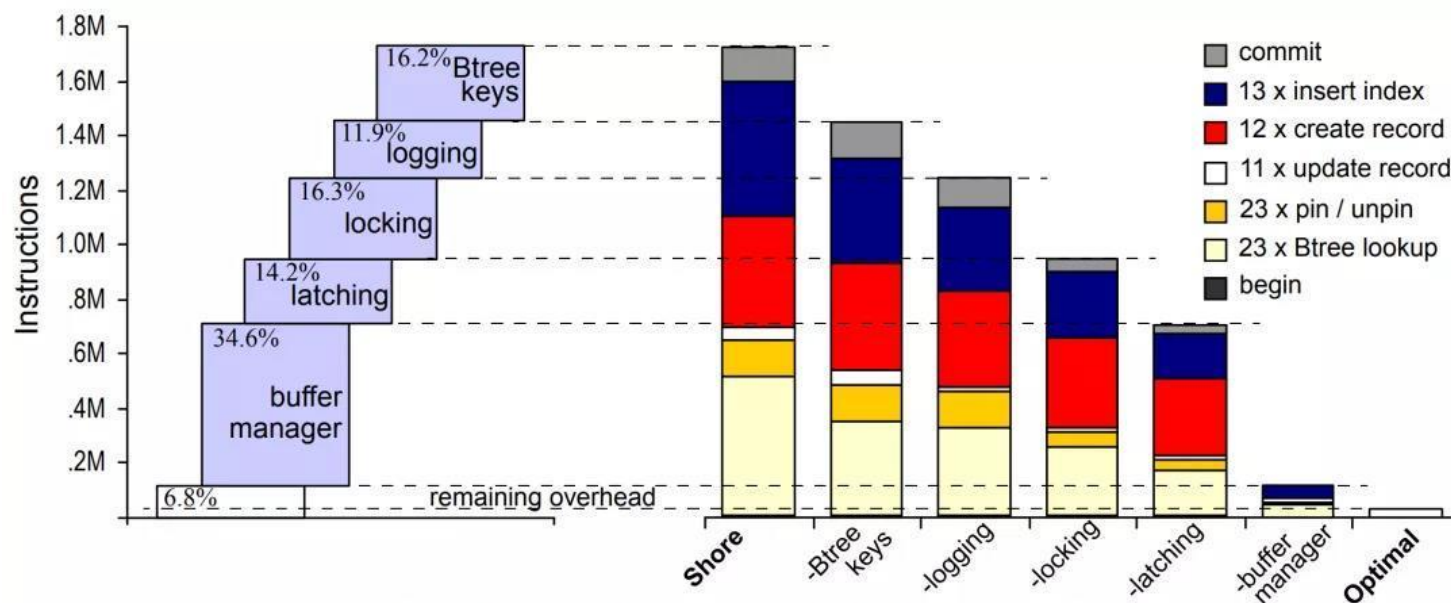


Figure 6. Detailed instruction count breakdown for New Order transaction.

- AI4DB, DB4AI

- 三类AI4DB功能

- ▣ SQL慢查询分析

- ▣ 索引推荐

- ▣ 参数优化

- 利用AI/机器学习（强化学习），动态调整数据库参数

- e.g. buffer大小

- 索引结构

Second Chance Algorithm

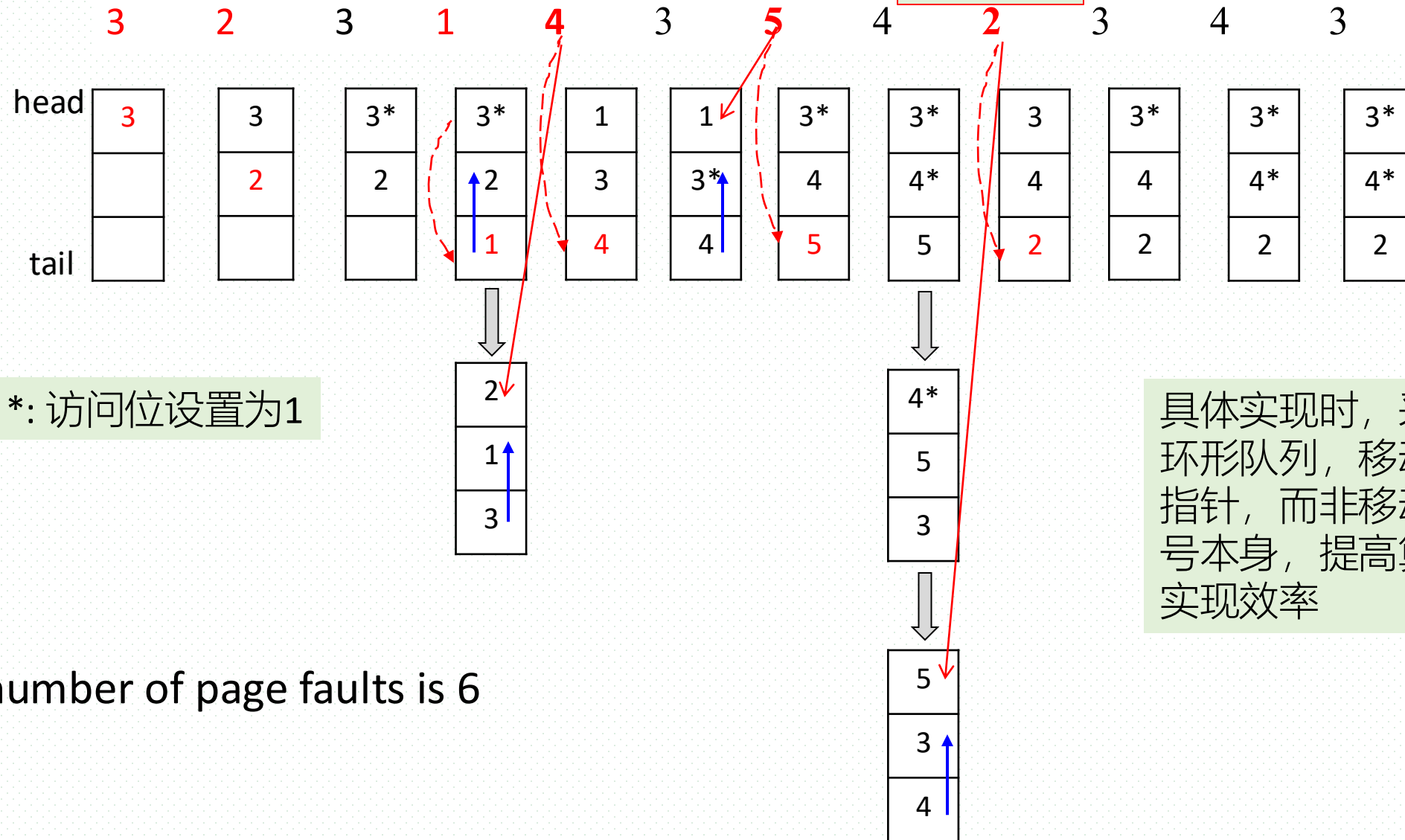
- LRU needs sufficient hardware supports, and is difficult to be used in many systems
- Some LRU approximation algorithms, which have lower implementation costs, are introduced
 - **second chance algorithm**, sometimes known as the **clock algorithm**
 - 【略: other algorithms:
LFU, MFU, second chance page-replacement algorithm,
enhanced second chance algorithm
】

Second Chance Algorithm: Principles 队列实现方式

- 采用FIFO队列，记录加载到内存的page号，较早进入内存的page位于队列前部
 - 队列中每个page维护1个访问/引用标志位
 - page第1次被加载进内存访问时，其访问位设置为0
 - page已在内存中，并被再次访问时，将访问位设置为1
 - e.g. reference bit in page table in Linux ▶
- 发生页置换，寻找被换出/swap out的page时，首先查看位于队列头的oldest page (最早加载进内存)
 - 如果该page的访问位为0，没有被置位为1，说明该page最近没有被再次访问，则该页被选为victim，被置换，新换入的page安排在队列尾
 - 如果该page的访问位为1，将该page置位为0，并移至队列尾（当作1个最近被访问的页），并从新的队列头开始重新寻找被置换的victim
- 如果队列中所有page的访问标志位都已经被置为为1，说明这些page最近都被访问过至少一次，那么在从队列头向队列尾的搜索过程中，这些page的访问位将依次被置为0，并且原本处于队列头的最早进入内存的page将再次回到队列头，并被选作victim，从而被换出

Example 1

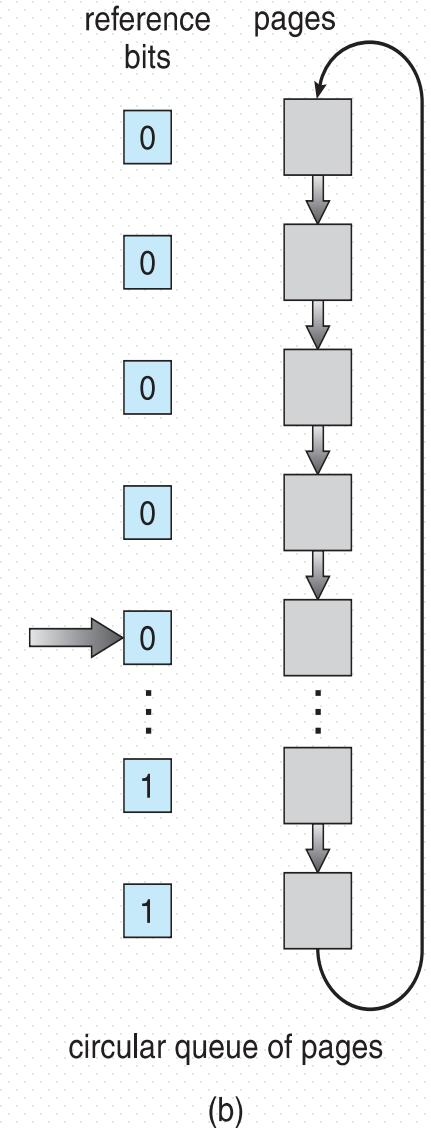
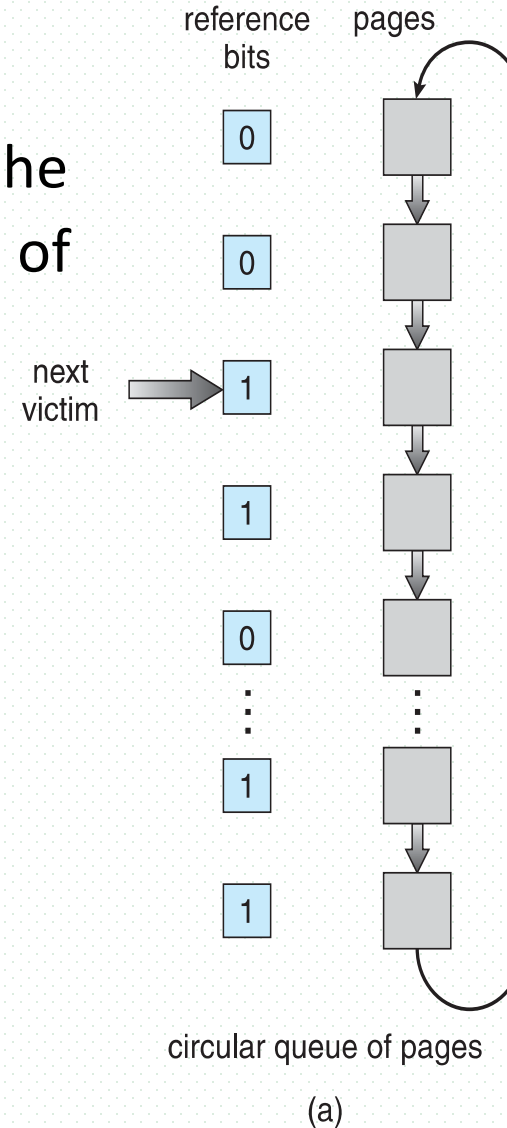
最近一段时间，
3、4被访问多次，
5只被访问1次，
被2置换



the number of page faults is 6

Principles: 另一种实现方式，循环队列实现方式

- Generally FIFO, plus hardware-provided reference bit
- If page to be replaced has
 - ▣ reference bit = 0 -> victim, be replaced; and the swapped-in page is inserted into the position of the victim in queue
 - ▣ reference bit = 1 then:
 - set reference bit 0, leave page in memory
 - replace next page, subject to same rules



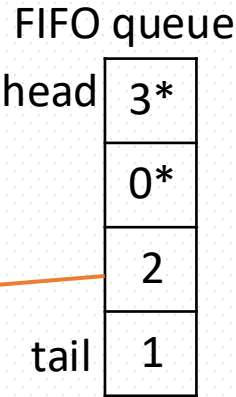
Example 2

- Given a system with 4 page frames, the following table indicates page, load time, last reference time, modify bit, and reference bit

Page	Load (into memory)time	Last reference time (the recently time access the page) Loaded page可以被多次referenced	Reference bit
0	160	370 【第2次访问本页，置位reference bit】	1
1	320: 队列尾	320	0
2	256	310	0
3	150: 队列头	340 【第3次访问本页，置位reference bit】	1

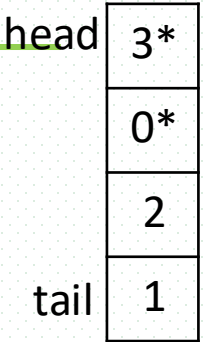
- (1) Which page will be replaced by FIFO? //依据Load time最小者/队列头, 3
- (2) Which page will be replaced by LRU? //依据last reference time最小者, 2
- (3) Which page will be replaced by the second chance algorithm?

//依据Load time/FIFO队列 + reference bit, 2



Example 2

FIFO queue



- (1) page 3,
 - ▣ FIFO select the page loaded first
- (2) page 2,
 - ▣ LRU select the page least recently used, and the last reference time of page 2 is 310, 离当前时刻最远, 被替换
- (3) page 2,
 - ▣ page 3, page 0, page 2 and page 1 are sequentially arranged in the queue; page 3 is at the queue head, and page 1 is at the queue tail
 - ▣ the second chance algorithm first checks the oldest page, page 3, since its reference bit is 1, the bit is reset to 0, and page 3 is moved to the tail of queue. The same thing happens to page 0, the next oldest page
 - ▣ page 2 is then checked, since its reference bit is 0, it is selected as the victim page to be replacement

9.5 Allocation of Frames

- Minimum number of frames for executing an instruction or a process
- Frame Allocation Algorithms
- Global and Local Replacement



【文章福利】小编在群文件上传了一些个人觉得比较好的学习书籍、视频资料，有需要的同学，群【977878001】领取！！额外赠送一份价值699的内核资料包（含视频教程、电子书、实战项目及代码）

一、CPU指令的执行过程

几乎所有的冯·诺伊曼型计算机的 CPU，其工作都可以分为 5 个阶段：取指令、指令译码、执行指令、访存取数、结果写回。



图1 CPU指令的执行阶段

1. 取指令阶段

取指令（Instruction Fetch，IF）阶段是将一条指令从主存中取到指令寄存器^①的过程。程序计数器 PC 中的数值，用来指示当前指令在主存中的位置。当一条指令被取出后，PC 中的数值将根据指令字长度而自动递增：若为单字长指令，则 $PC+1 \rightarrow PC$ ；若为双字长指令，则 $(PC)+2 \rightarrow PC$ ，依此类推。

2. 指令译码阶段

取出指令后，计算机立即进入指令译码（Instruction Decode，ID）阶段。在指令译码阶段，指令译码器按照预定的指令格式，对取回的指令进行拆分和解释，识别区分出不同的指令类别以及各种获取操作数的方法。在组合逻辑控制的计算机中，指令译码器对不同的指令操作码产生不同的控制电位，以形成不同的微操作序列；在微程序控制的计算机中，指令译码器用指令操作码来找到执行该指令的微程序的入口，并从此入口开始执行。在传统的设计里，CPU中负责指令译码的部分是无法改变的。不过，在众多运用微程序控制技术的新型 CPU 中，微程序^②有时是可重写的。

3. 执行指令阶段

在取指令和指令译码阶段之后，接着进入执行指令（Execute，EX）阶段。此阶段的任务是完成指令所规定的各种操作，具体实现指令的 ALU 功能。为此，CPU 的不同部分被连接起来，以执行所需的操作。例如，如果要求完成一个加法运算，算术逻辑单元 ALU 将被连接到一组输入和一组输出，输入端提供需要相加的数值，输出端将含有最后的运算结果。

4. 访存取数阶段

根据指令需要，有可能要访问主存，读取操作数，这样就进入了访存取数（Memory，MEM）阶段。此阶段的任务是：根据指令地址码，得到操作数在主存中的地址，并从主存中读取该操作数用于运算。

5. 结果写回阶段

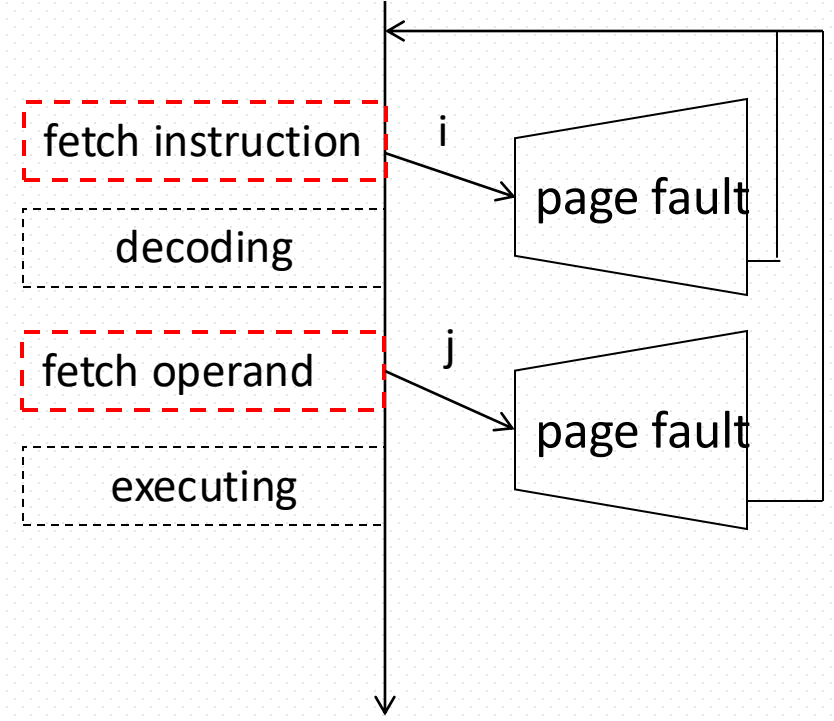
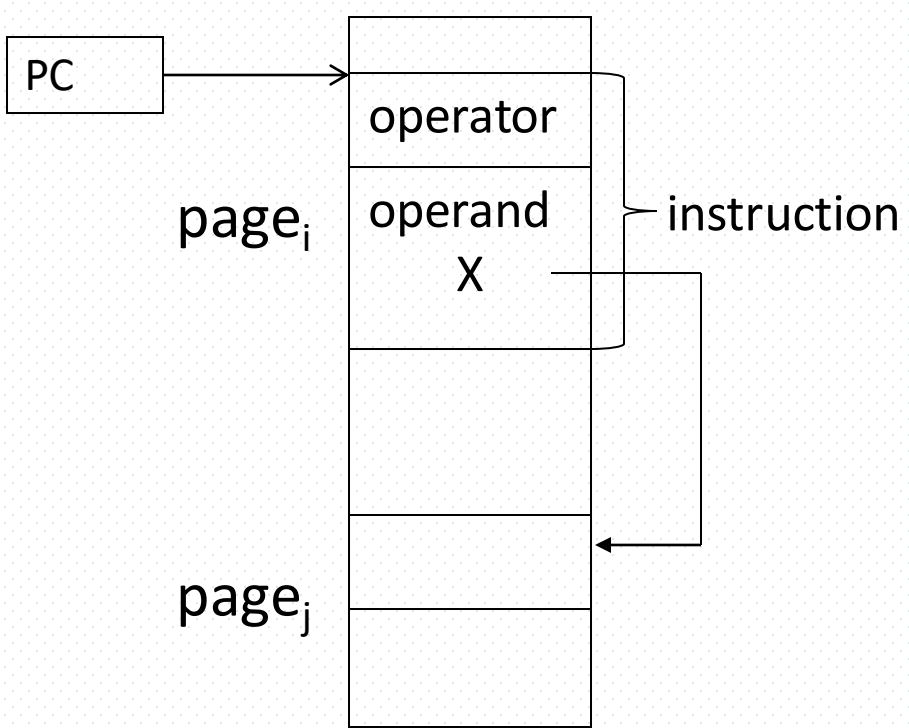
作为最后一个阶段，结果写回（Writeback，WB）阶段把执行指令阶段的运行结果数据^③写回到某种存储形式：结果数据经常被写到 CPU 的内部寄存器中，以便被后续的指令快速存取；在有些情况下，结果数据也可被写入相对较慢、但较廉价且容量较大的主存。许多指令还会改变程序状态字寄存器中标志位的状态，这些标志位标识着不同的操作结果，可被用来影响程序的运行。

在指令执行完毕、结果数据写回之后，若无意外事件（如结果溢出等）发生，计算机就接着

Minimum Number of Frames

e.g. LOAD A, X
in page_i and operand X in page_j

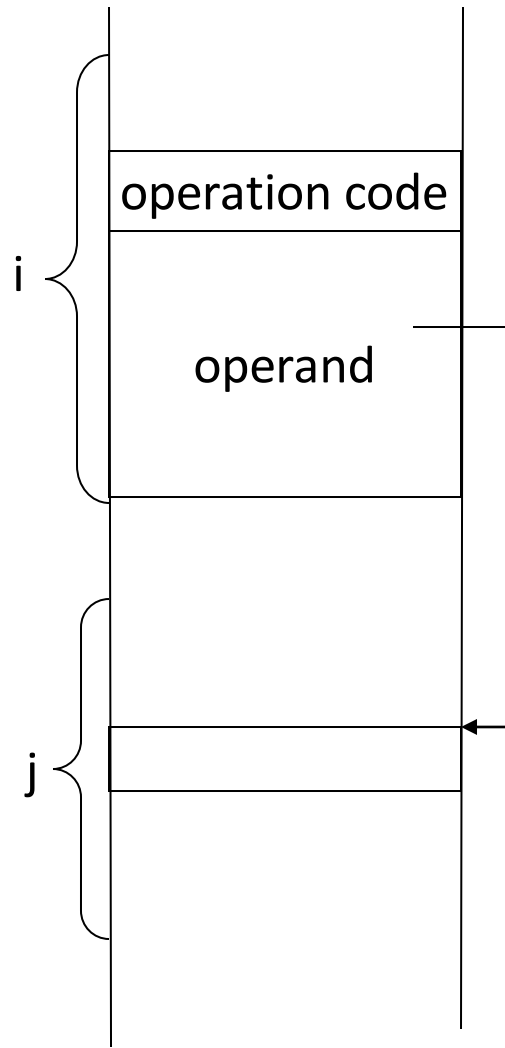
指令执行完成的前提：
指令本身、操作数必须都在内存中



assuming that,
page_i and page_j not in main memory

e.g. LOAD A, X

pages:



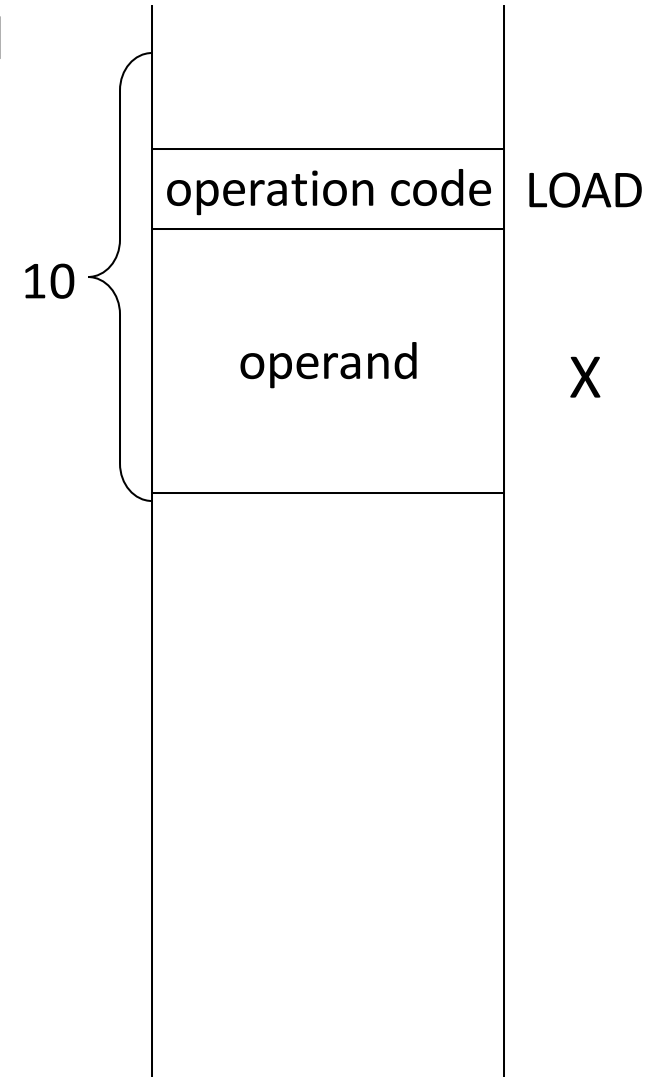
P_1 's logical address space
of size $k (>2)$

only **one** frame,
i.e. **frame 10**, is allocated
to P_1

p#	frame#
0	i -
1	i -
i	v 10
...	i -
j	i -
...	i -
k-1	i -

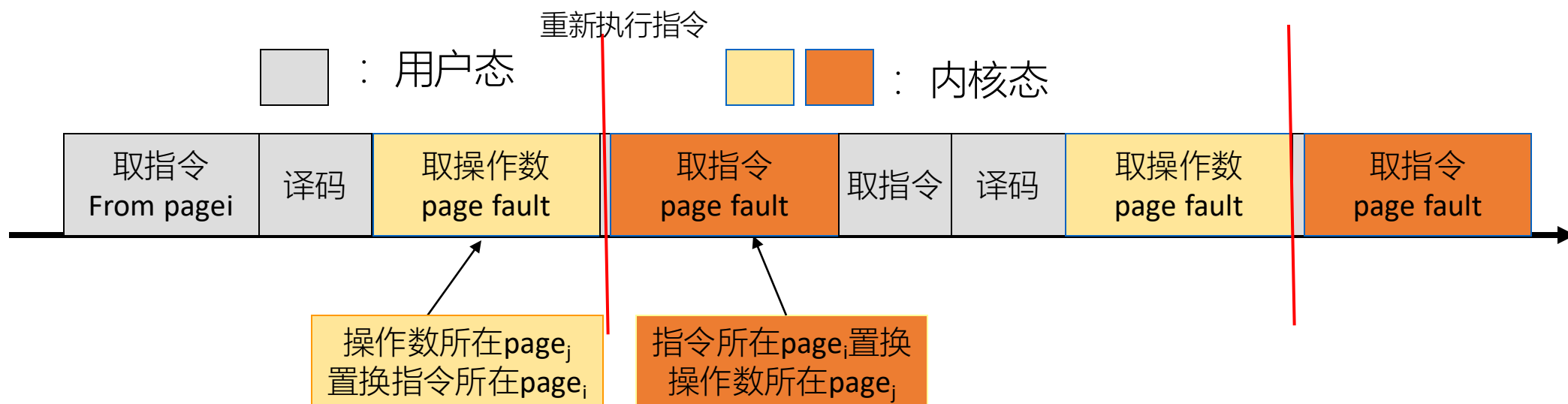
page table

frame#:



main memory

- 如果分配给进程的frame数目过少，如只有1个page用于存放进程指令和指令中的操作数，少于其正常执行需要的最低frame数目，进程将陷入某条指令的“取指令-取操作page-fault”循环中，该指令无法正常执行结束，进程停顿
 - e.g. 上图中，只分配进程1个frame，指令已在page_i中
 - 该指令执行时，CPU陷入page_i、page_j的“换进-换出”的page fault循环中，
 - 执行停顿



Minimum Number of Frames

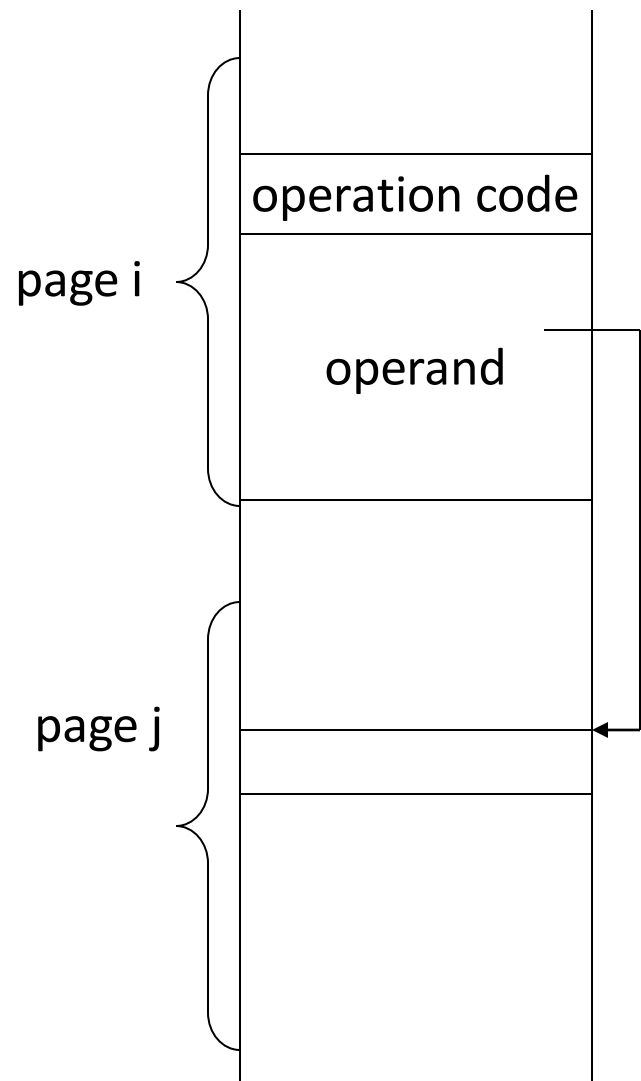
- Each process needs **minimum** number of frames to
 - ▣ meet the requirements of instruction-set architecture, or computer architecture, i.e. to accommodate the instruction and its operands that are all stored in memory
 - when a page fault occurs before an executing instruction is complete, the instruction must be *restarted*
- So, there must be enough frames to hold all pages that any single instruction can reference to, i.e. the pages that operation code and operands reside in !!!
 - ▣ avoid page-fault-rate increasing and slowing process execution

指令执行完成的前提：

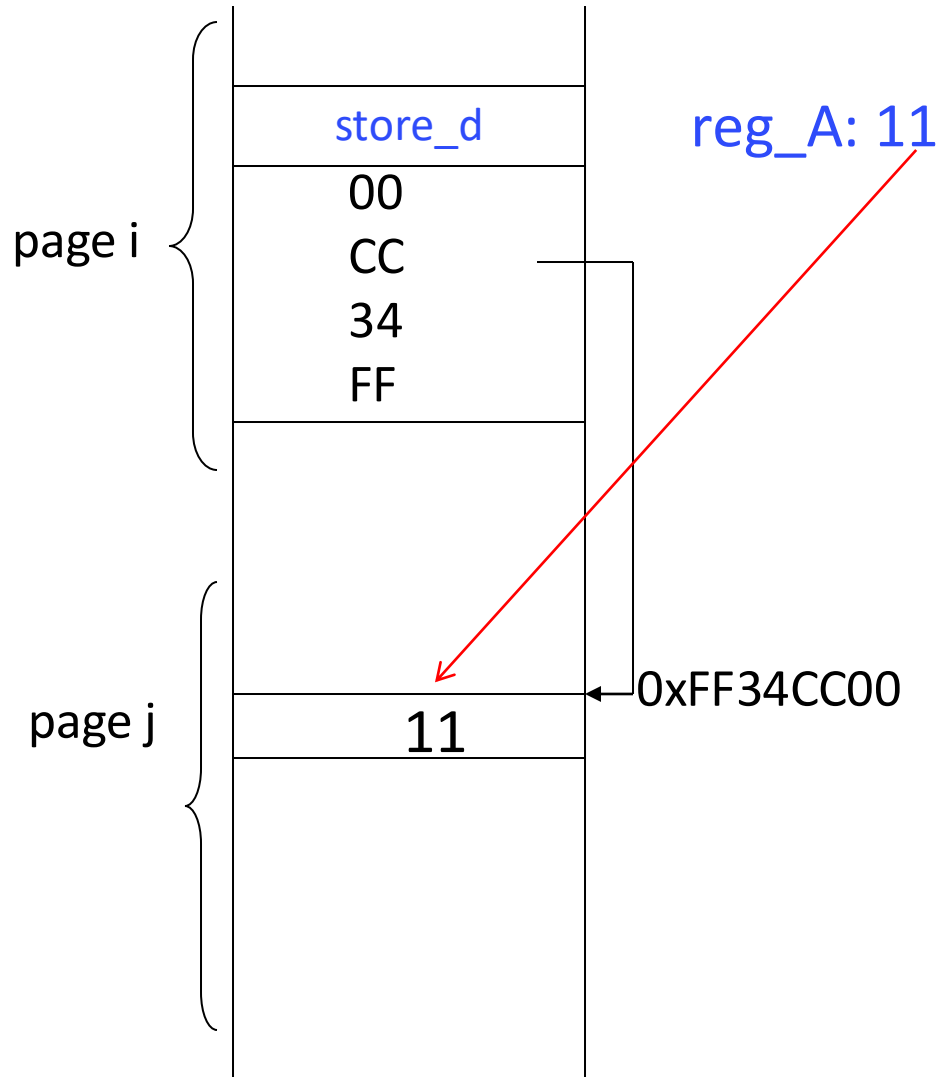
指令、操作数所在pages必须都在内存中

Example One

- A machine with all memory-reference instruction having only one memory address, and **one-level direct addressing**, for example, `store_d reg_A to X`, where `X` is a memory unit address
 - ▣ if it is supposed that an instruction and its operands are not allocated across two frames 【不跨物理页存放】, then
 - minimum number of frames needed is **two**
 - ▣ e.g. `store_d reg_A to 0xFF34CC00` in Fig. 9.0.1
 - writing register A into memory unit 0xFF34CC00
 - one frame needed to store the page, in which instruction `store_d` resides
 - another page/frame needed to store the page, in which 0xFF34CC00 resides



(a) direct addressing



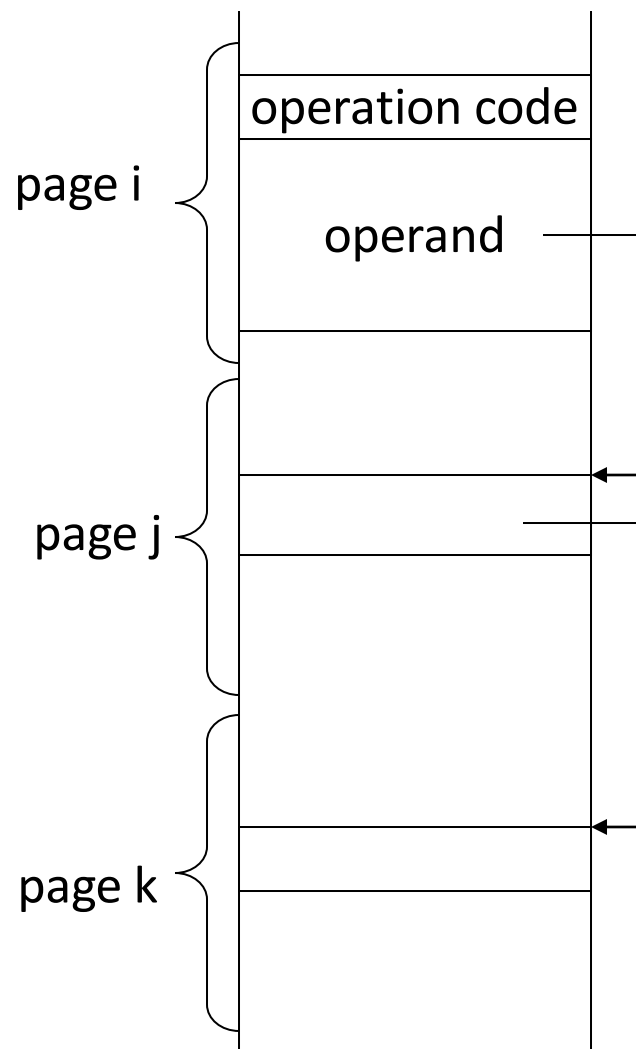
(b) store_d reg_A to 0xFF34CC00

Fig. 9.0.1

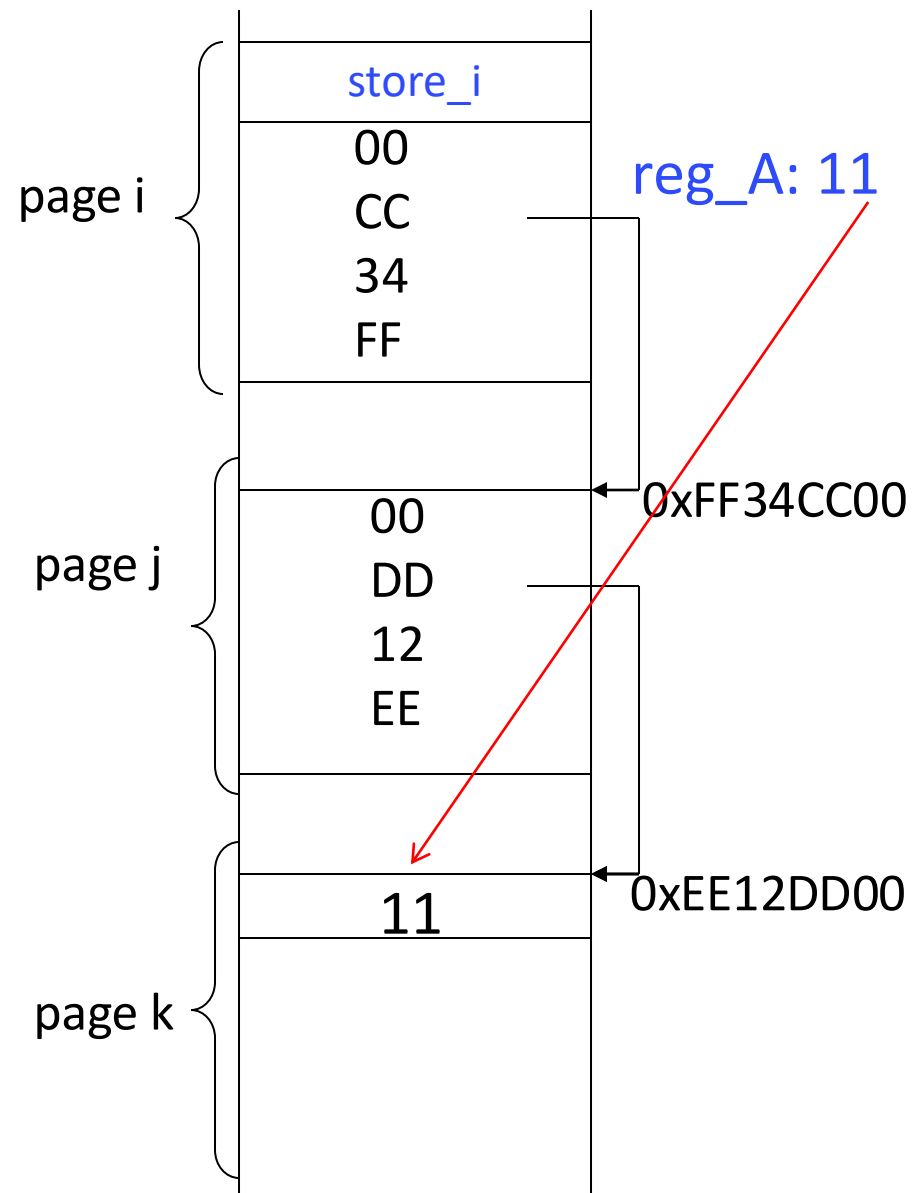
Example Two

- A machine with all memory-reference instruction having only one memory address, and one-level indirect addressing
 - minimum number of frames needed is three
 - e.g. `store_i reg_A to X`
 - writing register A into the memory unit, the address of which is stored in memory unit X
 - for instance, `store_i reg_A to 0xFF34CC00` in Fig. 9.0.2
 - one frame needed to store the page, in which instruction **store_i** resides
 - one frame needed to store the page, in which 0xFF34CC00 resides and points to 0xEE12DD00
 - one page/frame needed to store the page, in which the memory unit 0xEE12DD00 resides

Fig. 9.0.2



(a) indirect addressing



(b) store_i reg_A to 0xFFCC00

Questions

- 对双操作时指令

LOAD X, Y

X直接寻址, Y间接寻址, 指令、操作数非跨页存放

- 至少需几个page可以保证该条指令正常执行?

□ $1 + 1 + 2 = 4$

- 指令本身占用1 page
- X直接寻址, 占用1 page
- Y间接寻址, 占用2 pages

Minimum Number of Frames

- In IBM 370, 6 pages are needed to handle instruction SS MOVE (or MVC)
 - ▣ instruction is 6 bytes, might straddle 2 pages
 - ▣ 2 pages to handle **from**
 - ▣ 2 pages to handle **to**
- The minimum number of frames per process is defined by the (*instruction-set*) architecture, whereas the maximum number is defined by the amount of available physical memory!!!

Frame Allocation Algorithms

- When multiple processes request for frames, two major allocation schemes are used for allocating frames among them
 - ▣ fixed allocation, priority allocation
- Fixed allocation
 - ▣ equal allocation
 - e.g., if 100 frames and 5 processes, give each 20 frames pages
 - ▣ proportional allocation/比例分配
 - allocate frames for processes in proportion, according to the size of each process
 - e.g. example in next slide
- Priority allocation
 - ▣ allocate frames for processes in proportion, according to the priority of each process

Frame Allocation Algorithms

s_i = size of process p_i

$$S = \sum s_i$$

m = total number of frames

$$a_i = \text{allocation for } p_i = \frac{s_i}{S} \times m$$

$$m = 64$$

$$s_1 = 10$$

$$s_2 = 127$$

$$a_1 = \frac{10}{137} \times 64 \approx 5$$

$$a_2 = \frac{127}{137} \times 64 \approx 59$$

Global vs Local Replacement

- When multiple processes compete for frames, the page-replacement algorithms can be classified into 
 - ▣ **global** replacement
 - process selects a replacement frame, i.e. victim, from the set of all frames; one process can take a frame from another
 - ▣ **local** replacement
 - each process selects from only its own set of allocated frames

9.6 Thrashing [略]

- Cause of Thrashing (页面抖动/颠簸)
- Using following schemes to control allocating frames for concurrent processes, and avoid thrashing
 - ▣ Working-set Model and Locality
 - ▣ Page-fault Frequency

Cause of Thrashing

- Thrashing $\equiv$ the high paging swapping activity 
 - ▣ a process is in thrashing if it is spending more time on *paging* (i.e. on I/O) than executing, or
 - ▣ why?
 - the frames allocated to the process is not enough, the page in the process may be replaced that will be needed again right away, so the process is busy in swapping pages in and out 
 - ▣ 颠簸/抖动
 - 进程pages在物理内存与外存之间频繁swap in/out, 导致内核模式下OS swap in/out pages所需时间比用户模式下进程实际运行的时间还多, 系统效率急剧下降, 甚至导致系统崩溃
- Definition of **thrashing** in *Microsoft Computer Dictionary*
 - ▣ the state of virtual memory system that is spending almost all its time swapping in and out of memory rather than executing application

Cause of Thrashing

■ Thrashing Causes

- ❑ page replacement algorithms used are unreasonable
- ❑ **frames allocated to the processes is not enough** 
 - 进程获得的frame空间少于其正常执行需要的最低frame数目（见9.5.1节），导致进程陷入某条指令的“取指令-取操作 swap-in-out”循环中，指令无法正常执行结束，CPU频繁page fault进程停顿！
 - e.g. in Windows, 少于工作集

Cause of Thrashing

■ Case1

In Fig. 9.18  , with global replacement algorithms, if a process does not have “enough” pages, its page-fault rate will be very high. This process will preempt/take away frames from other processes

- ❑ the preempted processes must wait for and use paging devices to swap pages in and out, in the **waiting** state
- ❑ when these processes wait for paging devices, CPU utilization becomes low
- ❑ OS then increases CPU utilization by increasing the degree of multiprogramming, i.e. loading **new process** to the system
- ❑ if the new process tries to take away frames from running processes, more page faults occur, resulting in thrashing, in which CPU utilization drops sharply
- ❑ 进程处于running状态，但不断被page fault打断，CPU频繁在用户态、内核态间切换

Cause of Thrashing

■ Case2

For I/O bound processes, when the processes are in waiting state to conduct I/O operations, CPU utilization becomes low,

- ❑ OS then increases CPU utilization by increasing the degree of multiprogramming, i.e. loading **new process** to the system
- ❑ if the new process tries to take away frames from running processes, more page faults occur, resulting in thrashing, in which CPU utilization drops sharply

logical address space:

P ₁	P ₂	P ₃	P ₄	P ₅	P ₆
0	0	0	0	0	0
1	1	1	1	1	1
2	2	2	2	2	2
3	3	3	3	3	3
		4		4	4

In line with the instruction architecture in each process,

physical memory size	10					
process	P ₁	P ₂	P ₃	P ₄	P ₅	P ₆
the size of logical address space	4	4	5	4	5	5
type: I/O or CPU bound	I/O	I/O	CPU	CPU	CPU	CPU
minimum number of frames per process	2	2	3	2	3	3
minimum number of total frames required	15					

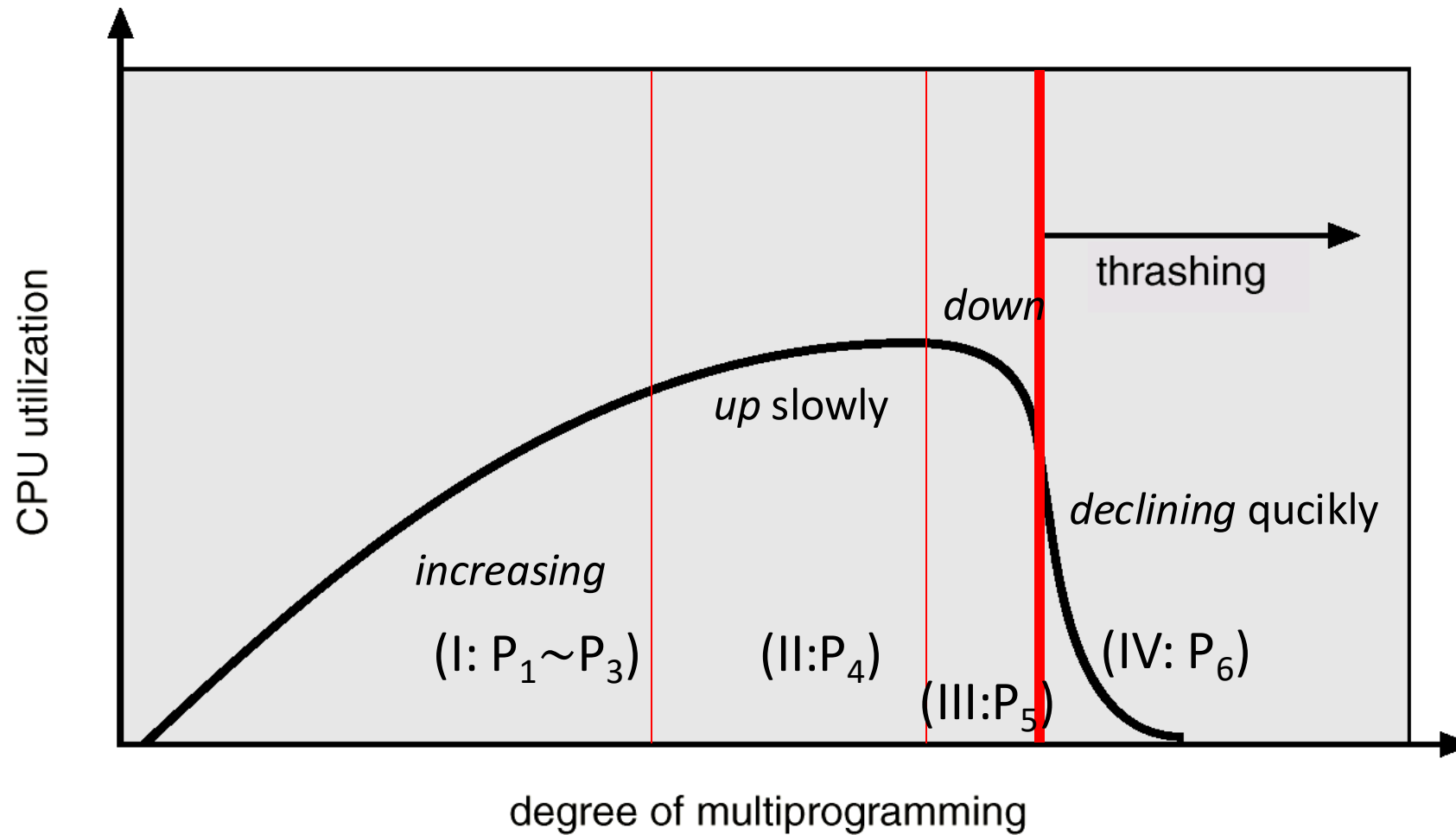


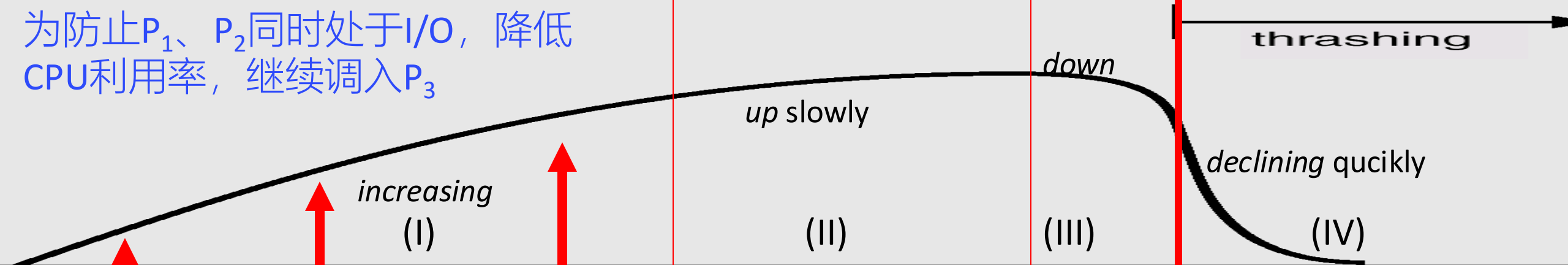
Fig. 9.18 Thrashing

当分配给 P_i 的frame数目 < 进程所需最小frames, 陷入page fault循环, 执行停顿

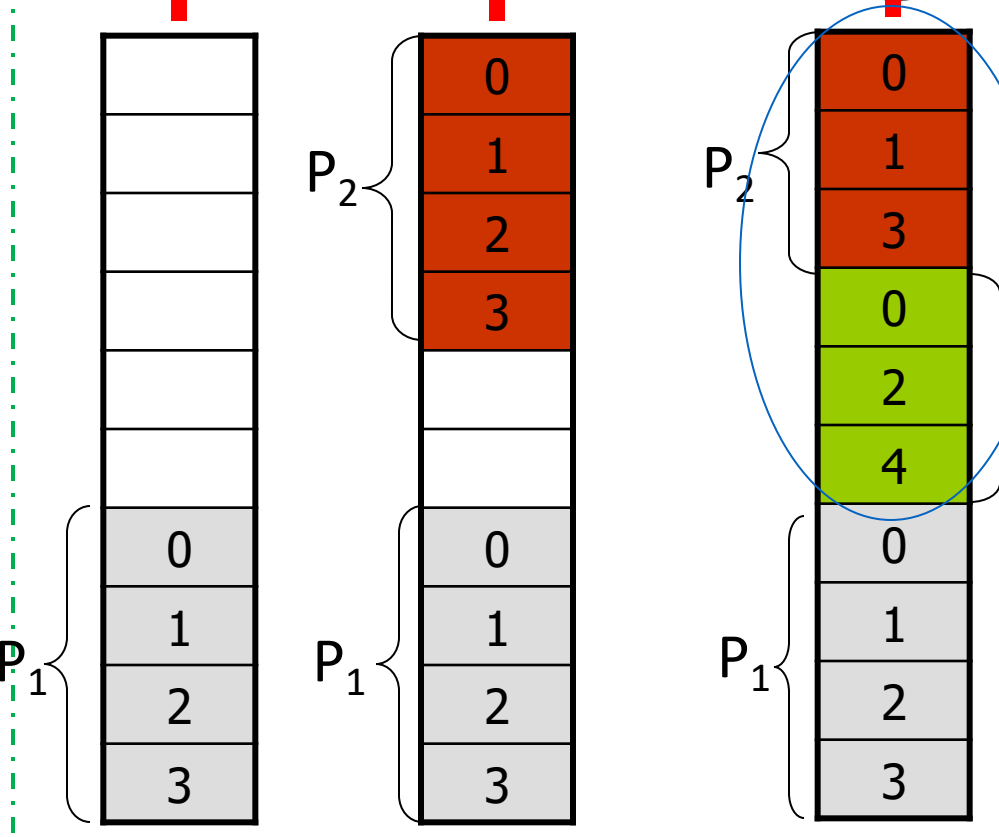
These six processes are loaded into memory sequentially, and pass 4 stages

- The 6 processes are loaded into memory sequentially, and pass 4 stages process sets in each stage:
 - stage 1. $\{P_1, P_2, P_3\}$
 - stage 2. $\{P_1, P_2, P_3, P_4\}$
 - stage 3. $\{P_1, P_2, P_3, P_4, P_5\}$
 - stage 4. $\{P_1, P_2, P_3, P_4, P_5, P_6\}$
- 1. 初始时, 将 P_1 其逻辑地址空间全部4 pages调入内存, 分配4 frames, 不会发生缺页中断
- 2. P_1 属于I/O bound进程, 当其I/O操作时, CPU利用率降低;
- 3. 为防止CPU利用率降低, 将 P_2 逻辑地址空间全部4 pages调入, 不会发生缺页中断;
- 4. 当I/O bound进程 P_1 、 P_2 同时处于I/O时, CPU利用率降低

为防止 P_1 、 P_2 同时处于I/O，降低CPU利用率，继续调入 P_3



degree of multiprogramming

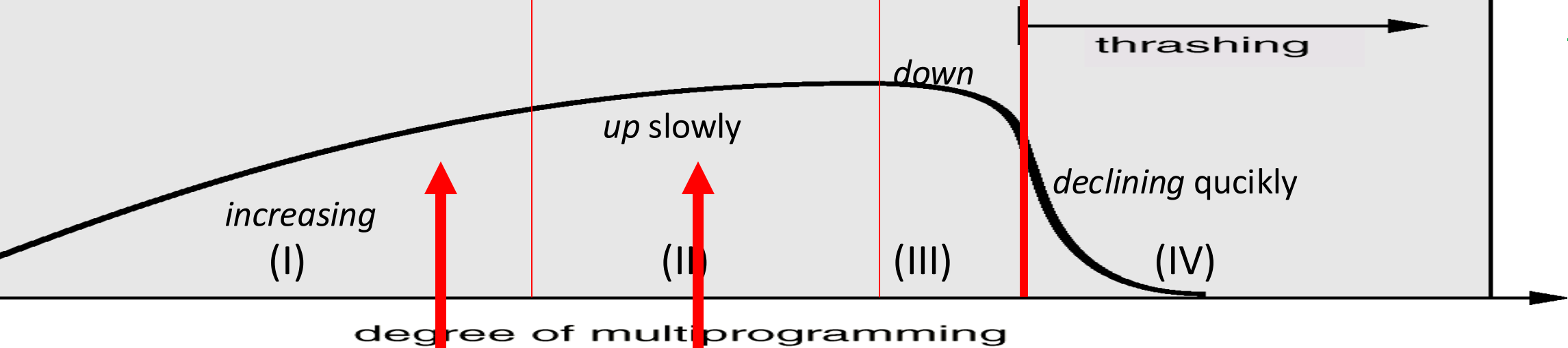


P_2, P_3 内存不足，
缺页中断增加

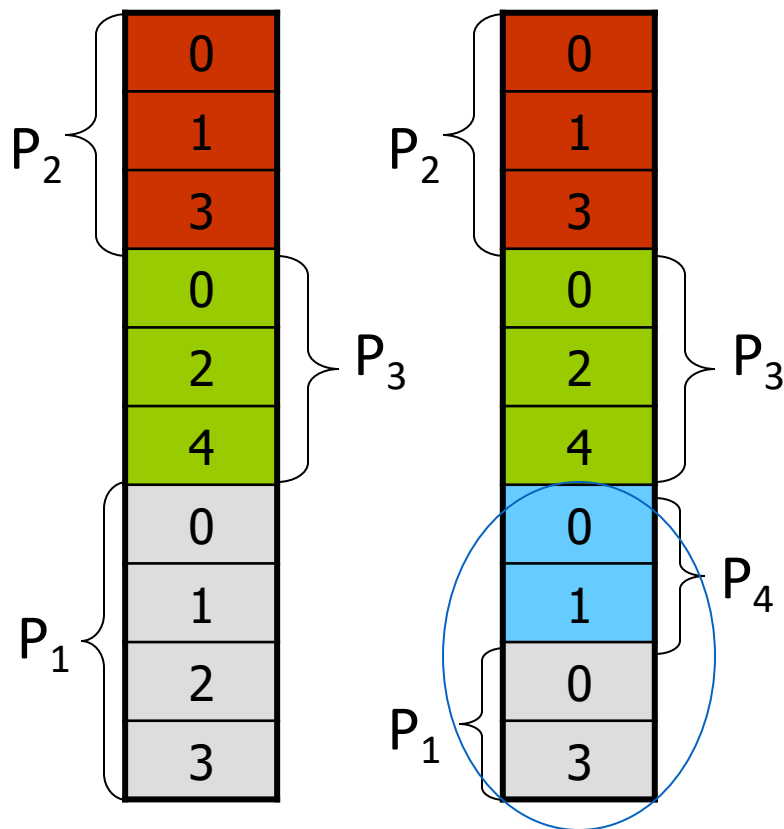
stage (I):
 P_1, P_2, P_3 run
concurrently, and the
utilization increases
smoothly

P_3 只获得3个
frames，最低
需求，运行慢，
CPU利用率提
高慢

page fault and replacement from P_2, P_3 ,



为防止 $P_1 \sim P_3$ 同时处于I/O, CPU利用率降低, 继续调入 P_4 , 导致 P_1 、 P_3 、 P_4 只能获得最低frame, 缺页中断增加, 每个进程运行减慢, CPU利用率增长进一步放缓

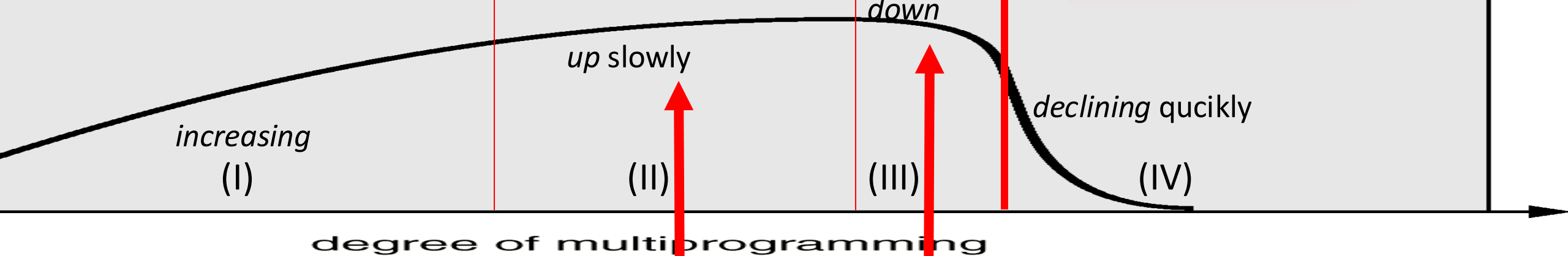


stage (II):

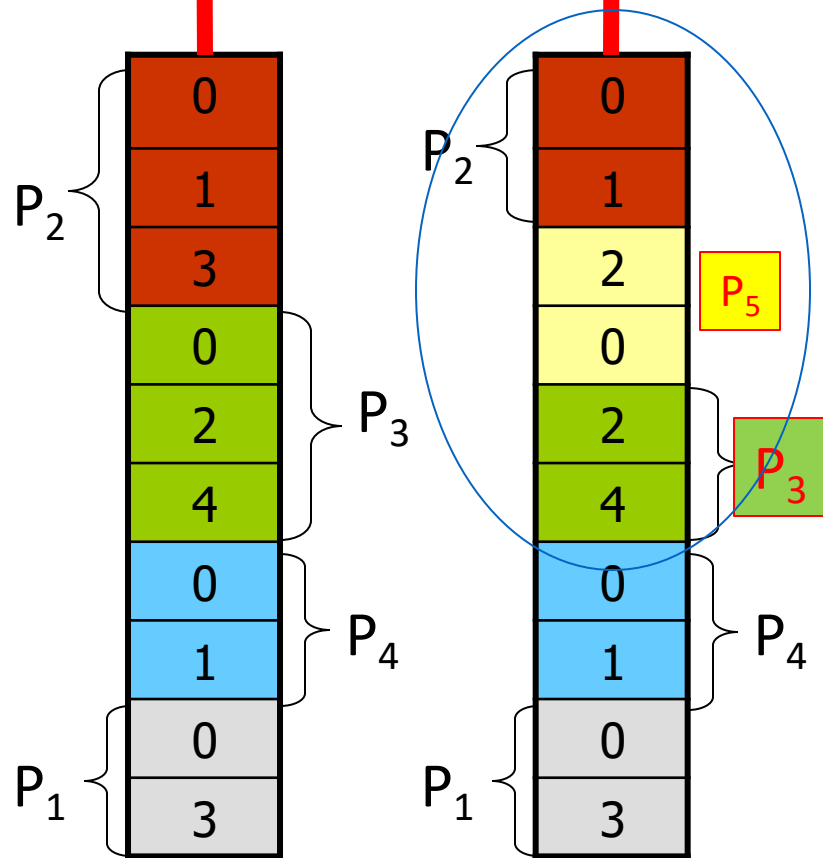
P_1, P_2, P_3, P_4 run concurrently, and the utilization increases *slowly*

process	P_1	P_2	P_3	P_4
least frames needed	2	2	3	2
frames allocated	2	3	3	2

page fault from P_2 and P_3 | page fault from $P_1 \sim P_4$; P_1, P_3, P_4 proceed slowly



为解决CPU利用率增长放缓，调入 P_5 ，导致 P_3 和 P_5 获得的frames少于最低需求，执行停顿； P_1 、 P_2 、 P_4 只获得最低frames，执行缓慢，CPU利用率开始下降

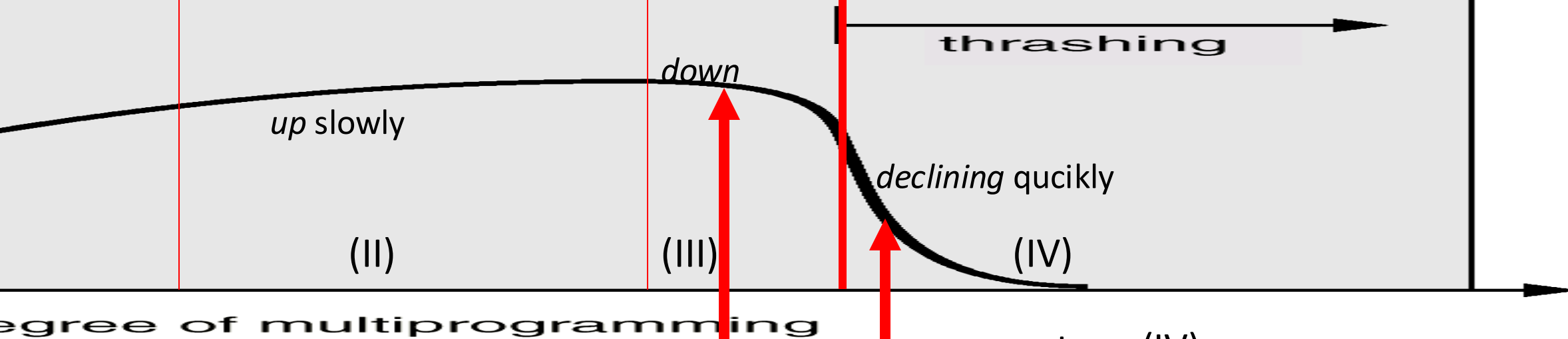


stage (III):
5 processes run concurrently, and the utilization decrease

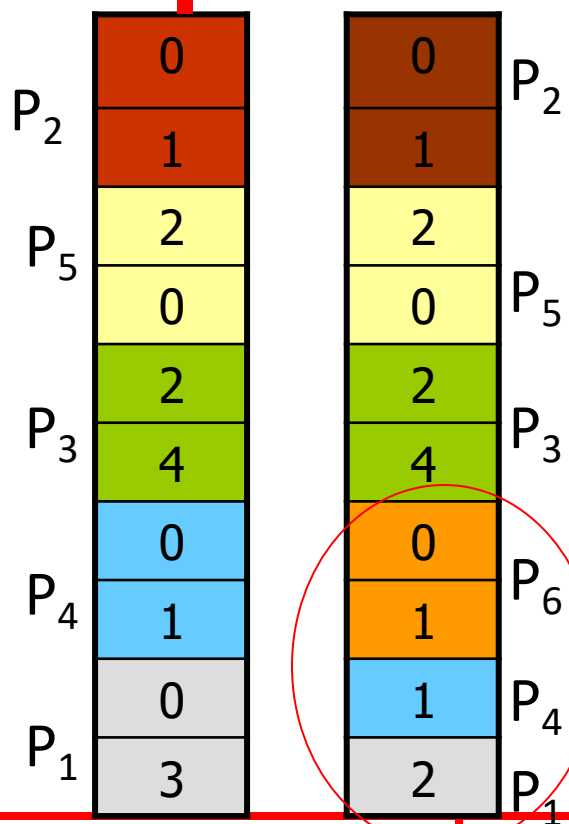
process	P_1	P_2	P_3	P_4	P_5
least frames needed	2	2	3	2	3
frames allocated	2	2	2	2	2

page fault from $P_1 \sim P_4$

page fault from $P_1 \sim P_5$, and P_3, P_5 cannot proceed



调入 P_6 ，导致除 P_2 外，其它进程获得的frames少于最低需求，执行停顿； P_2 只获得最低frames，执行缓慢，CPU利用率急剧下降



stage (IV):

6 processes run concurrently, thrashing occurs

process	P_1	P_2	P_3	P_4	P_5	P_6
least frames needed	2	2	3	2	3	3
frames allocated	1	2	2	1	2	2

page fault from $P_1 \sim P_6$, thrashing, 5 processes cannot proceed

P_3, P_5 cannot proceed

Cause of Thrashing

- 结论
 - 根据系统可用资源，如内存，合理控制系统多道程序设计粒度——并发执行的进程总数

Thrashing vs Deadlock

■ Thrashing

- 进程竞争使用内存
- 内存资源不足，导致进程在running态下陷入swap in/swap out, **running-waiting-ready**，停滞不前——指令重新执行
- CPU频繁进行用户态、内核态切换，在内核态下进行page fault处理

■ Deadlock

- 进程竞争互斥使用资源，包括内存资源
- 资源不足，导致进程处于**waiting态**，停滞不前——进程长久waiting

Cause of Thrashing

- In Fig. 9.18, with local replacement algorithms, if a process is thrashing due to the frames allocated to itself do not meet the minimum numbers of frames required
 - it can not cause other processes to thrash
 - the thrashing process will be in the waiting queue for most of time, thus the average service time for a page fault will increase, due to the longer queue for the paging devices
 - EAT will increase

Working-set Model

- To prevent thrashing, a process should be provided as many frames as it needed , if the system knows/estimates how many frames it actually need in the near future
- A mechanism named as **working-set strategy** is used to handle thrashing, and estimate the frames that the process need in recent periods
 - ▣ e.g. 工作集 in Windows ▶
- It is based on the **assumption of locality**

Locality

- Two types of **program/process** locality

- temporal, spatial

- Temporal locality

短时间内, 一个内存位置(memory location)被多次重复引用/访问(reference)

- e.g.1 a program using a variable (in a page) in a statement is likely to access that variable again in the next few instructions (in the same page)

$x := 1;$

$y := 2;$

$z := x + y$

x is stored in page i , y is stored in page j , and the next instruction z is stored in page k , then the reference string is i, j, k, i, j

- e.g.2 an instruction in a loop will be accessed again in a short period of time when the loop cycles back for subsequent iterations

for $n=1$ to 100 do $z:=x+y$

—— n, z, x, y are located in four different pages, 执行for循环将连续访问这4个pages

Locality

- Spatial locality 一个内存位置memory location被引用(reference), 很快地, 其临近位置也被引用

- e.g.1 two variables are typically allocated adjacent memory locations (perhaps locations on a stack or heap). Accessing these two variables results in references to memory locations in close proximity to each other, may be in the same page

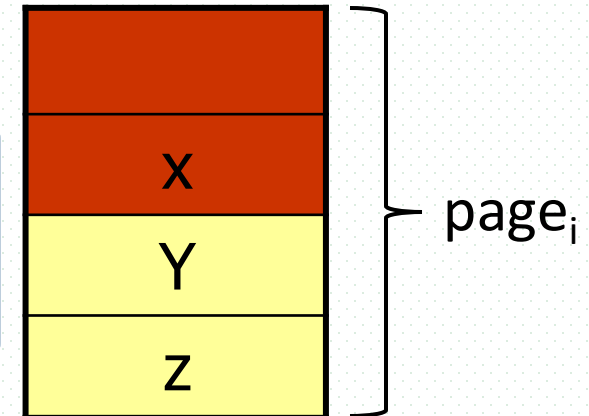
```
int x, y, z
```

```
x:=1
```

```
y:=1
```

```
z:= x+y
```

x, y, z is stored in page i
then the reference string is
i, i, i, i, i

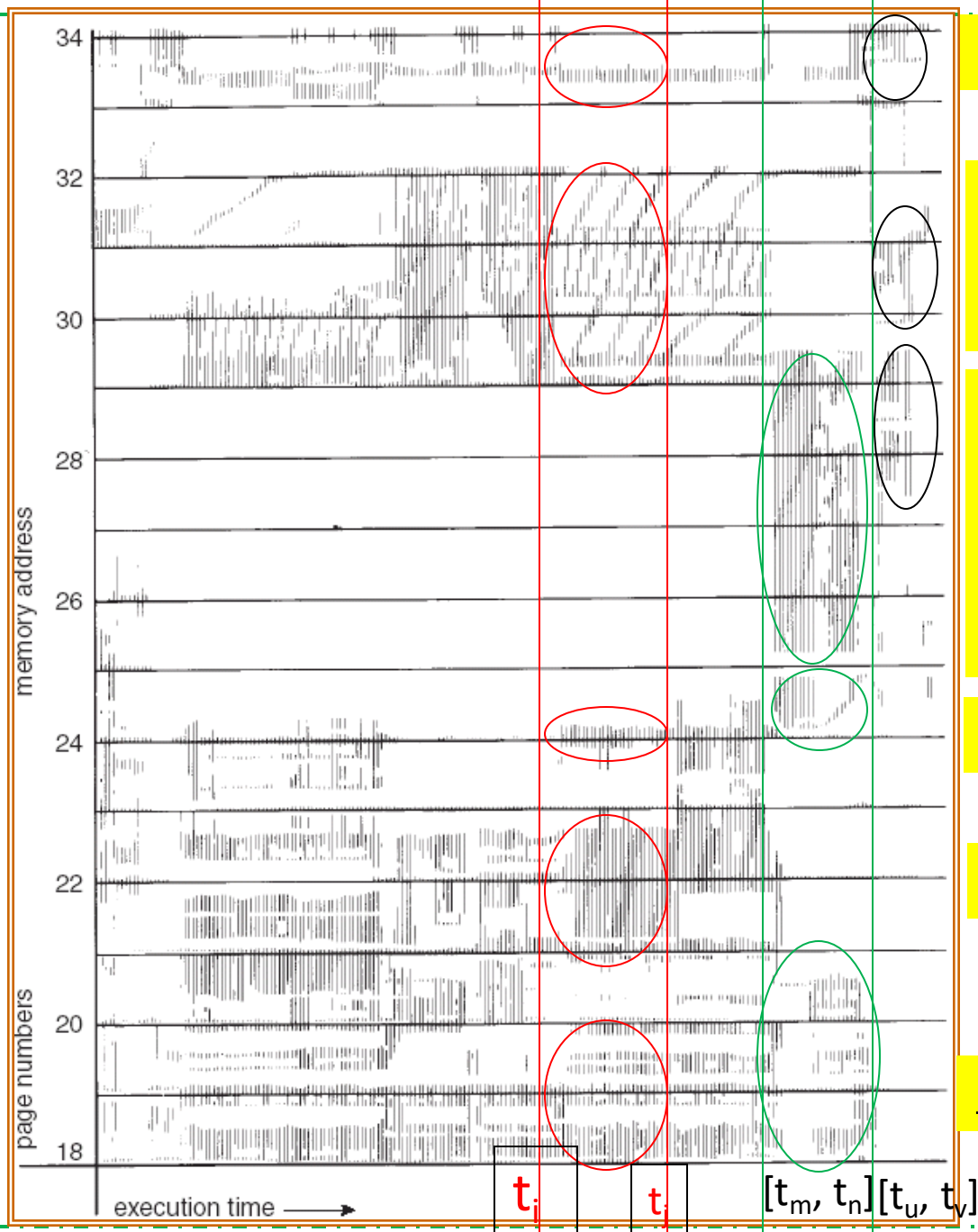


- e.g.2 sequential execution of instructions results in the next instruction executed being located in a memory location close to the previous location

■ 结论

- ▣ 进程执行过程中，在一段“短”时间内， e.g. 100ms ，不可能访问全部逻辑地址空间，访问局限在一些“局部区域”
 - sets of pages in logical address space, named localities

address space:
page 18~34



函数

过程

共享
数据区

堆

子程序

主程序

1. $[t_i, t_j]$ 内主要局限在5个locality中, 需要的frame数目较多, e.g. 12;
2. $[t_m, t_n]$ 内主要局限在3个locality, 需要的frame数目较多, e.g. 10
3. $[t_u, t_v]$ 内主要局限在3个locality, 需要的frame数目少, e.g. 4

Fig. 9.19 Localities in a memory reference pattern

Locality

- **Locality assumption!!** (or **locality model** of process execution), as shown in Fig. 9.19
 - Locality
 - a set of pages that are actively used together in a process, during a time window,
 - its size is lower than process' logical address space
 - a program is composed of several different localities, which may overlap
 - when process executes, it migrates from one locality to another
 - e.g. from the “main program” locality to the “function” locality
 - localities may overlap

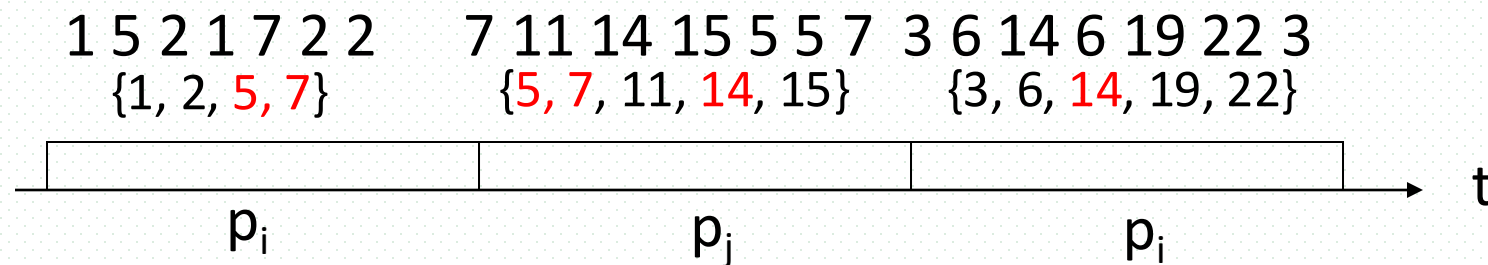


Fig. Reference string and localities during time windows

Working-set Model

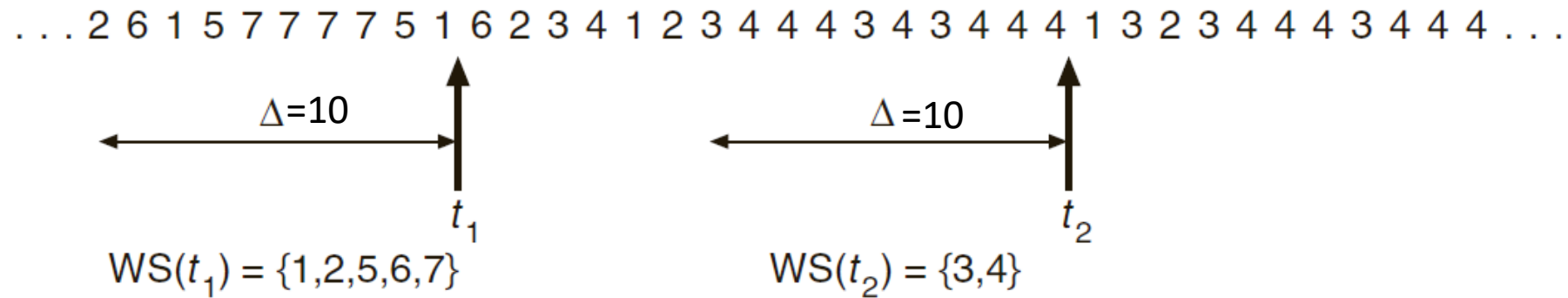


Fig. 9.20 Working set Model

- Working-set model
 - uses a parameter, Δ , to define the working-set window
 - examine the most recent Δ page references in window
 - the set of pages in the most recent Δ page references is the working set
- If a page is in active use, it will be in the working set. If it is no longer being used, it will drop from the working set Δ time units after its last reference
 - e.g. page 7 in $WS(t_1)$, not in $WS(t_2)$
- Thus, the working set is an approximation of the program's locality

Working-set and Thrashing

- The current locality of P_i is the minimum memory need of P_i , at present
- Why does thrashing occur? For all processes P_i ,
 $D = \sum_i (\text{size of current locality for } P_i)$
> available memory size
- On the basis of locality assumption, **working-set model** is used to estimate the size of the current locality of P_i

Using Working-set to Handle Thrashing

- Furthermore, working-set model is used to prevent thrashing while keeping the degree of multiprogramming as high as possible
 - ▣ OS monitors the working set of each process, and allocates enough frames to each working set
 - ▣ if there are enough extra frames, a new process can be initiated
 - ▣ if the current sum of the size of each working set increases, exceeding the total number of available frames, **to prevent thrashing, one process should be suspended**
 - 根据并发进程工作集、当前可用frames，判断可用内存是否满足进程需求

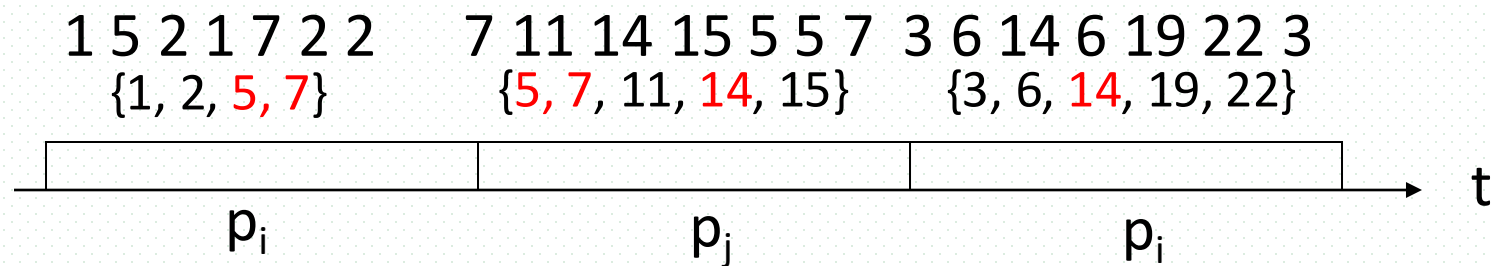


Fig. Reference string and localities during time windows

- Establish “acceptable” page-fault rate, on the basis of which, dynamically adjust the frames allocated to processes in memory
 - ❑ if actual rate too low, process loses frames
 - ❑ if actual rate too high, process gains frames

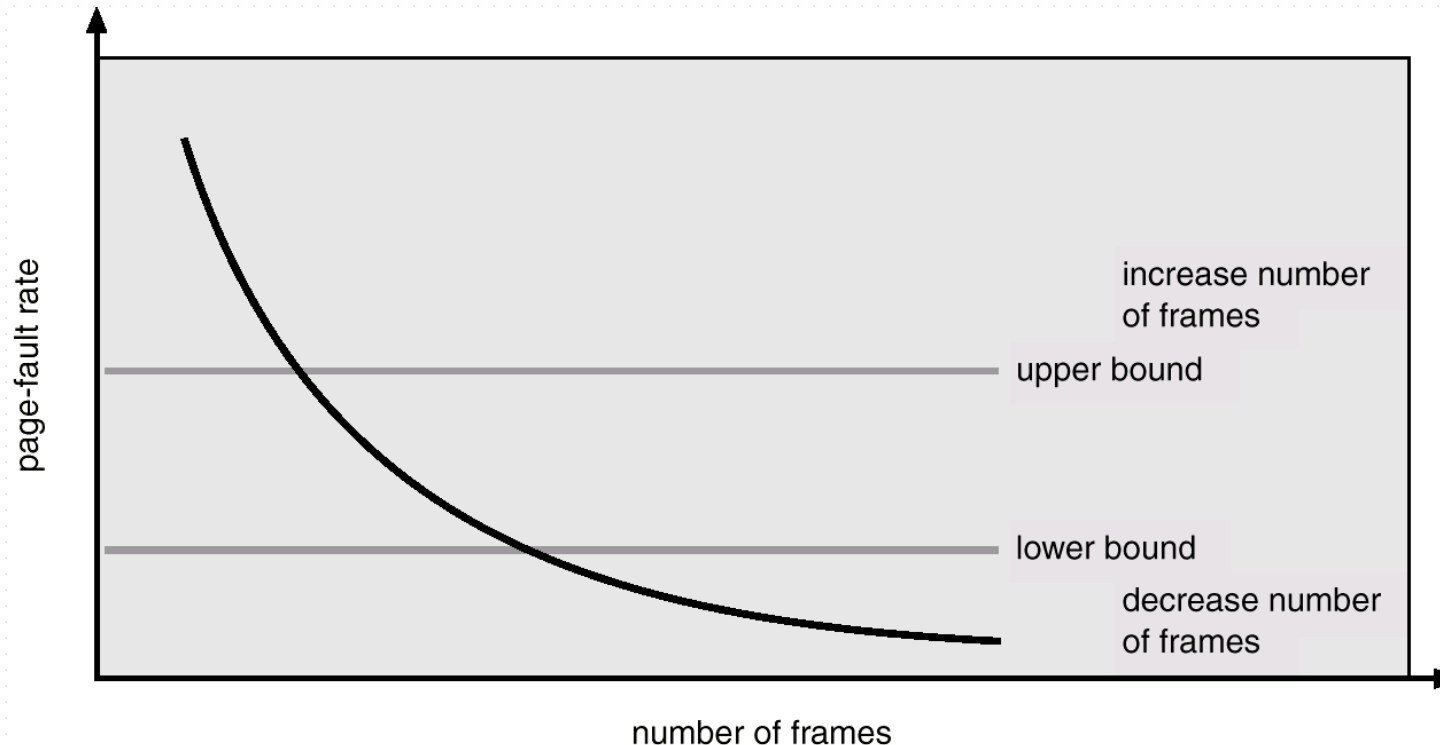


Fig. 9.21
Page-fault
frequency

- 评估进程内存资源需求，动态调整内存
 - ▣ 空闲页面追踪
 - Working Set Estimation
 - ▣ 压力拥塞感知/PSI(Pressure Stall Information)机制
 - ▣ 内存大小自动寻优
- 空闲页面追踪(idle page tracking)
 - ▣ 在page table中为每个page设置 Idle 标志位
 - ▣ 进程执行过程中，如果1个page访问到了(referenced)，则清除该标志位，否则将判定该page为 idle page
 - ▣ 根据各个page的空闲/活跃信息，可估算出进程保持 working 所需的 true memory，即 Working Setting Size(WSS)
 - ▣ 在mm/page_idle.c中实现
 - ▣ 缺点：需要对page table进行遍历搜索，不适合大型内存

- 压力和拥塞感知/PSI机制
 - 4.20 内核引入PSI, Pressure Stall Information
 - 自动监测内存等资源的使用强度
 - 如果进程因为等待内存资源而 stall了, 说明此时内存成为一个限制条件, 应适当增加分配给进程的内存资源
- 内存自动寻优
 - 利用PSI, 自动寻找进程所需最佳内存大小
 - 原理
 - 当一个memory cgroup (memcg)的内存使用超过memory.high的限制后, 该cgroup 内进程申请的内存分配将被 throttlen (超得越多, 限速越狠)
 - 在不给一个memcg 设置内存分配限制的情况下, 记录其运行数据和PSI, 而后给该memcg 设定一个high值, e.g. 700M, 测量其内存分配延迟, 如果无明显下降, 就将high值调低, 如调为600M。如果延迟变化还是不明显, 就再降到500M
 - 如果high值降低后, 内存分配延迟明显增加, 说明出现stall, 再适当调高high值
 - 利用PSI信息, 在进程运行过程中动态调整memory.high, 寻找memcg 所承载的workload 真正需要的内存大小

9.3 COPY-on-Write

- Virtual memory allows other benefits
 - ▣ copy-on-write (写拷贝) for quick process creation
 - ▣ memory-mapped files (内存映射文件) for quickly accessing files on disks

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/wait.h>
```

```
int main()
{
```

```
    pid_t pid;
    int num = 5;
    int status;
    pid = fork();
    switch(pid)
```

```
    {
```

失败 case -1:

```
        perror("create porcess is failed");
        exit(-1);
```

子进程 case 0:

```
        printf("child process num is:%d\n", num);
        exit(0);
```

父进程 default:

```
        wait(&status);
        printf("parent process num is: %d\n", num);
        break;
```

```
    }
```

```
    return 0;
```

```
}
```

■ 进程创建系统调用之一——fork()

- 功能：全面复制，创建出与父进程几乎相同的子进程

- 返回值

- 成功时，该函数被调用一次，但返回2次（对子进程返回0，对父进程返回非零的子进程PID）；不成功时，对父进程返回-1，对子进程无返回值

子进程共享父进程的地址空间和num

运行结果如下：

```
xuexi@xuexi1:~/summer/TeamB/member/zhuwenbo/linux-c/2week$ ./a.out
child process num is:5
parent process num is: 5
```

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/wait.h>
```

```
int main()
{
    pid_t pid;
    int num = 5;
    int status;
    pid = fork();
    switch(pid)
    {
        case -1:
            perror("create process is failed");
            exit(-1);
        case 0:
            num = 10;
            printf("child process num is:%d\n", num);
            exit(0);
        default:
            wait(&status);
            printf("parent process num is: %d\n", num);
            break;
    }
    return 0;
}
```

父进程中num保持不变;

子进程中num发生改变, num=10:

当子进程修改num值时, fork写拷贝机制,
为子进程创建独立进程映像, 为num分配新的
存储空间

运行结果如下:

```
xuexi@xuexi1:~/summer/TeamB/member/zhuwenbo/linux-c/2week$ gcc 1.c
xuexi@xuexi1:~/summer/TeamB/member/zhuwenbo/linux-c/2week$ ./a.out
child process num is:10
parent process num is: 5
```

https://blog.csdn.net/weixin_42250655

COPY-on-Write

- When a parent creates its child, the child duplicates its parent's pages, and the parent and the child processes initially **share** some pages
 - e.g. `fork()` in Linux/Unix
- The page that may be modified, such as the page containing data, need be marked as **COW** page, meaning a **non-sharing page**

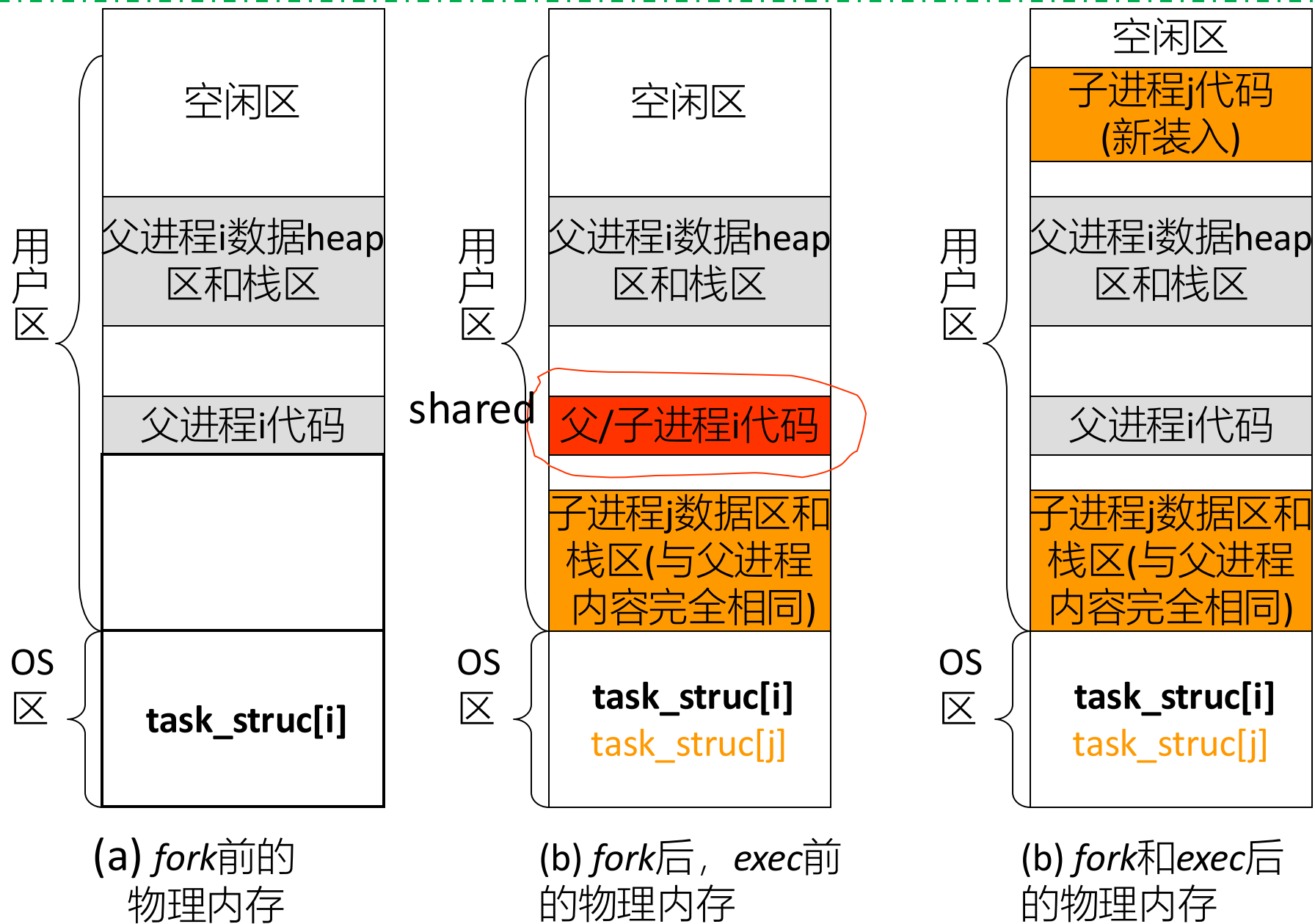


Fig. 9.3.0 `fork`, `exec` for process creation

Copy-on-Write (cont.)

- The pages initially marked as COW may then mapped into the private space of parent or child, and cannot be shared
- The page that may not be modified, and need to be shared by parent and child afterwards, such as the page containing shared executable code, ***are not be marked*** as COW
 - ▣ non-COW pages can still shared by parent and children processes
- If either process, e.g., the child, wants to modify/writes to a shared page marked as COW, a **copy** of this shared page is created by OS, and this copy of shared pages is **mapped** into the private address space of the child that want to writes to the shared page.
- The child modifies the **copied page**, not the shared page belonging to the parent

An Example of COW

- There are two processes, parent P_i and child P_j , the address space of P_i consists of (refer to Fig. 9.3.1 (a))
 - ▣ code section, which will be shared with P_j , locating in [page5](#)
 - ▣ ***read-only*** public data section, which will also be shared with P_j , locating in [page2](#)
 - ▣ ***private*** data section/heap/stack, which is not allowed to be shared with P_j , locating in [page3](#)

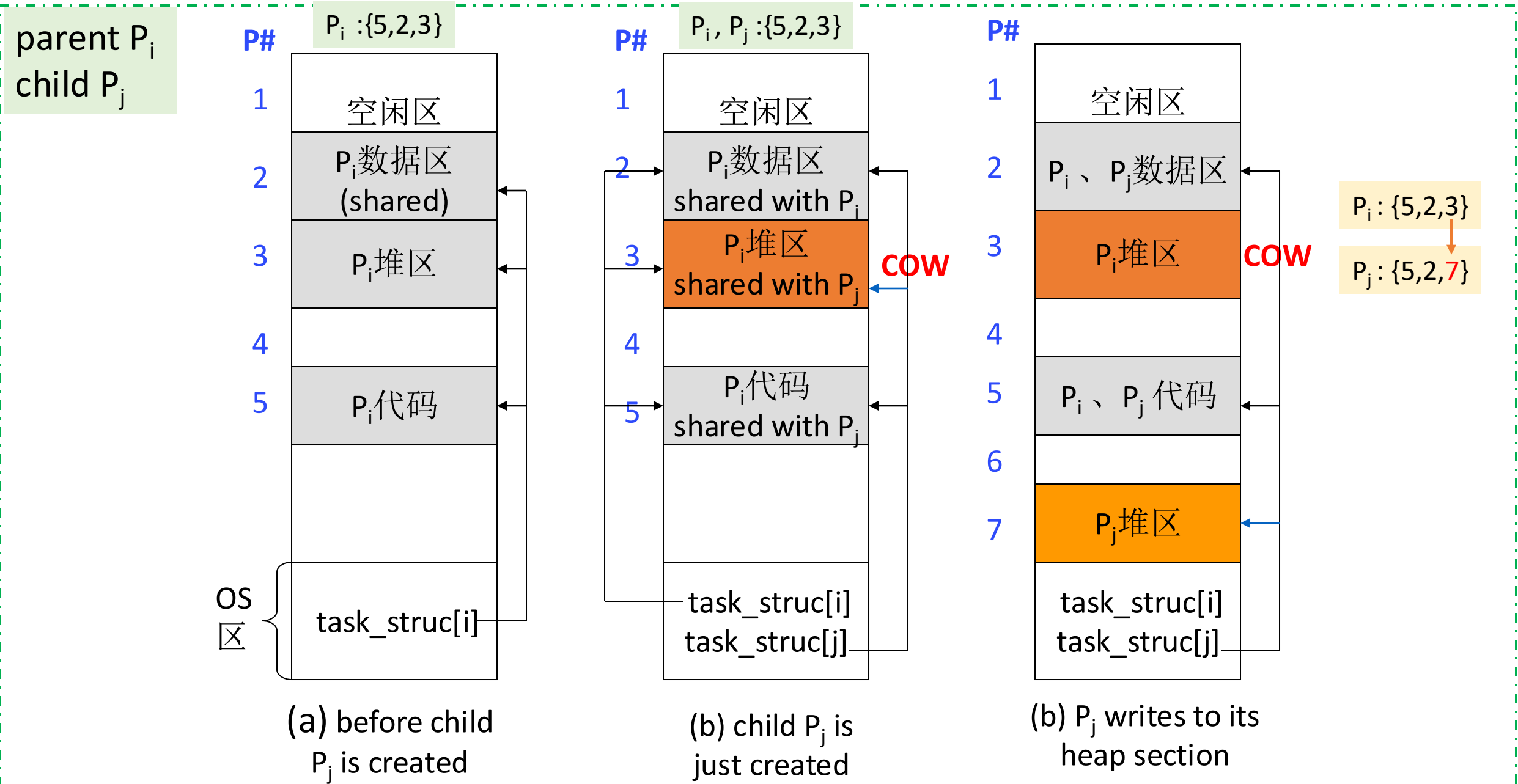


Fig. 9.3.1 An Example of COW

An Example of COW

- P_i creates P_j through, e.g., **fork()**. After just being created, P_j shares the address space of its parent P_i
 - **page3** are marked as **COW**, the private heap for P_j , these pages will be rewrote by P_j afterwards
 - **page2** and **page5** are not marked as COW, P_i and P_j will share and only read these pages
- During its execution times, when P_j wants to **modify/writes** to its heap section in shared **page 3** marked as COW, for example, modify a local variable x defined in P_j and stored in heap
 - a **copy** of these shared pages, i.e. **page7** is created by OS, and **mapped** into P_j 's address space
 - P_j modifies its private heap, i.e. the copied **pages7**

An Example of COW

- After COW is conducted, P_j 's address space consists of
 - ▣ code section in [page5](#)
 - ▣ public read-only data section in [page2](#)
 - ▣ private heap in [page7](#)

Copy-on-Write (cont.)

- COW allows more efficient **process creation**
 - ▣ only modified pages are copied
 - ▣ 父、子进程的页表项指向同一个内存中frame, 但该页标记为COW
/*一个物理页对应了多个逻辑页!
- COW is used in Windows2000, Solaris2, and Linux
- Refer to Appendix 9-1, for more details about COW ▶

9.7 Memory-Mapped Files

- Two mechanisms for secondary storage (files) access
 - ▣ by I/O operations, such as read()/write() system calls
 - ▣ by memory-access instructions, on the basis of memory-mapped file
- Memory-mapped file
 - ▣ a file whose storage space (i.e., disk block) is *mapped into/associated with* a part of process's virtual address space/virtual memory
 - mapping a file disk block to a page in memory
 - ▣ memory-mapping files access can be treated as memory access
 - /*将存储文件的（一部分）外存空间映射到进程逻辑地址空间/虚拟内存中, 进程利用机器指令, e.g. load A, X, 以访问内存的方式访问外存文件

Memory-Mapped Files

- Implementation — on the basis of demand-paging
 - ▣ a process prepare to access a page in its virtual memory space, and this page is associated with a file disk block by memory-mapping file
 - ▣ by means of demand paging, a page-sized portion of the file is read from the file system into a physical page (i.e. frame)
 - ▣ accesses on the file are treated as ordinary memory accesses
 - ▣ e.g. refer to Fig.9.7.1 in next page
- Merits
 - ▣ simplifies file access by treating file I/O through memory access rather than **read()** **write()** system calls
 - ▣ 节省IO过程中数据在内存buffer中的缓存次数, 提高IO速度

load A, X: refer to memory vs *load A, Y*: refer to the file on disk

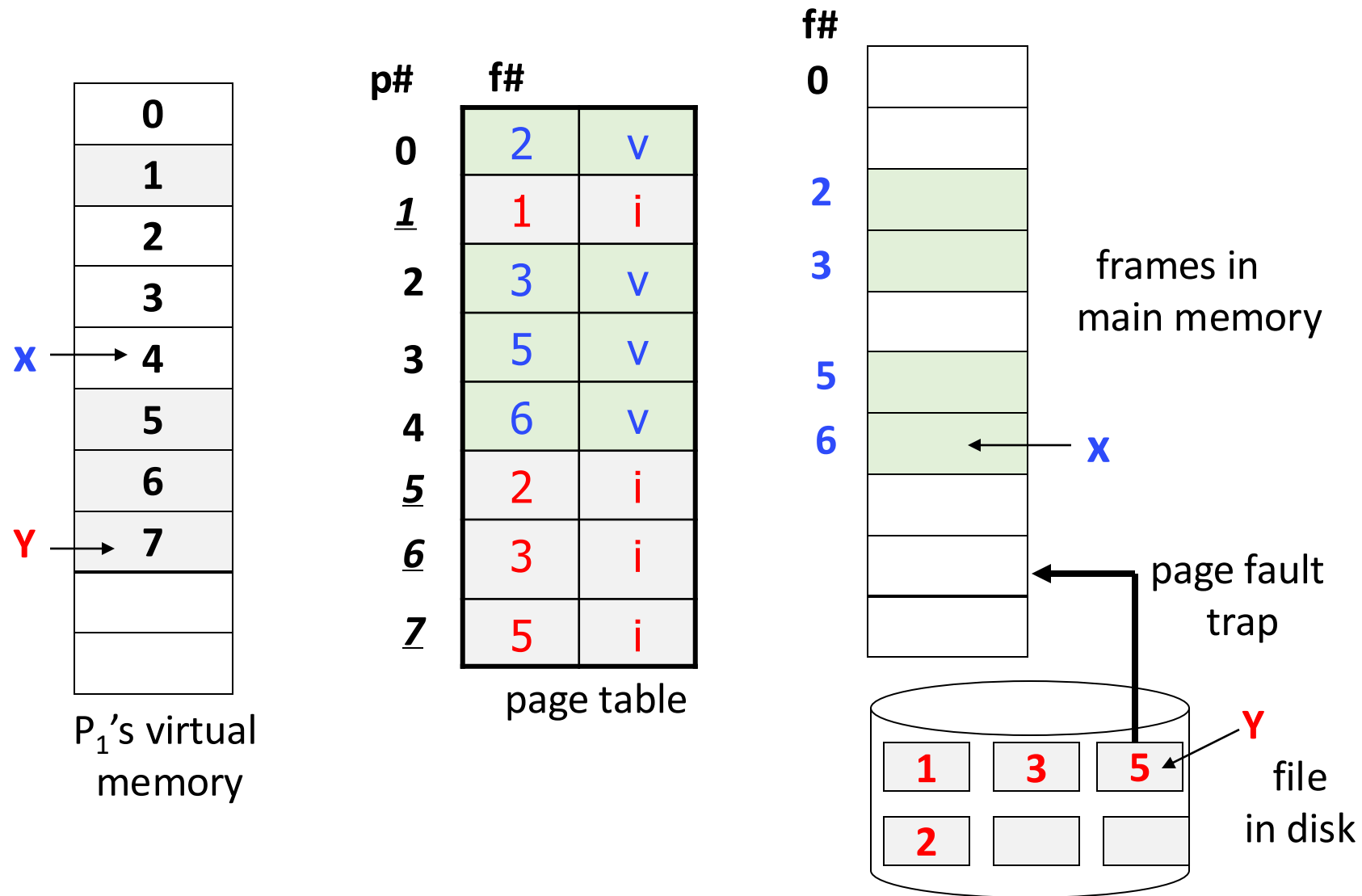


Fig. 9.7.1 Memory mapping file access

Memory-Mapped Files

■ Sharing file by memory-mapped file

several processes may be allowed to map the same file into their virtual space, to share the data in the file among these processes

- ▣ by page-sharing in VM, each process points to the same pages of physical memory, also to the pages contains the disk blocks of the files
- ▣ refer to Fig. 9.23
- ▣ /* 用内存映象实现文件共享时, 共享文件的2个进程可以分别将各自虚拟地址空间中的不同的page利用页表指向
 - physical memory中的同一frame, 或
 - 外存地址空间中的同一block
 - 例如, P_1 将page 2#、 P_2 将page2# 同时指向frame100# 或block2# (假设block2#调入内存后放在frame100#内)

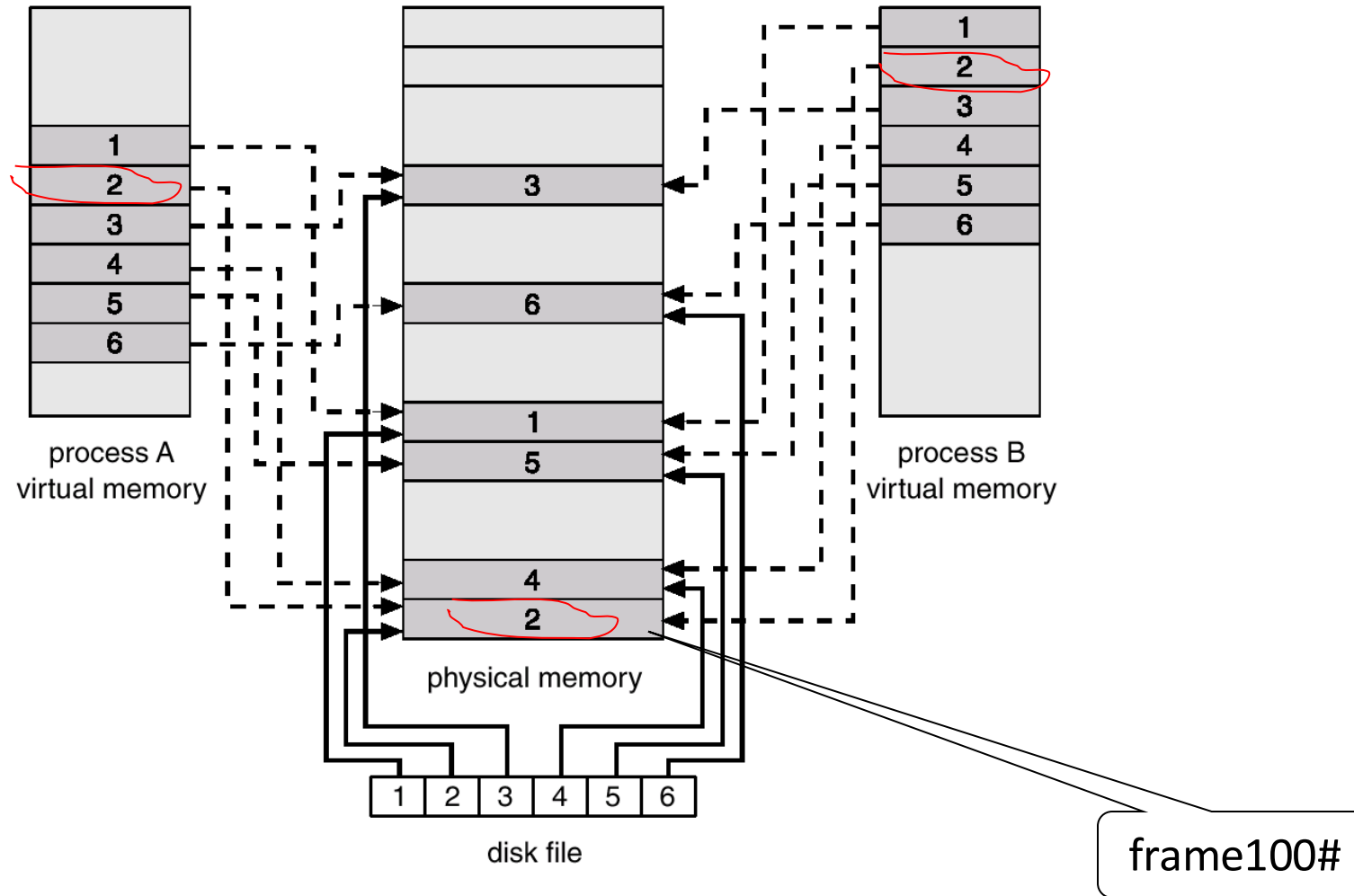
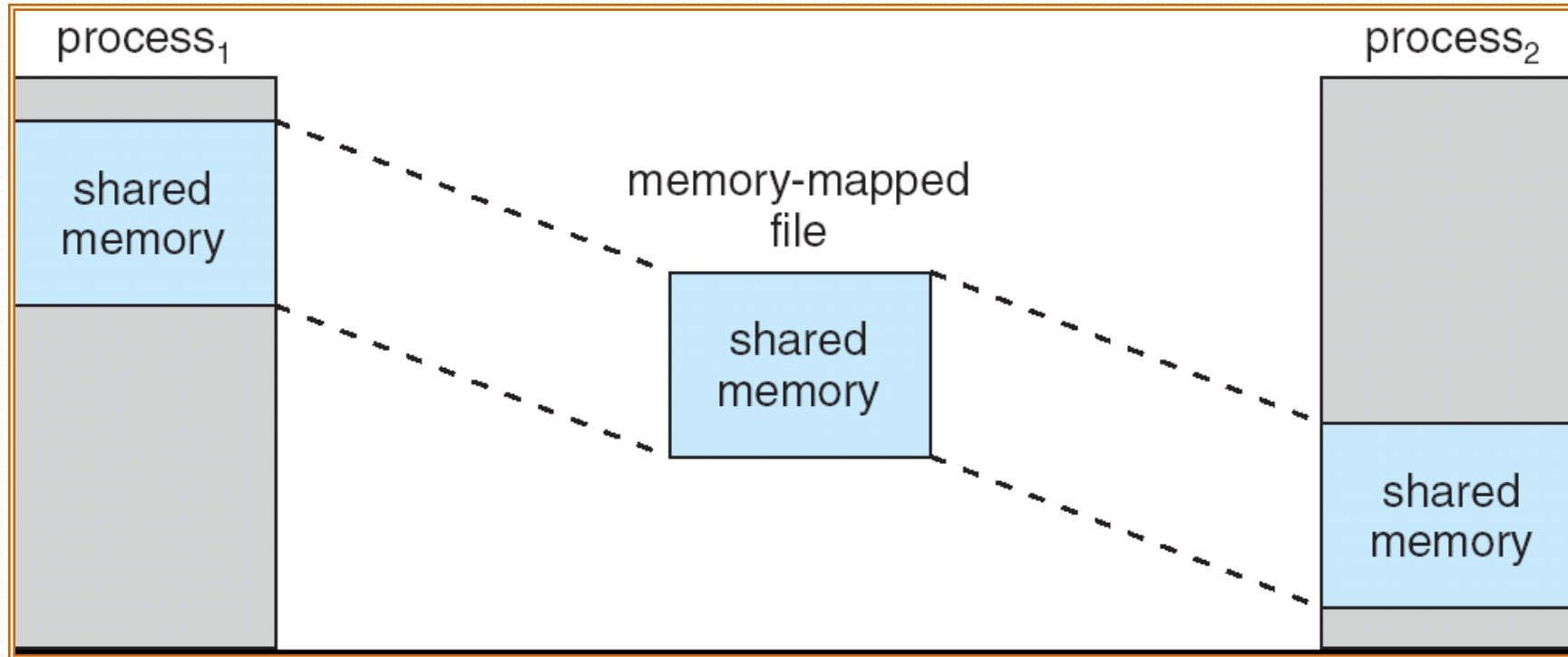


Fig. 9.23 Memory-mapped files and file sharing

Memory-Mapped Shared Memory in Windows

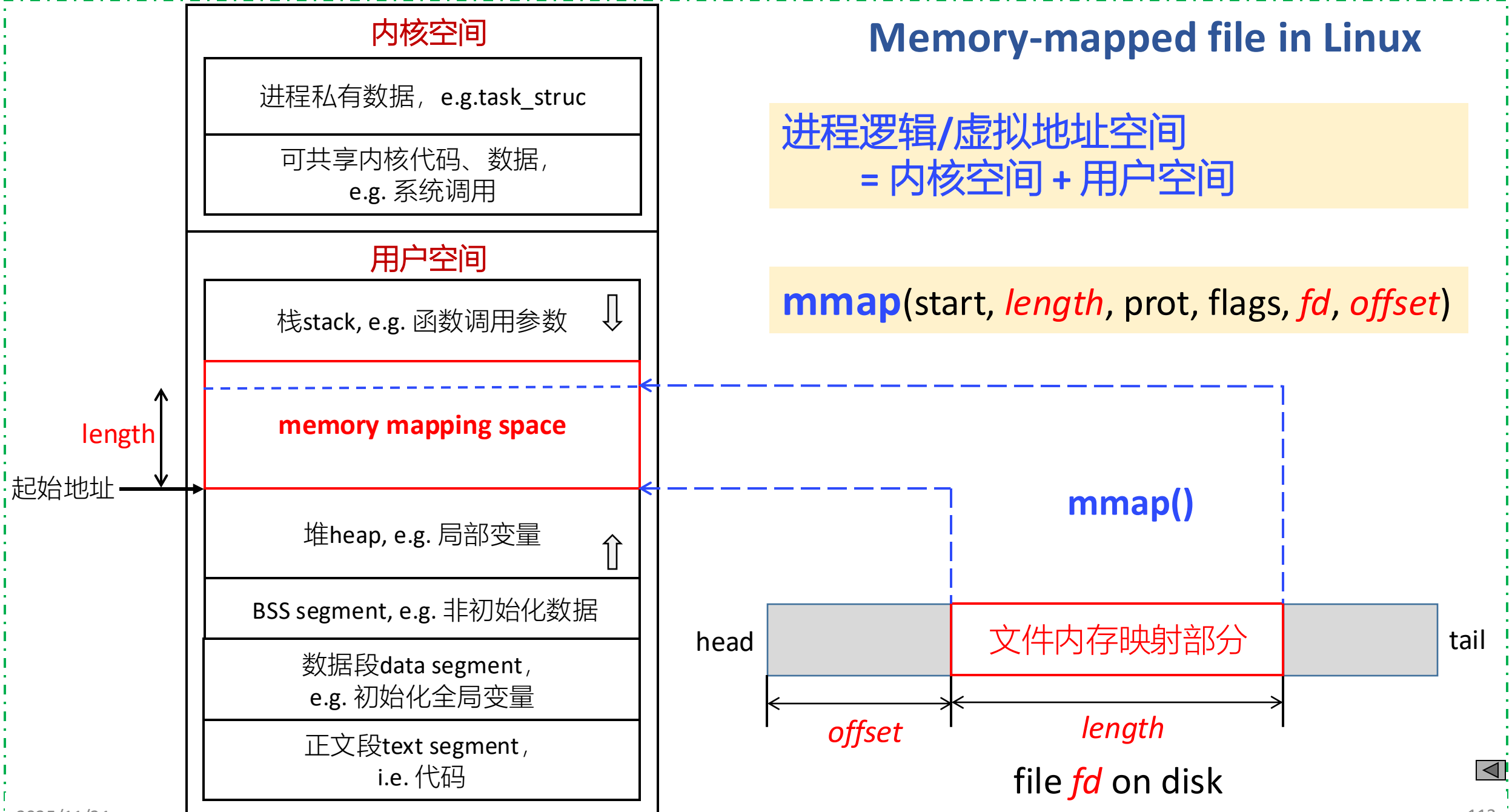


E.g. C# file-sharing-based inter-process communications

Memory-mapped file in Linux

进程逻辑/虚拟地址空间
= 内核空间 + 用户空间

`mmap(start, length, prot, flags, fd, offset)`



`mmap(start, length, prot, flags, fd, offset)`

- **start**: 映射区在物理内存中的开始地址，设置为0/null时表示由系统决定映射区起始地址
- **length**: 映射区长度。长度单位以字节为单位，不足一内存页按一内存页处理
- **prot**: 期望的内存保护标志，不能与文件的打开模式冲突
 - ▣ `PROT_EXEC, PROT_READ, PROT_WRITE`，页内容可被执行、读取、写
 - ▣ `PROT_NONE`，页不可访问
- **flags**: 指定映射对象的类型，映射选项和映射页是否可以共享
 - ▣ `MAP_PRIVATE`，建立一个写入时拷贝的私有映射，内存区域的写入不影响原文件
 - ▣ `MAP_FIXED, MAP_SHARED, MAP_DENYWRITE, ...`
- **fd**: 被映射的文件描述符
- **offset**: 被映射对象内容在文件中的起点

Memory-mapped file in Linux

```
/*-----map_normalfile1.c-----*/  
#include<sys/mman.h>  
#include<stdio.h>  
#include<sys/types.h>  
#include<sys/stat.h>  
#include<string.h>  
#include<fcntl.h>  
#include<unistd.h>  
int main()  
{  
    //
```

```

int main()
{ //Step1:
    int fd=open("test.txt", O_CREAT|O_RDWR, 0644);
    write(fd,"testbupt2020CS123456789", 23);
    //Step2:
    struct stat sbuff;
    stat("test.txt",&sbuff);
    char *share=mmap(NULL, sbuff.st_size,PROT_READ,
                     MAP_PRIVATE, fd, 0);
    if (share==MAP_FAILED) {
        perror("map error")
    }
    close(fd);
    //Step3:
    char mapbuffer[10];
    memset(mapbuffer,0,sizeof(mapbuffer));
    memcpy(mapbuffer,share+4, 10);
    //Step4:
    putc(mapbuffer);
    munmap(share, sbuff.st.size); //length is map size
}

```

Step1.创建文件
test.txt, 并写入23
个字符

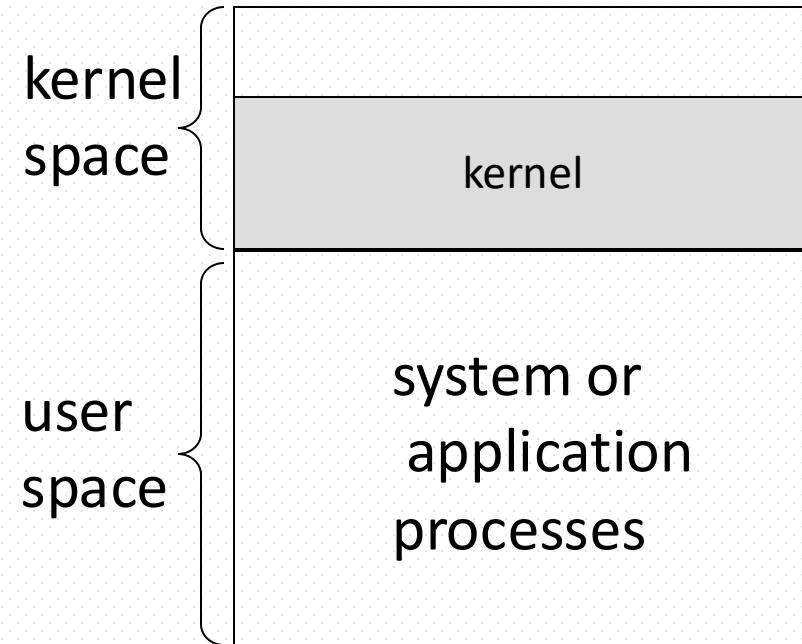
Step2.使用函数mmap,
从文件头开始, 将文件
中全部字符, 映射到进
程虚拟地址空间, 返回
被映射区的指针share;
如果映射失败, mmap()
返回MAP_FAILED

Step3-1.用memset清零mapbuffer
Step3-2.指定偏移为4, 用
memcpy从文件中读取长度为10
个字节的内容"bupt2020CS"到
mapbuffer

Step4.用putc将mapbuffer内容输出到流/终端中; 用munmap删除share处的文件对象映射

9.8 Allocating Kernel Memory

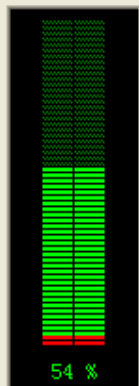
- Two categories of processes concurrently running in memory
 - ▣ kernel processes, running in kernel mode
 - ▣ other application and system processes, in user mode
- Physical memory is divided into two parts
 - ▣ one part, in kernel space, in which OS kernel resides
 - organized as pages, **kernel page tables (内核页表)** are needed, **or/and**
 - organized as contiguous memory blocks
 - ▣ the other, in user space, for application or other system processes, organized as, e.g. paging



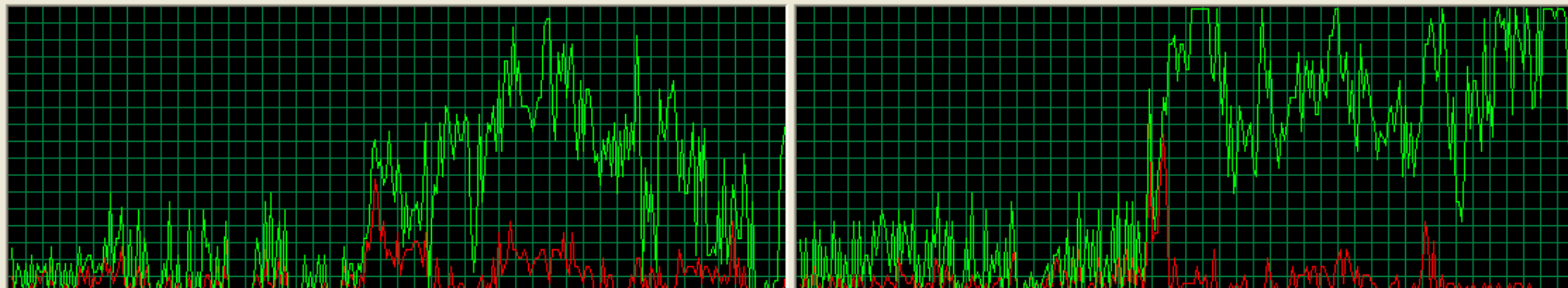
Allocating Kernel Memory

- OS kernel memory is treated differently from user memory
- E.g. process logical/virtual address space in Linux 
- In addition to allocating physical memory to processes in use space, OS has to assign physical memory to
 - ▣ OS kernel processes running in kernel space
 - e.g. kernel process for memory management
 - ▣ kernel data structures with large size
 - e.g. four-level **page tables** for 64-bit process in Linux, maybe up to $2^{(48-12)} * 4$ bytes
 - ▣ kernel data structures with small size, i.e. small object data structure
 - e.g. **dentry** for the directory entry in Linux, **0.19K**=195 bytes in size
<file name, #inode>

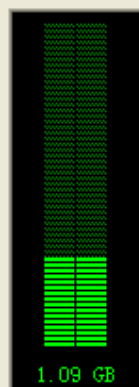
CPU 使用



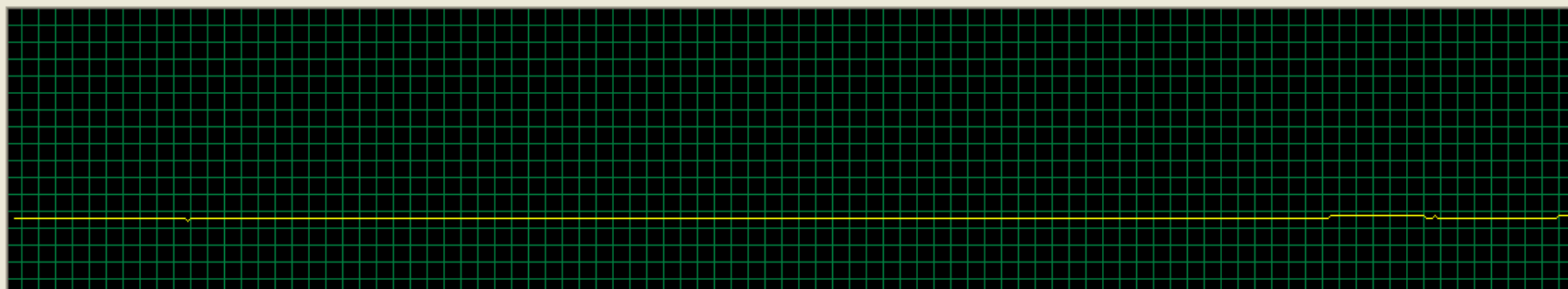
CPU 使用记录



PF 使用率



页面文件使用记录



总数

句柄数 17780
线程数 880
进程数 72

物理内存 (K)

总数 1022244
可用数 157552
系统缓存 244080

认可用量 (K)

总数 1143852
限制 4097108
峰值 1164392

核心内存 (K)

总数 166440
分页数 89064
未分页 77376

Allocating Kernel Memory

- Memory requested by the kernel is often allocated from a free-memory pool (空闲内存池) residing in kernel space
 - ▣ kernel requests memory for structures of varying sizes, e.g.
 - when a process is created, OS requests memory to hold its PCB/task_struct in Linux, page table
 - when a process defines a new semaphore, the semaphore is stored in kernel space
 - message-passing communication, mailbox
 - ▣ some kernel memory needs to be contiguous
 - i.e. for I/O device buffer

Allocating Kernel Memory

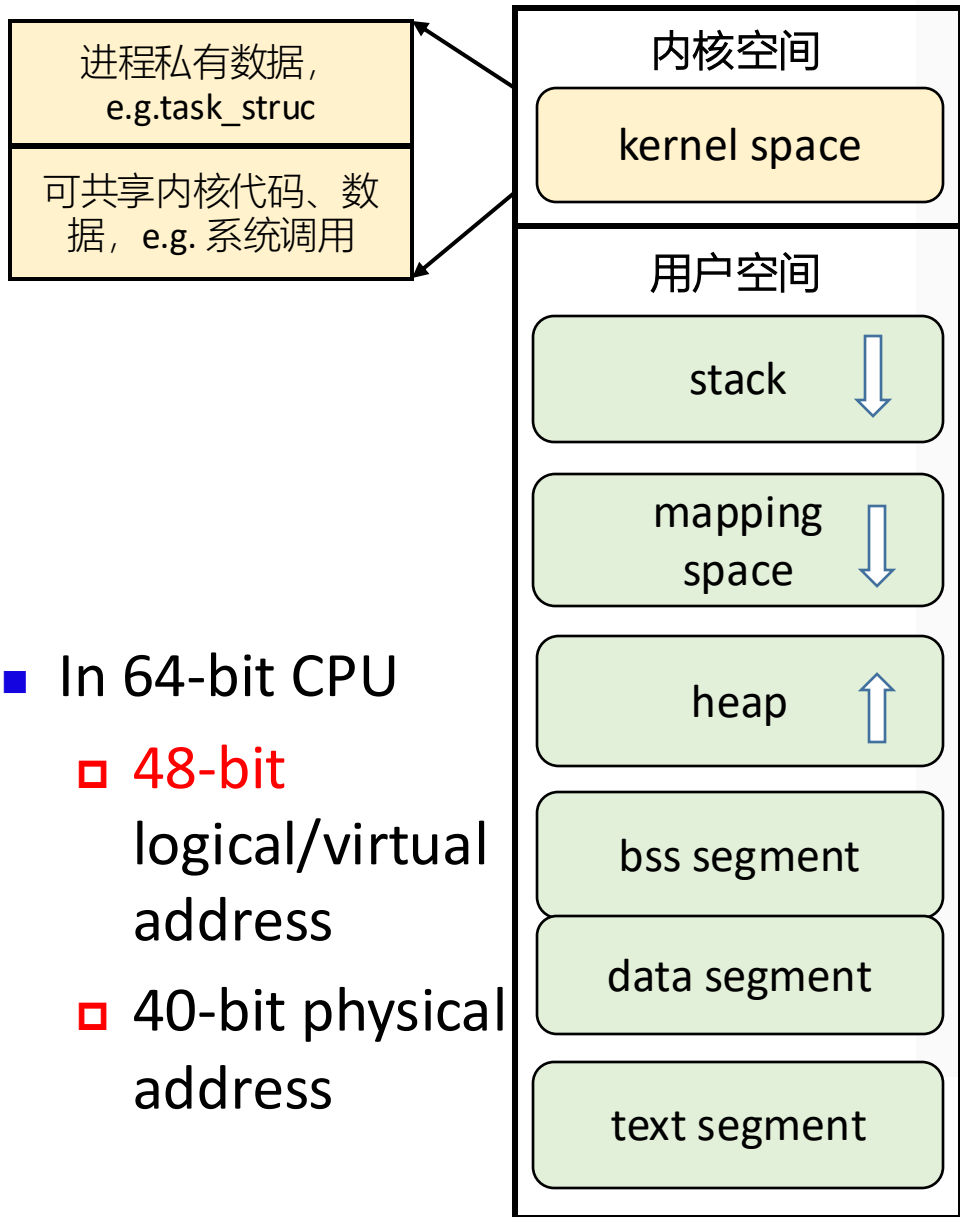
- The size of kernel memory requested by processes is not fixed
- So fixed-sized partitions of kernel space, such as paging, is also not efficient for kernel space management (未必?)
- Many OS do not subject kernel code or data to the paging system??!! (未必?)
 - kernel page table in Linux
- Two schemes for allocation of kernel memory, similar to MVT
 - **buddy system**, used in Unix
 - buddy + slab allocator, used in Linux

Linux内核页表

- Linux进程的逻辑地址/虚拟地址空间由内核空间、用户空间组成，内核空间为OS内核代码和数据
 - e.g. OS创建进程时，在内核空间为新创建的进程分配进程控制块task_struct
- 内核空间逻辑上分为连续的页，e.g. page size=4K or 1M in Linux，通过内核页表将虚拟地址空间映射至物理内存空间
 - 一般将内核中不同区域的连续的虚拟地址空间（中的连续pages）映射到连续的物理地址空间，两者位置上相差offset，利用页表，直接将虚拟地址加上offset得到物理地址，
 - 系统创建时，建立这种映射关系
 - 类似于段表
- E.g.1 In 64-bit Linux, the kernel space consists of
 - 线性映射区, KASAN shadow, BPF JIT region, modules, vmalloc, fixed mapping, PCI IO space, vmemmap
 - 段内连续，段间不连续

Linux as example

逻辑地址空间

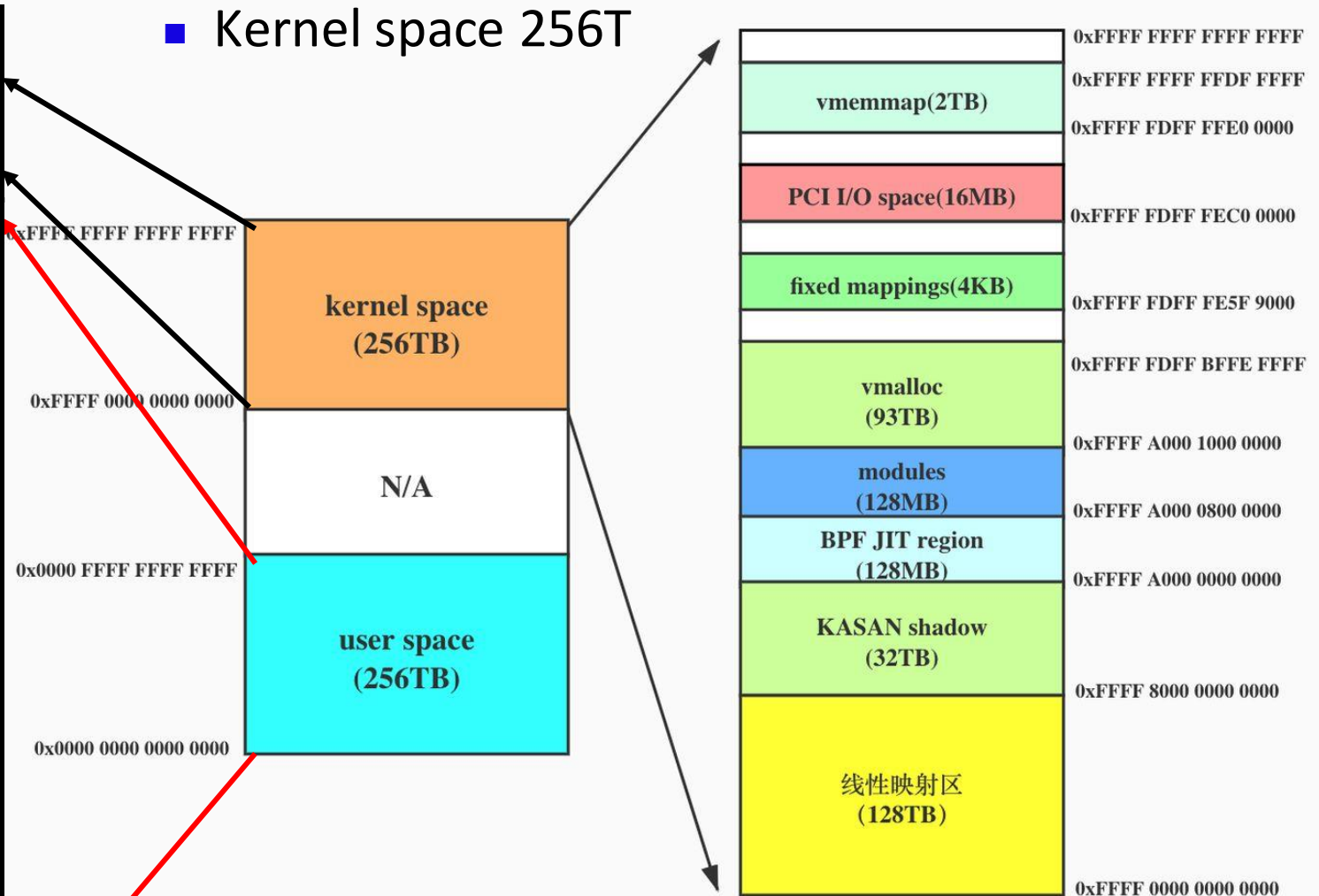


In 64-bit CPU

- 48-bit logical/virtual address
- 40-bit physical address

User space 256T

Kernel space 256T



- E.g. 2 直接存取访问DMA控制器内存分配
 - ▣ x-86下，DMA需要配置内存地址，采用物理地址pa
 - ▣ DMA driver通过内核内存分配申请，得到一块物理内存，其地址为虚拟地址va，与物理地址pa间关系为： $va = pa + offset$
- E.g. 3 32-bit Linux
 - ▣ **内核页表**在系统初始化时创建，记录了4G的逻辑/虚拟地址空间中从0xC000,0000到0xffff,ffff的1G内核空间的物理地址分布
 - ▣ 内核页表中每一个页表项描述了1M大小的一段虚拟地址空间的物理地址，注：内核页size为1M
 - ▣ 用户进程创建时，将内核页表复制到新创建的用户页表中

- 为提高OS内核运行效率，内核空间一般不做swap，一般内核代码全部加载至内存，也没有page fault（注：vmalloc可能例外）
- 内核空间映射至连续的物理空间

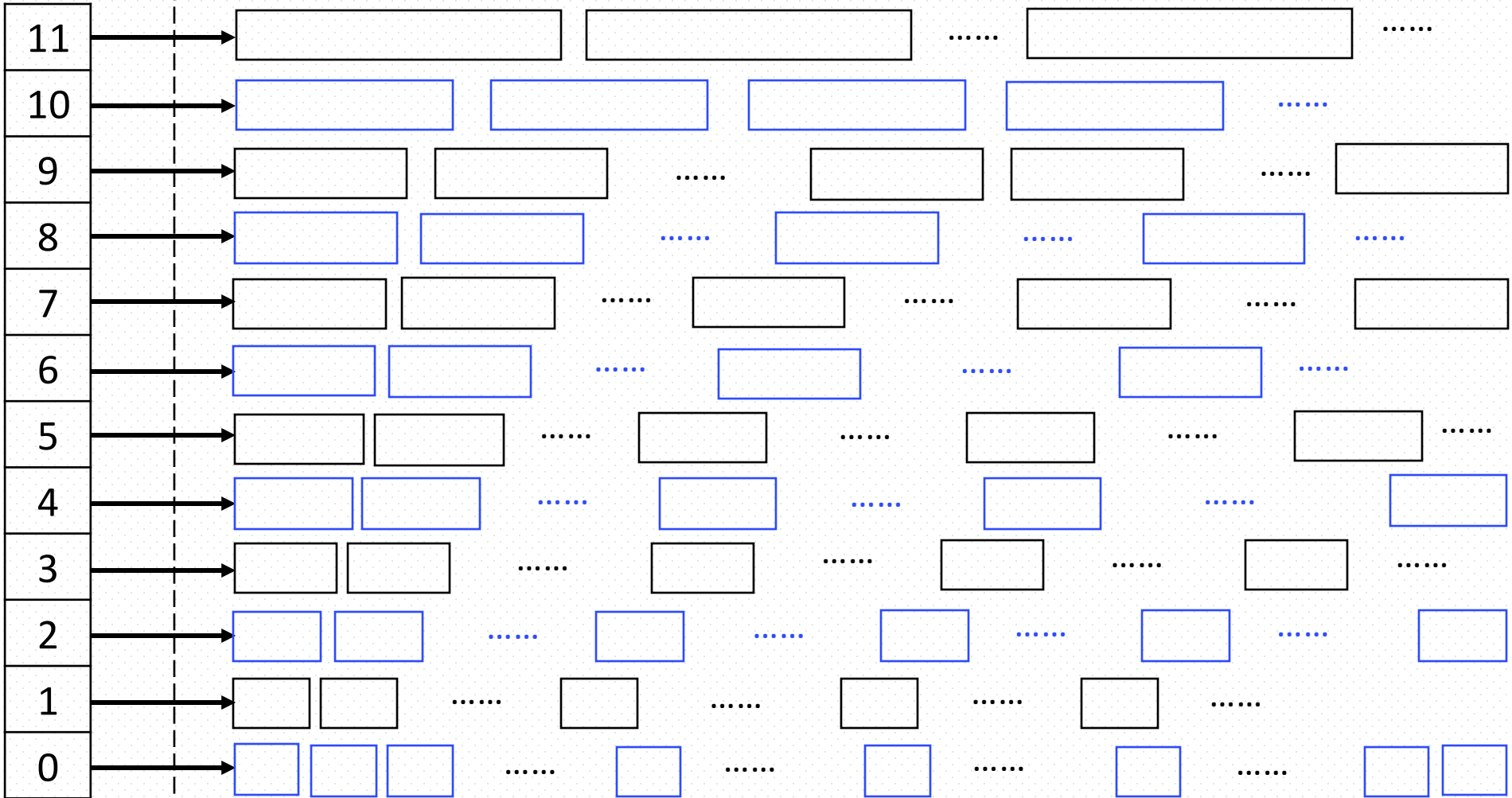
Buddy System

- 用于为OS内核分配由连续物理页/frames组成的、大小不一的连续内存区域
- 物理内存划分为由连续物理页组成的chunk，以chunk为基本分配单位
- chunk按照其大小，分为不同的阶order，如 $order=0,1,2,\dots,max_value$, $max_value=11$
 - 对第n阶的chunk，其大小为 2^n page size，由 2^n 个连续的物理页frames组成
 - chunk的最大size（即最大阶）可由开发者设置，即设置 max_value 值，其默认值为11
 - e.g. frame size=4KB, chunk size可以是 2^0*4K , $2^1*4K=8K$, $16K,\dots$, $2^{11}*4K=2^{13}K=8M$
- 满足以下条件的两个n阶chunk互为伙伴
 - 两者是相邻的，即物理地址是连续的
 - chunk中第1个物理页的页号是 2^n 的整数倍
 - 当两者合并成 $(n+1)$ 阶chunk后，第1页的物理页号必须是 2^{n+1} 的整数倍
- Using **power-of-2 allocator** to allocate memory from **different** fixed-size chunks/**segments** consisting of **physically-contiguous pages/frames**, each chunk/**segment** is in size of power of 2 bytes, e.g. 4K, 8K, 16K, 32K, 64K, 128K,..., if frame size is 4KB

Buddy System

physical memory, e.g. 32G

order: n



size: 2^n pages

2^{11}	8M
2^{10}	4M
2^9	2M
2^8	1M
2^7	512K
2^6	256K
2^5	128K
2^4	64K
2^3	32K
2^2	16K
2^1	8K
2^0	4K

Buddy System

- For a memory request of m frames, allocate a chunk that is in size of 2^n physically-contiguous frames, where

$$2^{n-1} < m \leq 2^n$$

- e.g. for the request of 75K, $64 < 75 \leq 128$, $n=7$,

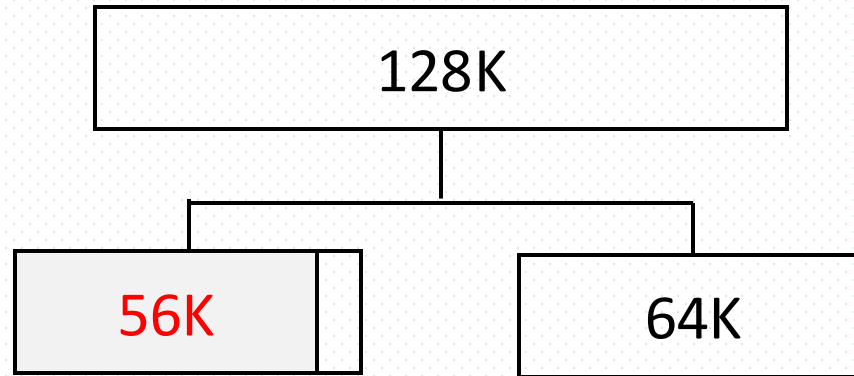
allocate 128 ($= 2^7$)K

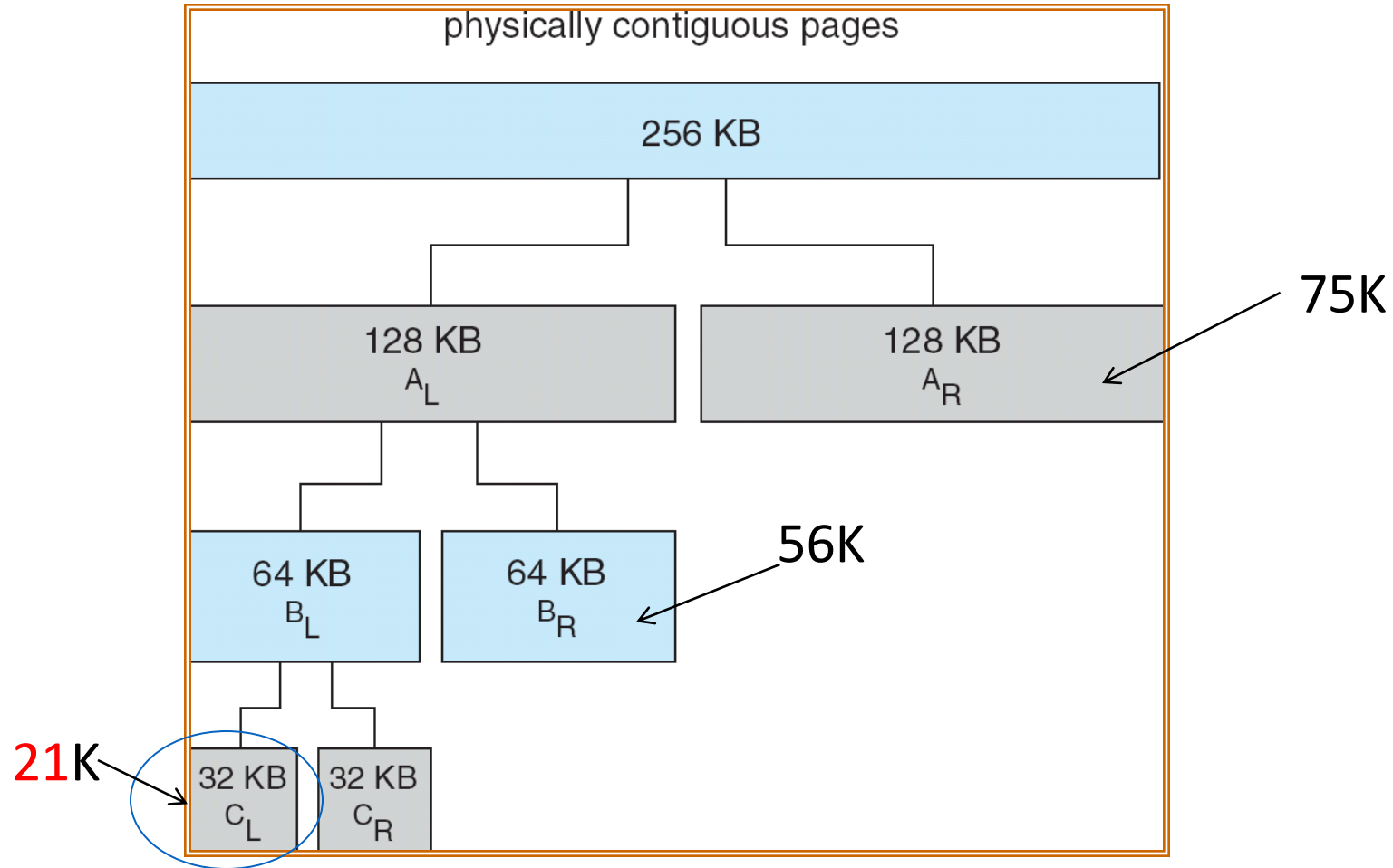
- 如果找不到满足上述条件的空闲chunk，只有 $2^{n-1} > m$ 的空闲chunk，将空闲chunk对半切分，直到找到满足 $2^{n-1} < m \leq 2^n$ 的 n 阶空闲chunk
- 切分后的两个小的chunk互为buddies，1个子chunk用于分配，另一个空闲
- 1个chunk被释放后，检查其buddies是否也是空闲的，如果是，这对空闲buddies合并为高1阶的空闲chunk

Buddy System

- When smaller allocation needed than the current available segments/chunks, the chunk split into two buddies of next-lower **power of 2**, continue until appropriate sized chunk available
 - e.g. for a free chunk of 128K, to meet the request of **56K**, the chunk is divided into two free chunks of 64 K

— 互为伙伴

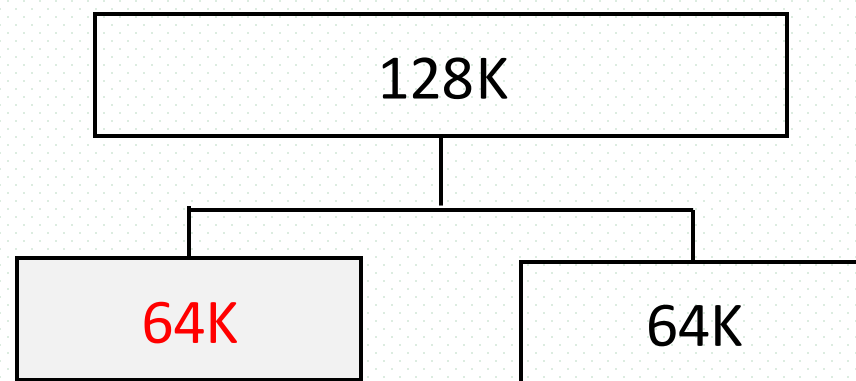




- A **21** KB request for PCB is satisfied by the segment C_L
- When the kernel releases C_L , the system coalesce into a 64-KB segment, ultimately we can end up with the original 256-KB segment

Fragments in Buddy System

- 伙伴算法中，对n阶chunk，最大碎片、最小碎片、平均碎片？
 - ▣ n阶chunk，大小为 $2^n \times \text{page size}$ ，e.g. $n=5$, $\text{page size}=4\text{K}$, $2^5 \times 4\text{K}=128\text{K}$
- 最小碎片
 - ▣ 1, e.g. $n=5$, a process in size of $(2^5 \times 4\text{K} - 1 = 127\text{K})$ is allocated to a 128K-byte chunk
- 最大碎片
 - ▣ $(2^{n-1} - 1) \times \text{page size}$, a process in size of $(2^{5-1} \times 4\text{K} + 1 = 65\text{K})$ is allocated to a 128K-byte chunk
- 平均碎片
 - ▣ $[(2^{n-1} - 1) + 1] / 2 = 2^{n-2}$



Example

- For a kernel space with the size of $1M = 2^{20}$ bytes, the page size is $4K$, then there are 9 chunks/segments splitted by the buddy system, and these segments can be of the size $4K = 2^{12}$, ..., 2^{19} , 2^{20} bytes respectively
 - ▣ the smallest size of segments equals to $4K$, i.e. the page size

	0	128K	256K	384K	512K	640K	768K	896K	1M	holes 数目
初始 状态										1
申请70	A	128		256		512				3
申请35	A	B	64	256		512				3
申请80	A	B	64	C	128	512				3
回收A	128	B	64	C	128	512				4
申请60	128	B	D	C	128	512				3
回收B	128	<u>64</u>	<u>D</u>	C	128	512				4
回收D	256			<u>C</u>	<u>128</u>	512				3
回收C	1024									1

内核空间分布

申请过程： A: 70K; B: 35K; C: 80K; D: 60K

Slab Allocator

- Why slab allocator needed?
 - ▣ buddy系统分配单位是1个物理内存页/页框frame（4KB, or 1M in size）
 - ▣ 很多情况下，内核数据结构（如PCB, task_struct in Linux）大都为几十或数百bytes，采用buddy分配直接为这些数据结构分配1个frame，导致严重内部碎片
 - ▣ SLAB allocator
 - 与buddy allocator配合，从buddy allocator分配的chunk/frame中为内核数据结构分配小块内存
- SLAB allocator
 - ▣ firstly introduced in Sun Solaris 2.4 in 1990+
 - ▣ in 2000+, improved as SLUB and SLOB
 - ▣ SLAB, SLUB and SLOB are collectively called SLAB
- SLAB is the default memory allocator in **Linux-2.6.23** and its successors

Slab Allocator

- 目标
 - ▣ 配合buddy allocator, 满足OS频繁为内核小对象（数据结构）分配内存的需求
 - ▣ small object data structure in kernel, e.g. in Linux
 - task_struct, i.e. process control block
 - page table entry
 - semaphore
 - message
 - dentry, inode for file management
 - vm_area_struct for memory management

Slab Allocator

- A **slab** is one or more physically contiguous pages/frames, allocated by the **buddy** allocator
 - ▣ consist of one frame or multiple contiguous pages/frames

//一个page frame或多个连续page frame组成, 通常为一个page, page size is, e.g.
 $2^{12}\text{bytes}=4\text{K}$

- A slab is divided into small **objects** with a fixed size 2^n bytes, and of same type
 - ▣ each object in slab is dedicated to a particular kernel small object data structure
 - ▣ e.g. slab is 4K in size, consists of $16=2^4$ objects, the object size is $2^n = 2^8$ bytes, and this object is for **dentry** in file management
 - $n=8, 3 \leq n < 12, 8 \text{ bytes} \leq \text{object size} = 2^n = 2^8 = 256 \text{ bytes} < \text{page size} = 4\text{k}$

//每种固定大小的小对象内存块object分配给一个特定的内核小对象数据结构

 - e.g. object with size of $2^8=256\text{bytes}$ is allocated to **dentry** (目录项)

Slab Allocator

cache-slab-object

- A **cache** consists of one or more **slabs**, and each slab owns small **objects** of the same size
- 分配给同一种内核小对象数据结构的多个slab组成一个内存资源池**cache**
 - ▣ single cache for each unique kernel **data structure**
 - /*每种内核数据结构对应一个cache
 - ▣ e.g. in Linux,
 - caches for **dentry**, **inode** in file management
 - cache for **vm_area_struct** in memory management
 - ▣ each cache filled with slabs and **objects** – instantiations of the data structure

Slab Allocator

- When cache created, filled with objects marked as *free*
 - /cache刚创建, 其中的slab、 slab还没有分配给小对象数据结构, object标记为空free
- When a small object data structure is stored in an object, the object is marked as *used*
 - /object分配后, 标记为used
- If slab is full of used objects, next object allocated from empty slab
 - if no empty slabs, a new slab is allocated by buddy system
- Benefits include no **fragmentation** (?), fast memory request satisfaction

cache-slab-object

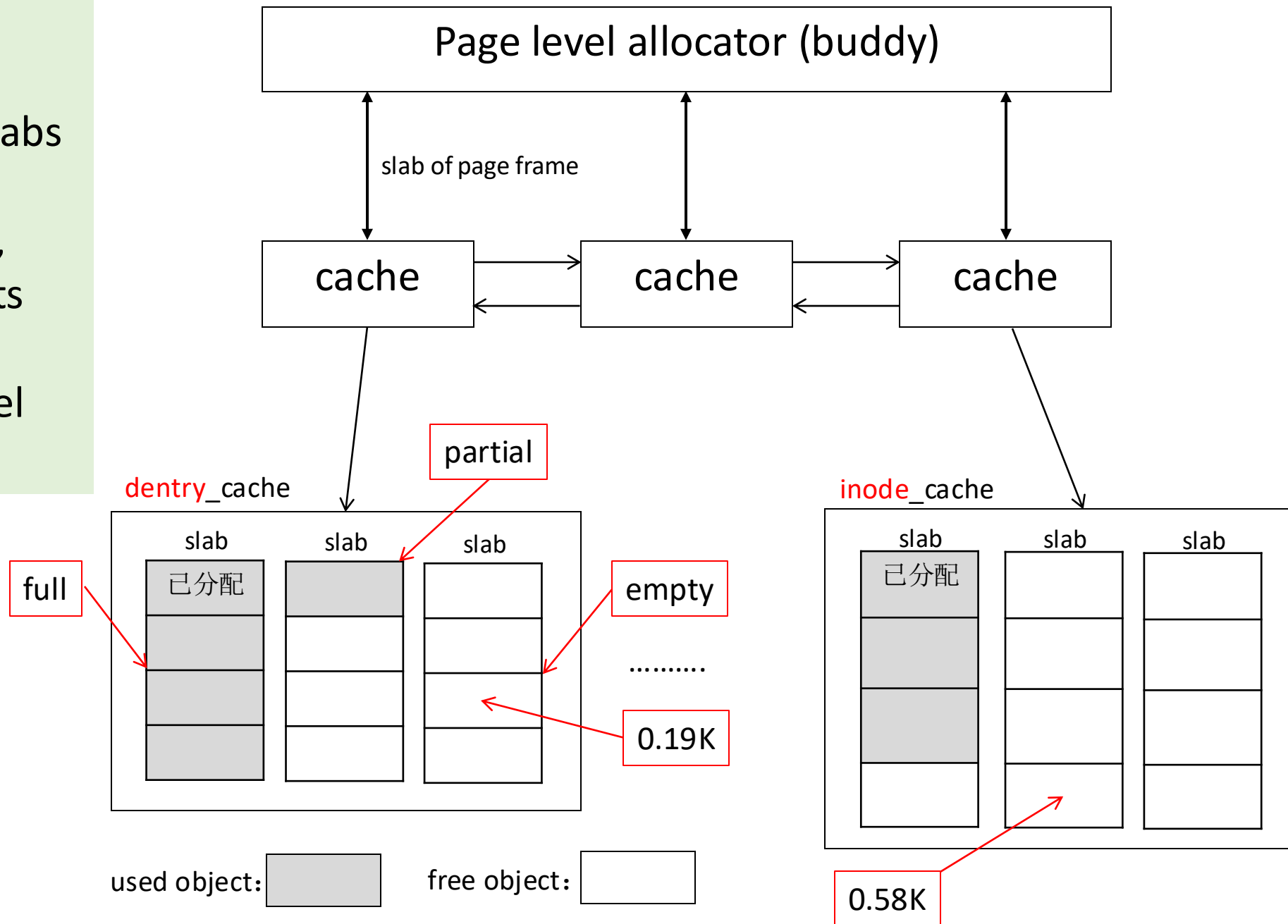
cache: set/buffer of slabs

slab:

allocated by buddy,
consisting of objects

object:

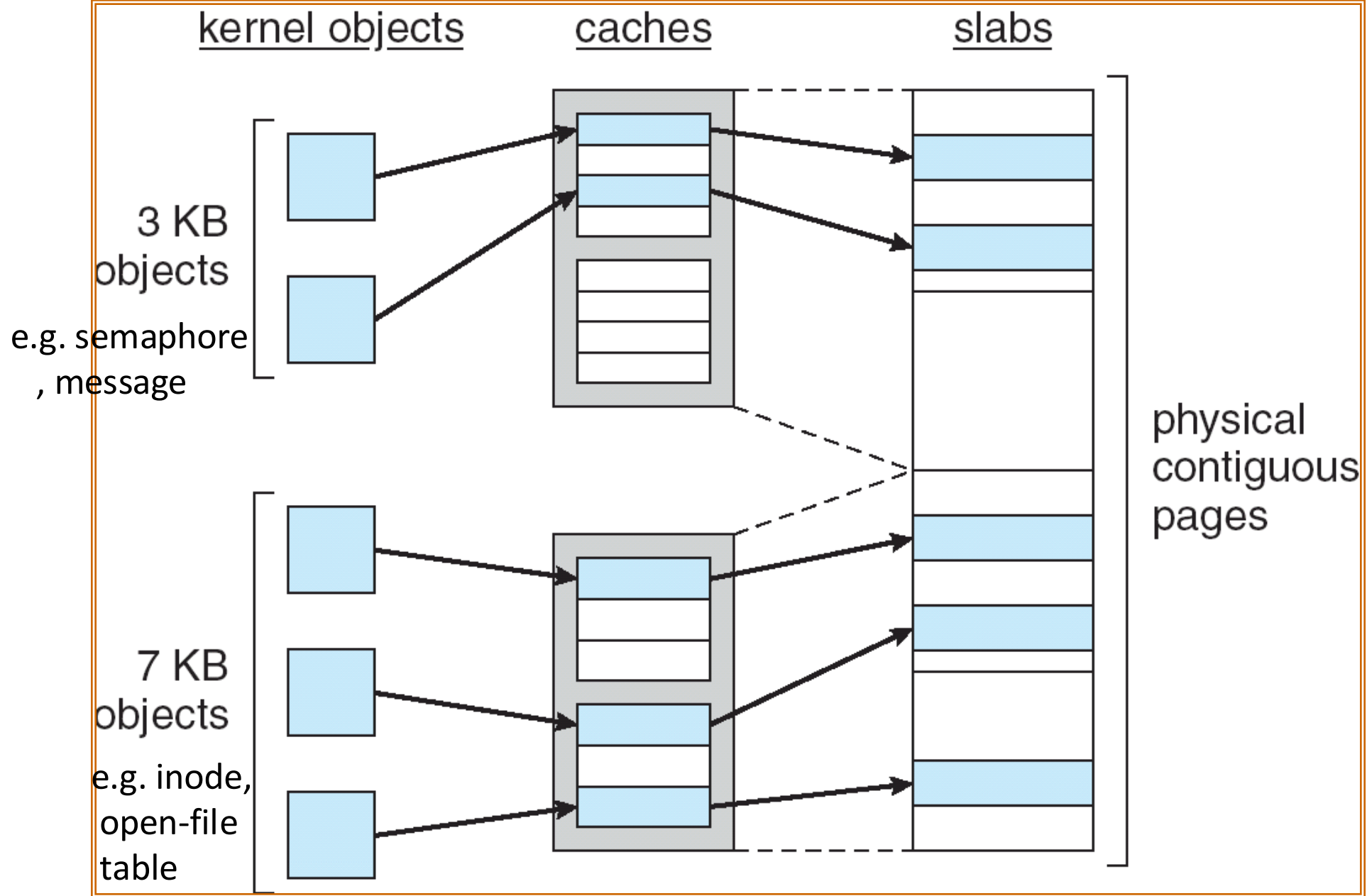
dedicated to kernel
data structure



Slab Allocator

- In Linux, cache由Linux中的结构体**struc_kmem_cache**描述

```
struct kmem_cache {  
    unsigned int gfporder;    /* order of pages per slab (2^n) */  
    gfp_t allocflags;         /* force GFP flags */  
  
    unsigned int num;         /* objects per slab */  
    int object_size;          /* object大小, e.g. 0.19K */  
    unsigned int colour_off;  /* colour offset */  
    size_t colour;           /* cache colouring range */  
  
    struct list_head list;    /* cache creation & removal */  
}
```



通过"slabtop"命令查看当前系统的slab分配情况:

dentry cache

- 1182 slabs, 24822 objects, 21 obj/slab

- obj size:
0.19K=195 bytes

- slab size:
 $0.19 \times 21 \approx 4\text{K}$

- cache size:
 $0.19 \times 21 \times 1182 \approx 4716\text{K}$

```
Active / Total Objects (% used) : 164309 / 167451 (98.1%)
Active / Total Slabs (% used)   : 5779 / 5779 (100.0%)
Active / Total Caches (% used)  : 73 / 101 (72.3%)
Active / Total Size (% used)    : 36770.93K / 38226.50K (96.2%)
Minimum / Average / Maximum Object : 0.01K / 0.23K / 8.00K
```

OBJS	ACTIVE	USE	OBJ	SIZE	SLABS	OBJ/SLAB	CACHE	SIZE	NAME
44811	43905	97%	0.10K	1149	39	4596K	buffer_head		
24822	24822	100%	0.19K	1182	21	4728K	dentry		
13090	13090	100%	0.12K	385	34	1540K	kernfs_node_cache		
10506	10506	100%	0.04K	103	102	412K	selinux_inode_security		
9035	9035	100%	0.58K	695	13	5560K	inode_cache		
8160	8160	100%	0.04K	80	102	320K	ext4_extent_status		
7680	7274	94%	0.06K	120	64	480K	kmalloc-64		
6960	6898	99%	1.00K	435	16	6960K	ext4_inode_cache		
6035	6035	100%	0.05K	71	85	284K	shared_policy_node		
3584	3584	100%	0.02K	14	256	56K	kmalloc-16		
3640	3566	97%	0.57K	260	14	2080K	radix_tree_node		
3528	3313	93%	0.21K	196	18	784K	vm_area_struct		

- 695 slabs, 9035 objects, 打开9035个文件, 13 obj/slab

- obj size: 0.58K=594 bytes

- slab size $0.58 \times 13 \approx 6.5\text{K}$,

- total size: $0.58 \times 13 \times 695 \approx 5240\text{K}$

inode_cache

OBJS	ACTIVE	USE	OBJ SIZE	SLABS	OBJ/SLAB	CACHE SIZE	NAME
44811	43905	97%	0.10K	1149	39	4596K	buffer_head
24822	24822	100%	0.19K	1182	21	4728K	dentry
13090	13090	100%	0.12K	385	34	1540K	kernfs_node_cache
10506	10506	100%	0.04K	103	102	412K	selinux_inode_security
9035	9035	100%	0.58K	695	13	5560K	inode_cache
8160	8160	100%	0.04K	80	102	320K	ext4_extent_status
7680	7274	94%	0.06K	120	64	480K	kmalloc-64
6960	6898	99%	1.00K	435	16	6960K	ext4_inode_cache
6035	6035	100%	0.05K	71	85	284K	shared_policy_node
3584	3584	100%	0.02K	14	256	56K	kmalloc-16
3640	3566	97%	0.57K	260	14	2080K	radix_tree_node
3528	3313	93%	0.21K	196	18	784K	vm_area_struct
3072	3072	100%	0.01K	6	512	24K	kmalloc-8
3828	3000	78%	0.66K	319	12	2552K	proc_inode_cache
2091	2091	100%	0.08K	41	51	164K	anon_vma
1680	1680	100%	0.07K	30	56	120K	ext4_io_end
1696	1339	78%	0.25K	106	16	424K	kmalloc-256
1281	1281	100%	0.19K	61	21	244K	kmalloc-192
1176	1176	100%	0.07K	21	56	84K	avc_node
1152	1152	100%	0.03K	9	128	36K	kmalloc-32
1008	1008	100%	0.09K	24	42	96K	kmalloc-96
976	930	95%	1.00K	61	16	976K	kmalloc-1024
780	780	100%	0.66K	65	12	520K	shmem_inode_cache
768	768	100%	0.12K	24	32	96K	kmalloc-128

2091	2091	100%	0.08K	41	51	164K	anon_vma
1680	1680	100%	0.07K	30	56	120K	ext4_io_end
1696	1339	78%	0.25K	106	16	424K	kmalloc-256
1281	1281	100%	0.19K	61	21	244K	kmalloc-192
1176	1176	100%	0.07K	21	56	84K	avc_node
1152	1152	100%	0.03K	9	128	36K	kmalloc-32
1008	1008	100%	0.09K	24	42	96K	kmalloc-96
976	930	95%	1.00K	61	16	976K	kmalloc-1024
780	780	100%	0.66K	65	12	520K	shmem_inode_cache
768	768	100%	0.12K	24	32	96K	kmalloc-128
612	612	100%	0.11K	17	36	68K	jbd2_journal_head
640	547	85%	0.50K	40	16	320K	kmalloc-512
330	330	100%	0.13K	11	30	44K	ext4_groupinfo_4k
416	305	73%	2.00K	26	16	832K	kmalloc-2048

OBJS	ACTIVE	USE	OBJ	SIZE	SLABS	OBJ/SLAB	CACHE	SIZE	NAME
------	--------	-----	-----	------	-------	----------	-------	------	------

294	294	100%	0.19K	14	21	56K	cred_jar
256	256	100%	0.02K	1	256	4K	jbd2_revoke_table_s
228	228	100%	0.62K	19	12	152K	sock_inode_cache
180	180	100%	2.06K	12	15	384K	idr_layer_cache
180	180	100%	0.11K	5	36	20K	task_delay_info
170	170	100%	0.02K	1	170	4K	fsnotify_mark_connector
168	168	100%	0.38K	8	21	64K	mnt_cache
165	165	100%	1.06K	11	15	176K	dmaengine-unmap-128
160	160	100%	0.12K	5	32	20K	pid
156	156	100%	2.62K	13	12	416K	task_xstate

9.9 Other Considerations[略]

- Prepaging
- Page Size
- TLB Reach
- Inverted Page Tables
- Program Structure
- I/O Interlock and Page Locking

Prepaging

- When a new process is started, or a swapped-out process is restarted, all its pages are on the disk, and each must be brought in by its own page fault, resulting in the large number of page faults that occur when a process is started or swapped in
- Prepaging
 - ▣ attempt to prevent this high level of initial paging
 - ▣ the strategy is to bring into memory at one time all the pages that will be needed
 - ▣ some operating systems—notably Solaris—prepage the page frames for small files
- In a system using the working-set model
 - ▣ keep with each process a list of the pages in its working set
 - ▣ When a process is suspended due to an I/O wait or a lack of free frames, remember the working set for that process
 - ▣ When the process is to be resumed (because I/O has finished or enough free frames have become available), bring back into memory its entire working set before restarting the process

Example

- For improving system performance, when OS loads a program and creates its corresponding process, the first n pages in the process virtual address space, i.e. page 0, 1,..., $n-1$, are often successively loaded into n frames that are allocated to the process by prepaging.
- In a demand-paging system, a process is allocated with 4 frames , and then the number of the frames will not be changed. The size of the page/frame is 4KB. The replacement is local replacement (each process selects from only its own set of allocated frames)
- When the process is executed, the following logical address sequence (in hexadecimal values) are visited in the first 20us:
 - ▣ 0x000 F00, 0x003 6D0, 0x002 2E0, 0x004 2A0, 0x006 6B0, 0x005 4E4,
0x007 204, 0x006 F40, 0x002 140, 0x003 524, 0x004 5F0, 0x003 004,

Example

- Answer the following questions:
 - ▣ (1) What is the reference string?
 - ▣ (2) Illustrate the replacement procedure by a figure for LRU replacement algorithms. How many page faults and page replacements would occur for the LRU replacement algorithms?
- (1) reference string
 - ▣ page size is 4K, in a 24-bit logical address, e.g. the low 12 bits represent page offset, and high 12 bits contain page number
 - e.g. in 0x000 F00, the page number is 000.
 - ▣ the string is
0,3,2,4,6,5, 7,6,2,3,4,3

Example

- (2) page fault and page replacement



0, 3, 2, 4, 6, 5, 7, 6, 2, 3, 4, 3

0	0	0	0	0	0	5	5	5	5	3	3	3
1	1	1	1	4	4	4	4	4	2	2	2	2
2	2	2	2	2	2	2	7	7	7	7	4	4
3	3	3	3	3	6	6	6	6	6	6	6	6

prepaging

- 7 page faults, 7 page replacement

Page Size

- Page size vs internal fragment, page table size (the number of page table entries)
 - ▣ page size $\uparrow \rightarrow$ fragment $\uparrow$, page table size $\downarrow$
 - ▣ page size $\downarrow \rightarrow$ fragment $\downarrow$, page table size $\uparrow$
- Optimal page size
 - ▣ refer to Example 3 Optimal page size in “操作系统概念（英文）——8- Memory Management-例题与作业答案”
- Page size vs time required to read or write a page (I/O cost for a page), the number of page faults
 - ▣ page size $\uparrow \rightarrow$ time/cost $\uparrow$, page fault occurring $\downarrow$
 - page size大, I/O系统读写1 pages耗时更多; 但1个page中包含更多指令/数据, CPU访问的指令/数据不在page中导致page fault的概率小
 - ▣ page size $\downarrow \rightarrow$ time/cost $\downarrow$, page fault occurring $\uparrow$

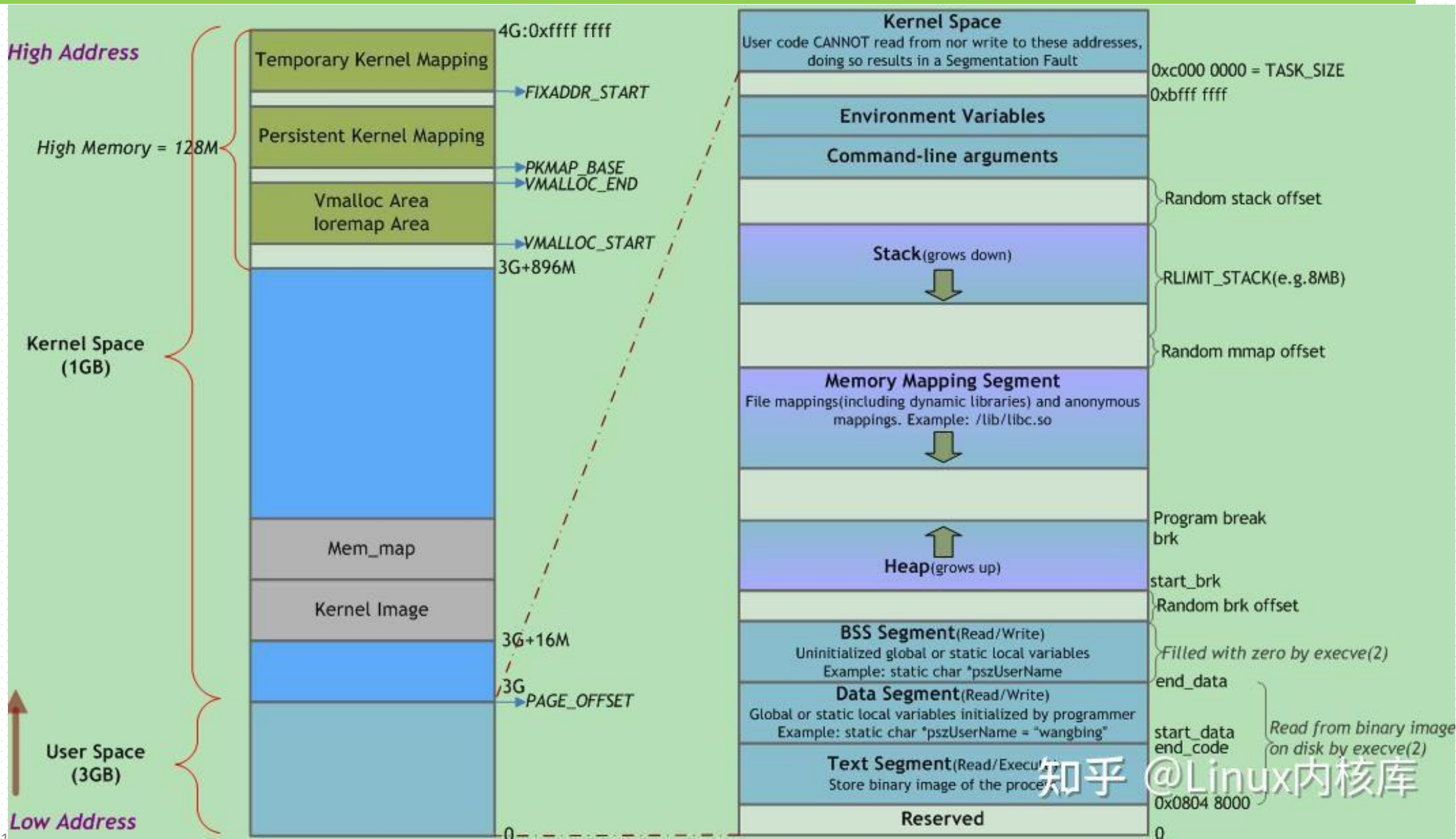
Appendix 9-1 COW in Process Creation

- 进程的逻辑/虚拟地址空间由可执行代码、（私有）堆栈、数据（共享、非共享）、共享内存区（例如用于进程间通信）、共享库等几部分构成。进程的地址空间对应了一定的存储区域
- 子进程初始创建时，通过页表拷贝复制父进程的页表——子进程页表设置成与父进程页表内容一样，从而将子进程的地址空间完全指向父进程的地址空间；子进程同时也复制了父进程的地址空间结构，但各自地址空间内的内容可能发生变化
- 优点
 - ▣ 子进程初始化时，不需要OS为子进程分配新的存储空间
 - ▣ 可以快速创建子进程

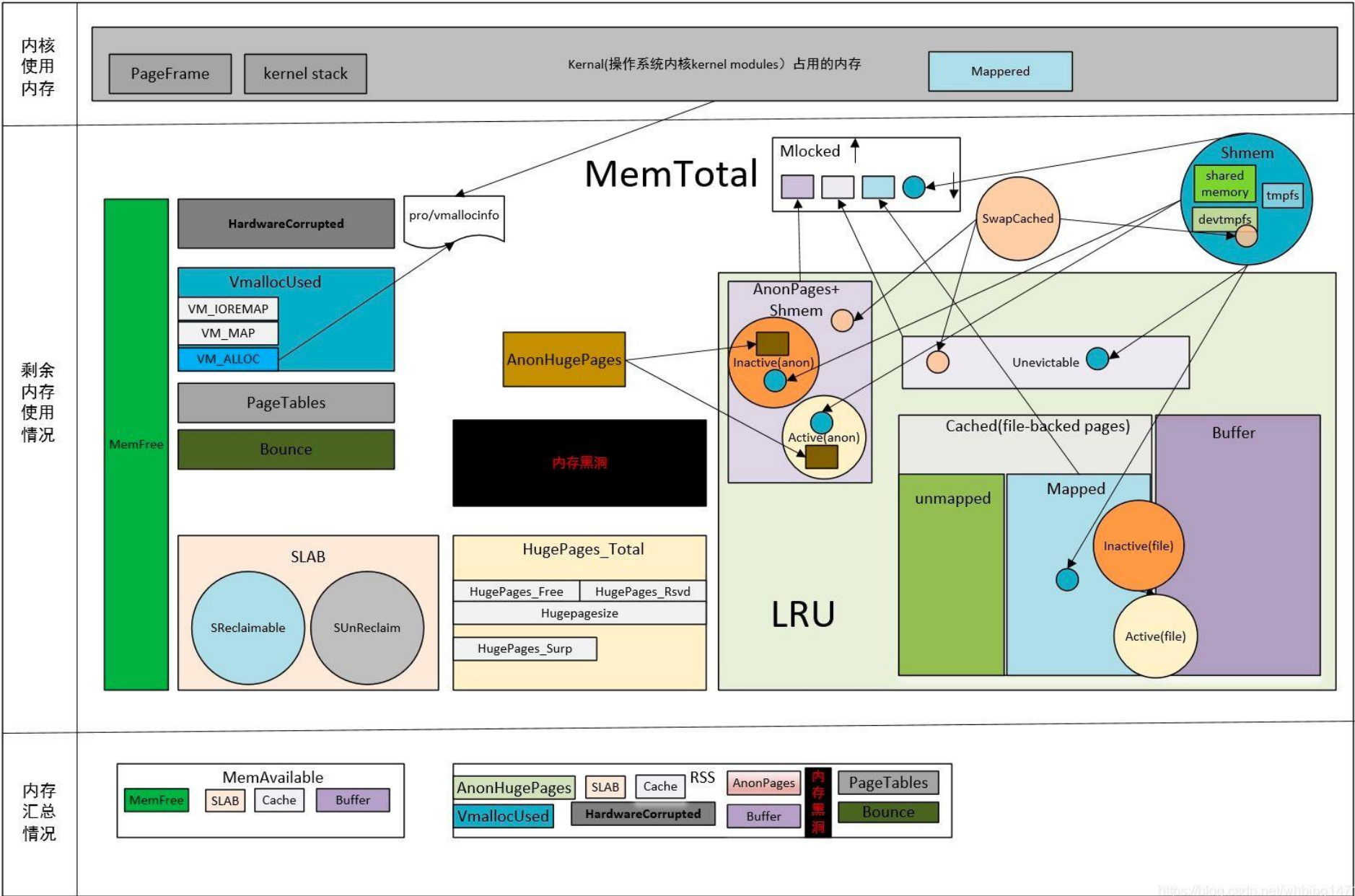
Appendix 9-1 COW in Process Creation

- 在子进程所拷贝的父进程地址空间中，某些页是父进程所独有、非共享的，如父进程自己的堆栈和私有数据结构所在pages，这些page被标记为COW
 - ▣ 非共享数据所在page标志为COW，表明不允许子进程修改
 - ▣ 未标识为COW的page，如存放代码的page，允许父进程和子进程共享
- 当子进程在后续运行过程中访问COW页时，表明子进程需要访问自己私有的堆栈或数据结构，但当前访问的是父进程的私有数据结构。此时，将触发COW机制，OS根据父进程的COW页为子进程新创建一个页，用于存放子进程自己的私有数据结构

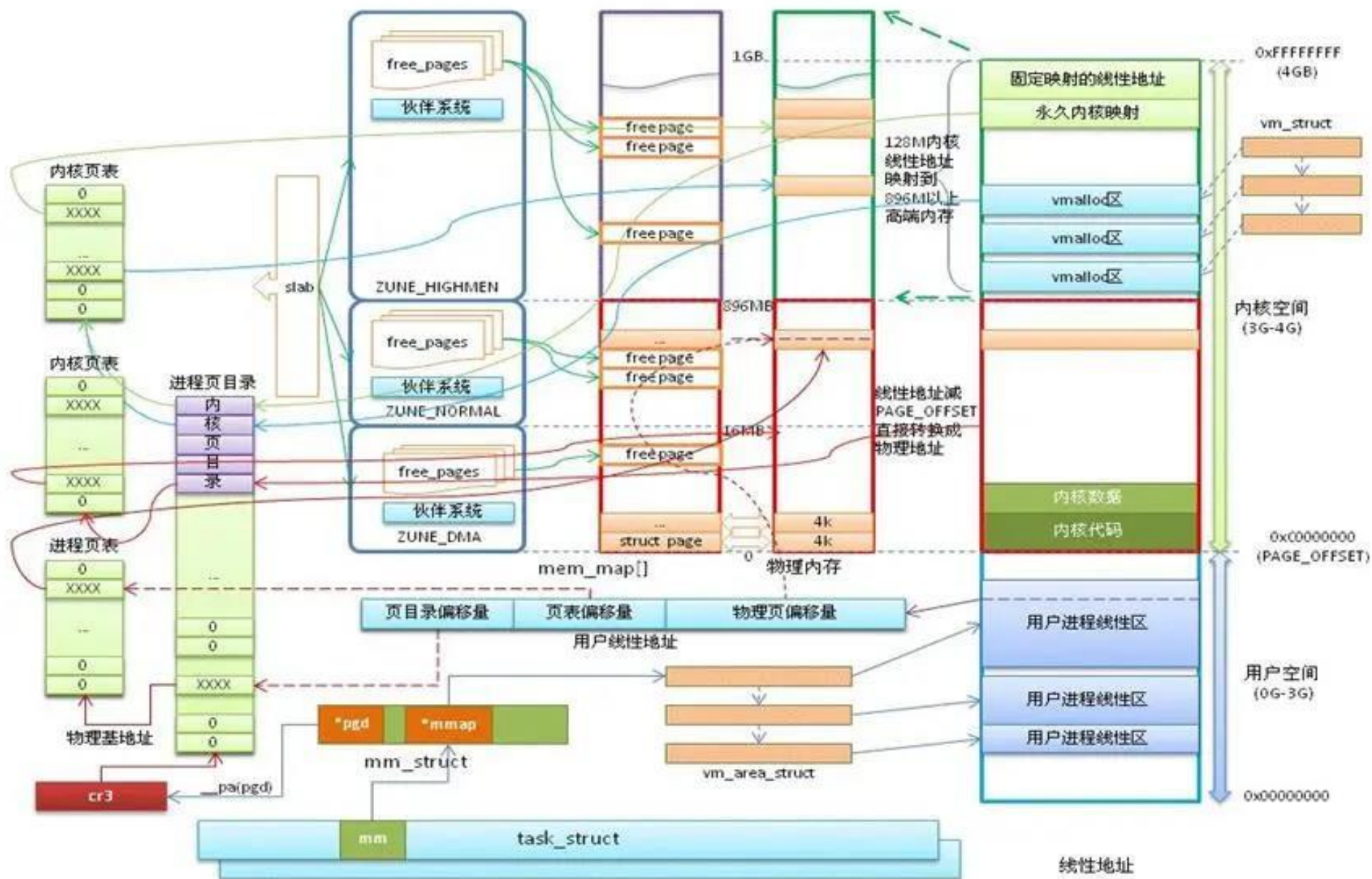
Appendix 9-2 Virtual Address Space in Linux



Appendix 9-1 Memory Management in Linux



Appendix 9-1 Memory Management in Linux



Virtual Address Space in Linux

名称	存储内容
栈	局部变量、函数参数、返回地址等
堆	动态分配的内存
BSS段	未初始化或初值为0的全局变量和静态局部变量
数据段	已初始化且初值非0的全局变量和静态局部变量
代码段	可执行代码、字符串面值、只读变量

- 在将应用程序加载到[内存空间](#)执行时
 - ▣ 操作系统负责[代码段](#)、数据段和BSS段的加载，并在内存中为这些段分配空间
 - BSS段、数据段和代码段在程序编译时确定
 - ▣ 栈stack，由操作系统分配和管理
 - ▣ 堆heap，由程序自己管理，显式地申请和释放空间
- BSS段、数据段和代码段是可执行程序编译时的分段，运行时还需要栈和堆
- `execve(2)`负责为进程代码段和数据段建立映射，真正将代码段和数据段的内容读入内存是由系统的[缺页异常](#)处理程序按需完成的。另外，`execve(2)`还会将BSS段清零

Virtual Address Space in Linux

- 1. 代码段(text)
 - ▣ 程序可执行代码, 字符串字面值, 只读变量, 一般为只读区域
 - ▣ 代码段指令由操作码和操作对象(或对象地址引用)组成。如果操作对象是立即数(具体数值), 将直接包含在代码中; 若是局部数据, 将在栈区分配空间, 然后引用该数据地址; 若位于BSS段和数据段, 同样引用该数据地址
- 2. 数据段(Data)
 - ▣ 程序中已初始化且初值不为0的全局变量和静态局部变量
 - ▣ 属于静态内存分配(静态存储区), 可读可写
- 3. BSS(Block Started by Symbol)段
 - ▣ 未初始化的全局变量和静态局部变量
 - ▣ 初始值为0的全局变量和静态局部变量(依赖于编译器实现)
 - ▣ 未定义且初值不为0的符号(该初值即common block的大小)

Virtual Address Space in Linux

- 程序运行时，数据段和BSS段的整个区段通常称为数据区，数据段与BSS段的区别如下：
 - ▣ 1) BSS段不占用物理文件尺寸，但占用内存空间；数据段占用物理文件，也占用内存空间
 - E.g. 对于大型数组如`int ar0[10000] = {1, 2, 3, ...}`和`int ar1[10000]`，`ar1`放在BSS段，只记录共有`10000*4`个字节需要初始化为0，而不是像`ar0`那样记录每个数据1、2、3...，此时BSS为目标文件所节省的磁盘空间相当可观
 - ▣ 2) 当程序读取不在内存的数据段数据时，系统产生page fault，为数据段数据分配相应的物理内存；当程序读取BSS段的数据时，内核会将其转到一个全零页面，不会发生page fault，也不会为其分配相应的物理内存

■ 4.栈stack

- 存储局部变量、函数参数、返回地址等，用于
 - 为函数内部声明的非静态局部变量(C语言中称“自动变量”)提供存储空间
 - 记录函数调用过程相关的维护性信息，称为栈帧(Stack Frame)或过程活动记录(Procedure Activation Record)，包括：函数返回地址，不适合装入寄存器的函数参数及一些寄存器值的保存

除递归调用外，堆栈并非必需。因为编译时可获知局部变量，参数和返回地址所需空间，并将其分配于BSS段

 - 临时存储区，用于暂存长算术表达式部分计算结果或alloca()函数分配的栈内内存
- 每个进程/线程都有属于自己的栈，栈的内容随着线程执行动态变化（增长，收缩）
- 栈有最大值限制RLIMIT_STACK(通常是8M)，使用Linux命令ulimit -s可查看和设置堆栈最大值

■ 5.堆heap

- 存放进程运行时动态分配的内存段，由程序自己控制，可动态扩张或缩减
 - 调用`malloc(C)/new(C++)`等分配内存时，新分配的内存动态添加到堆上(扩张)
 - 调用`free(C)/delete(C++)`等释放内存时，被释放的内存从堆中剔除(缩减)
- 堆管理器通过链表管理程序申请的内存，由于堆申请和释放是无序的，最终会产生内存碎片。堆内存一般由应用程序分配释放，回收的内存可供重新使用。若程序不释放，程序结束时操作系统可能会自动回收
- 堆中内容是匿名的，无法按名字直接访问，只能通过指针间接访问
- 堆的末端由`break`指针标识，当堆管理器需要更多内存时，可通过系统调用`brk()`和`sbrk()`来移动`break`指针以扩张堆，一般由系统自动调用
- 堆使用时经常出现两种问题：
 - 内存破坏：释放或改写仍在使用的内存
 - 内存泄漏：未释放不再使用的内存。释放次数少于申请次数，可能造成内存泄漏

■ 6. 内存映射段(mmap)

- 将硬盘文件的内容直接映射到内存—内存映射文件
- 应用程序使用Windows `CreateFileMapping()/MapViewOfFile()`或Linux的`mmap()`系统调用建立这种映射
- 内存映射是一种方便高效的文件I/O方式，被用于装载动态共享库。内存映射段用于映射可执行文件用到的动态链接库
- 用户也可创建匿名内存映射，该映射没有对应的文件，用于存放程序数据
 - 在Linux中，通过`malloc()`请求一大块内存，C运行库将创建一个匿名内存映射，而不使用堆内存。“大块”意味着比阈值 `MMAP_THRESHOLD` 还大，缺省为128KB，可通过`mallopt()`调整

Virtual Address Space in Linux

- 7. 保留区
 - ▣ 位于虚拟地址空间的最低部分，未赋予物理地址
 - ▣ 对它保留区的引用都是非法，用于捕捉使用空指针和小整型值指针引用内存的异常情况
- 参考文献
 - ▣ Linux虚拟地址空间布局，知乎
- 实验：观察进程虚拟地址空间内的变量、数据分布



LinuxÄÚ'æ¹ÜÀíÊµÑé-½ø³ÌDéÄâ

实验：观察进程虚拟地址空间内的变量、数据分布

C语言程序实例分析如下所示：

```
#include<stdio.h>
#include <malloc.h>

void print(char *,int);
int main()
{
    char *s1 = "abcde"; // "abcde"作为字符串常量存储在常量区 s1、s2、s5拥有相同
    char *s2 = "abcde";
    char s3[] = "abcd";
    long int *s4[100];
    char *s5 = "abcde";
    int a = 5;
    int b =6;//a,b在栈上，&a>&b地址反向增长

    printf("variables address in main function: s1=%p s2=%p s3=%p s4=%p s5=%p\n",
           s1,s2,s3,s4,s5,&a,&b);
    printf("variables address in processcall:n");
    print("ddddddddd",5);//参数入栈从右至左进行,p先进栈,str后进 &p>&str
    printf("main=%p print=%p \n",main,print);
    //打印代码段中主函数和子函数的地址，编译时先编译的地址低，后编译的地址高main<pr
}
```



LinuxÄÚ'æ¹ÜÀíÊµÑé-½ø³ÌÐéÄâ

C语言程序实例分析

```
void print(char *str,int p)
{
    char *s1 = "abcde"; //abcde在常量区, s1在栈上
    char *s2 = "abcde"; //abcde在常量区, s2在栈上 s2-s1=6可能等于0, 编译器优化了
    //而&s1>&s2
    char s3[] = "abcdeee"; //abcdeeee在常量区, s3在栈上, 数组保存的内容为abcdeeee的
    long int *s4[100];
    char *s5 = "abcde";
    int a = 5;
    int b = 6;
    int c;
    int d; //a,b,c,d均在栈上, &a>&b>&c>&d地址反向增长
    char *q=str;
    int m=p;
    char *r=(char *)malloc(1);
    char *w=(char *)malloc(1) ; // r<w 堆正向增长

    printf("s1=%p s2=%p s3=%p s4=%p s5=%p a=%p b=%p c=%p d=%p str=%p q=%p p=%p\n",
           s1,s2,s3,s4,s5,&a,&b,&c,&d,&str,q,&p,&m,r,w);
    /* 栈和堆是在程序运行时候动态分配的, 局部变量均在栈上分配。
       栈是反向增长的, 地址递减; malloc等分配的内存空间在堆空间。堆是正向增长的
       r,w变量在栈上(则&r>&w), r,w所指内容在堆中(即r<w)。*/
}
```

```
[root@iZuf608nsghhv3odyer887Z test]# ./a
variables address in main function: s1=0x400800 s2=0x400800 s3=0x7ffc78ad4c40 s4=0x7ffc78ad4920 s5=0x400800 a=0x7ffc78ad491c b=0x7ffc78ad4918
variables address in process call: ns1=0x400800 s2=0x400800 s3=0x7ffc78ad48b0 s4=0x7ffc78ad4590 s5=0x400800 a=0x7ffc78ad458c b=0x7ffc78ad4588 c=0x7ffc78ad4584 d=0x7ffc78ad4580 str=0x7ffc78ad4568 q=0x40087b p=0x7ffc78ad4564 m=0x7ffc78ad457c r=0x11f9010 w=0x11f9030
main=0x40057d print=0x40063e
[root@iZuf608nsghhv3odyer887Z test]#
```



Thanks for your
attention



北京邮电大学